

C++ PROGRAMMING

DR. YIHSIANG LIOW (MAY 20, 2026)

Contents

-1.1	if statements	debug: 1-07-if-part1.tex	4
-1.2	Mental picture: flow of execution for if	debug: 2-07-if-part1.tex	10
-1.3	Gotchas	debug: 3-07-if-part1.tex	12
-1.4	Multiplication game: Part 1	debug: 4-07-if-part1.tex	13
-1.5	Blocks	debug: 5-07-if-part1.tex	14
-1.6	Random integers	debug: 6-07-if-part1.tex	16
-1.7	The important swap “trick”	debug: 7-07-if-part1.tex	20
-1.8	First sorting example	debug: 8-07-if-part1.tex	21

07.if and if-else: Part 1

Objectives

- Write if statements (including blocks)
- Write if-else statements (including blocks)
- Write nested if-else statements

We will now talk about writing statements that make decisions. In other words, statements that decide whether to run other statements or not.

This is significant . . . we are now embarking on writing program with intelligence . . .

ONWARD!!!

-1.1 `if` statements debug: 1-07-if-part1.tex

So far you know that C++ can do arithmetic, print integer values, print string values, accept integer values from the keyboards, create and update variables.

You should expect more! (Otherwise C++ is pretty dumb.)

For instance you would expect programs to make decisions. Now wouldn't it be nice if C++ can do this:

```
if alien is killed:
    score = score + 10
```

or

```
if my ship' s energy is 0:
    end the game
```

or

```
if plane' s left engine has failed and
plane' s right engine has failed:
    activate seat ejection!!!
```

Note that for the first case, if the alien is not killed, you do *not* want to execute the statement `score = score + 10`. Otherwise everyone would keep getting the same score! What kind of dumb game is that?

In fact C++ *does* understand the `if` statement.

Run this program

```
int x = 0;
std::cin >> x;
if (x > 0)
    std::cout << "spam!" << std::endl;
std::cout << "eggs" << std::endl;
```

Run the program several time, entering different values for `x`. Note that the statement printing `"spam!"` is controlled by the `if`. However the statement that prints **"eggs"** is not.

For readability, the `if` statement is written like this:

```
int x = 0;
std::cin >> x;
if (x > 0)
    std::cout << "spam!" << std::endl;
std::cout << "eggs" << std::endl;
```

Big picture: The format of the `if` statement looks like this

if ([bool expr])
[stmt]

where `[bool expr]` is a **boolean expression** – this means that it's an expression that can be evaluated to a boolean value – and `[stmt]` is a C++ statement (of course there's a semi-colon at the end). Since we need boolean expressions it's a good time now to recall the following:

<code>!</code>	"not"
<code>&&</code>	"and"
<code> </code>	"or"
<code>==</code>	"is equal to"
<code>!=</code>	"is not equal to"
<code><</code>	"is less than"
<code><=</code>	"is less than or equal to"
<code>></code>	"is greater than"
<code>>=</code>	"is greater than or equal to"

An example of a boolean expression is

[variable] [bool op] [value]

where `[variable]` is a variable of `int` or `double` or `float` type, `[bool op]` is one of the above boolean operators and `[value]` is an `int` or `double` or `float`. Don't forget that if there is a type mismatch, C++ will try to

type promote one value to match the type of the other. For instance if `x` is an `int` variable and the boolean expression is

```
x < 1.23
```

then C++ will use the `double` of `x` (i.e., `x` is automatically promoted to a `double`):

```
double(x) < 1.23
```

Exercise -1.1.1. Modify the above program so that "spam!" is printed when the integer entered is 42. Test your program.

Exercise -1.1.2. Modify the above program so that the program prompts the user for his/her favorite number and print "you don't like 42?!?" if the integer entered is not 42. Here's an execution:

```
what' s your favorite number? 42
```

Here's another:

```
what' s your favorite number? 1
you don't like 42?!?
```

Exercise -1.1.3. Write a program that prompts the user for an integer and prints "That's even. Am I smart?" if the integer entered is even. Here's an execution

```
100
```

```
That' s even. Am I smart?
```

Here's another

```
101
```

Exercise -1.1.4. Write a program that prompts the user for his/her age and prints "You're lying! Think you can fool me???" if the value entered is less than 18. Here's an execution:

```
Enter your age: 16
You' re lying! Think you can fool me???
```

Here's another:

```
Enter your age: textbf100
```

Of course this is another boolean expression:

[var1] [bool op] [var2]

In this case you're comparing two (integer or double) variables.

Exercise -1.1.5. Write a program that prompts the user for two integer values and prints "First is bigger" if the first integer entered is larger than the second. Here's an execution of the program:

```
1 2
```

Here's another

```
2 -1
First is bigger
```

Here's yet another boolean expression:

[expr] [bool op] [value]

where [expr] is an (integer or double) expression.

Exercise -1.1.6. Write a program that prompts the user for his/her height (in ft) and weight (in lbs) and prints "you will live more than 100 years!!!" if the product of the height and weight is at least 200.45. (This is not supported by any form of research.) Here's an execution of the program:

```
6.1 180.180
```

```
you will live more than 100 years!!!!
```

Here's another

```
3.4 23.4
```

Exercise -1.1.7. Write a program that prompts the user for the year of his/her date of birth and the current year and prints his/her approx-

imate age and then prints “you must have watched lots of movies by now” if the difference of the two values is at least 20.

Here’ s an execution:

```
1900 2000
you are about 100 years old
you must have watched lots of movies by now
```

Here’ s another:

```
2000 2006
you are about 6 years old
```

One last one ... here’ s one more boolean expression:

[expr1] [bool op] [expr2]

Now, we’re comparing two expressions.

Exercise -1.1.8. Write a program that prompts the user for the month, day of his/her date of birth and also his/her height (in ft) and weight (in lbs). Print the approximate number of days from January 1 of the year he/she was born and the sum of his/her height and weight. If the first printed number is greater than the second, print “According to the theory of relativity, you are going to win the next powerball.”.

```
Enter the month, day of your DOB: 1 1
Enter your height and weight: 6 100
1 106
```

Here’ s another:

```
Enter the month, day of your DOB: 12 25
Enter your height and weight: 6 200
355 206
According to the theory of relativity,
you are going to win the next powerball.
```

(You may assume the number of days in a month is 30.)

Exercise -1.1.9. Try this:


```
int numHeads = 0;
std::cout << "How many heads do you have? ";
std::cin >> numHeads;
if (numHeads == 2)
    std::cout << "Are you Zaphod?" << std::endl;
```

And then this:

```
int numHeads = 0;
std::cout << "How many heads do you have? ";
std::cin >> numHeads;
if (numHeads != 2)
    std::cout << "You are not Zaphod" << std::endl;
```

Combine both into one program so that I get the following execution:

```
How many heads do you have?1
You are not Zaphod
```

And here's an execution of the same program:

```
How many heads do you have?2
Are you Zaphod?
```

Next modify the program so that, in addition to the above requirements, if the number of heads entered is greater than 2, the program prints "No kidding!". Finally, modify the program so that, in addition to the above, if the number of heads is 1, the program prints "Let me introduce you to our surgeons." Test your program by entering 0, 1, 2, 3.

(There' s a better way to do this exercise ...)

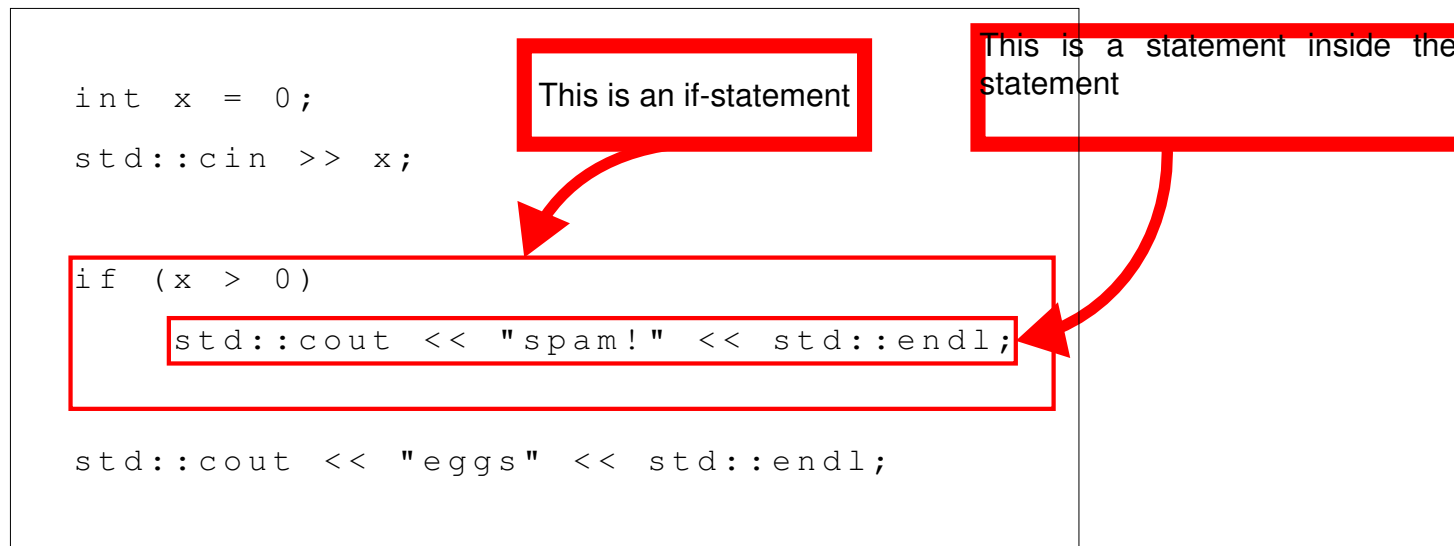
-1.2 Mental picture: flow of execution for `if`

debug: 2-07-if-part1.tex

The `if` statement is difficult for some beginning programmers because it alters the flow of execution of a program.

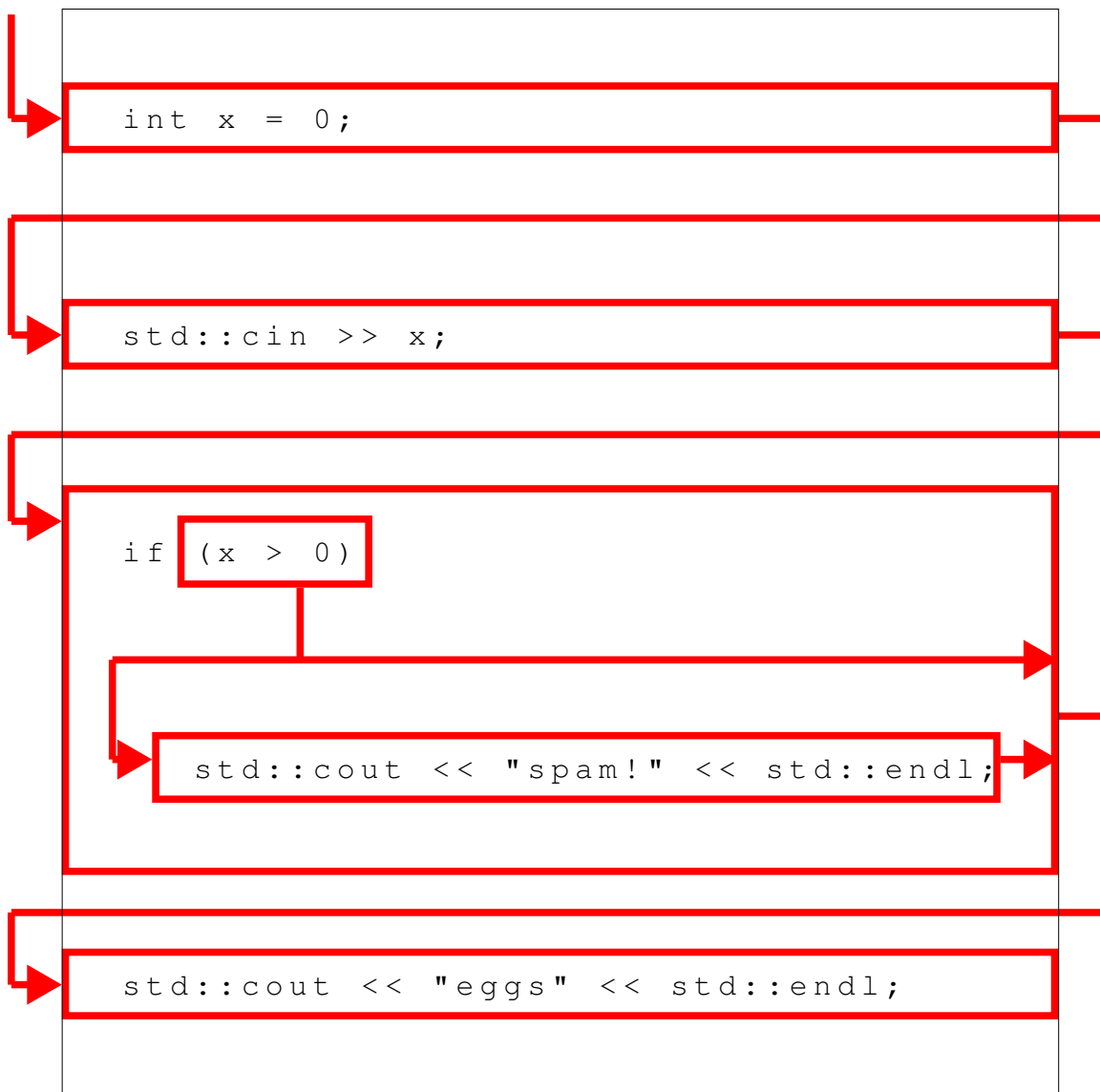
Before this set of notes, C++ executes one statement at a time from top to bottom; it executes every statement.

The `if` statement is different because it has a statement **inside** the `if` statement.



The statement in the `if` statement is executed only when the `x > 0` is true. If the condition is not true, the statement in the `if` statement is not executed.

In the following diagram, you can follow the arrows to get a sense of the flow of execution:



Of course the program is still executing one statement at a time from the top to the bottom if you view the `if` statement as a statement (i.e. forget about the internals of the `if` statement).

-1.3 Gotchas debug: 3-07-if-part1.tex

Exercise -1.3.1. Try this:

```
int x = 0;
std::cin >> x;
if x > 0
    std::cout << "spam!" << std::endl;
std::cout << "eggs" << std::endl;
```

Fix it.

Exercise -1.3.2. Try this:

```
int x = 0;
std::cin >> x;
if (x > 0);
    std::cout << "spam!" << std::endl;
std::cout << "eggs" << std::endl;
```

What do you learn from this example?

Exercise -1.3.3. Here's another wacky example ...

```
int x = 0;
std::cin >> x;
if (x > 0);
std::cout << "eggs" << std::endl;
```

-1.4 Multiplication game: Part 1

debug: 4-07-if-part1.tex

Exercise -1.4.1. Write a program that tests if the user can do two digit multiplication. Right now we will just ask the one question: the product of 97 and 94. The program congratulates the user if the right answer is entered. Otherwise the program stops immediately. Here's an execution of the program:

```
What is the product of 97 and 94? 1
```

Here's another execution of the program:

```
What is the product of 97 and 94? 9118
You smart dawg!
```

This is kind of a dumb program because it asks the same question again and again.

Here's the **pseudocode** (pseudocode = informal description of steps in a program):

```
Declare integer variable guess
Print a prompt string to user
Prompt user for integer value for guess
If guess is 9118
    print congratulatory message
```

Note that this is not a C++ program yet!!! You can't run it!!!

Complete the program using the above pseudocode. Test your program.

-1.5 Blocks debug: 5-07-if-part1.tex

What if you want to execute **several** statements when a boolean expression is true? Here's one way:

```
int x = 0;
std::cin >> x;
if (x > 0)
    std::cout << "spam!" << std::endl;
if (x > 0)
    std::cout << "ham!" << std::endl;
std::cout << "eggs" << std::endl;
```

It does work ...but ...that's the **BAD** way of executing two statements when a condition holds.

This is the right way:

```
int x = 0;
std::cin >> x;
if (x > 0)
{
    std::cout << "spam!" << std::endl;
    std::cout << "ham!" << std::endl;
}
std::cout << "eggs" << std::endl;
```

The **{...}** is called a **block** or **block of statements**. (You've actually seen this already. Look at your `main()` ...) You can have as many statements as you like in a block, not just two.

Exercise -1.5.1. Here's part of a game that you're writing ... complete it. If the missile hits the alien, increment the `score` by 100, set `play_explosion` to `true`, increment your `energy` by the alien's energy (i.e. `alien_energy`), and set the alien's energy to 0. Test your code with different inputs.

```
int score = 1234;
int energy = 6789;
int alien_energy = 42;
bool play_explosion = false;
bool missile_hit_alien;
std::cin >> missile_hit_alien;

std::cout << "score: " << score << '\n'
          << "energy: " << energy << '\n'
          << "alien_energy: " << alien_energy << '\n'
          << "play_explosion: " << score << '\n';
```

-1.6 Random integers debug: 6-07-if-part1.tex

Generating a random integer occurs commonly in computer programs. For instance if you want to write a strategy role-playing game, you might have different rooms in a mansion and you want to put different things in different rooms. Suppose you have a magic potion in a bottle. Now suppose the rooms are numbered from 1 to 1000. You don't want your magic potion bottle to be in room 5 whenever you start the game. You want each game to be different.

At a more complex level you might want to randomly make doors between two rooms. This creates a random maze.

So you'd better learn how to generate random things.

Try this:

```
#include <iostream>

int main()
{
    std::cout << "the potion is in room " << rand()
               << std::endl;
    std::cout << "the potion is in room " << rand()
               << std::endl;
    std::cout << "the potion is in room " << rand()
               << std::endl;
    std::cout << "total number of rooms: "
               << RAND_MAX << std::endl;

    return 0;
}
```

(Note: You might need `#include <cstdlib>` just after `#include <iostream>`. This depends on your compiler.)

Looks like you're getting random integers. But wait ... run the program a second time ... a third time ... a fourth time. Notice something?

OK. Let's modify the program:


```
#include <iostream>
#include <ctime>
int main()
{
    srand((unsigned int) time(NULL));
    std::cout << "the potion is in room " << rand()
               << std::endl;
    std::cout << "the potion is in room " << rand()
               << std::endl;
    std::cout << "the potion is in room " << rand()
               << std::endl;
    std::cout << "total number of rooms: "
               << RAND_MAX << std::endl;
    return 0;
}
```

Run this program several times. Notice that the room number is different each time you run the program.

I will not go into details on the statement

```
srand((unsigned int) time(NULL));
```

The only thing I will say is that it “seeds” the random generator so as to make it “more random”. The important thing to remember is that you must execute this statement and furthermore you execute it **once** (usually) and **before** you call the `rand()` function. The standard practice is to execute the seeding in `main()` and at the beginning:

```
#include <iostream>
#include <ctime>
int main()
{
    srand((unsigned int) time(NULL));
    . . .
    return 0;
}
```

Since `rand()` gives you a random number from 0 to `RAND_MAX`,

$$\text{rand()} / \text{RAND_MAX}$$

will give you a random `double` between 0.0 and 1.0. But this is not right because the `/` is an **integer** division. Therefore we do this:

$$\text{double}(\text{rand}()) / \text{RAND_MAX}$$

to get a random double from 0.0 to 1.0.

Exercise -1.6.1. Check that I was not lying by printing 10 random doubles from 1.0 to 0.0. By the way, if the numbers are truly random, you would expect their average to be close to 0.5. Check that too.

There are times when you need to generate a random *integer* between two bounds. For instance if you have a maze game and the rooms are numbered 1 to 100, you want a random room number for a magic potion. How many values are there from 1 to 100? You have 100. Since `rand()` gives you a random integer from 0 to `RAND_MAX`, if you do

```
rand() % 100
```

you get 100 numbers, from 0 to 99. You add 1 to it and you get a random integer from 1 to 100. In other words

```
1 + rand() % 100
```

does the trick.

Exercise -1.6.2. How do you generate random integers from 1,..., 6? (For instance to simulate a dice roll.) Verify with a C++ program.

Exercise -1.6.3. How do you generate random integers from 1,..., 10? Verify with a C++ program.

What about generating random integers from 5,..., 20? In this case the the lower bound is 5 and not 1. You use the same idea. Note that if you do something like

```
rand() % 10
```

or

```
rand() % 18
```

the range of numbers starts from 0. So if you want to generate random numbers in the range of 5,..., 20, the first thing you do is to think about the range

0, ..., 15

first. In other words move the given range 5, ..., 20 so that the lower

bound of the range becomes 0. That means subtracting 5 from the range. To generate a number in the range of 0, ..., 15, you do

```
rand() % 16
```

This gives a number in the range 0, ..., 15. You then add 5 to it:

```
5 + rand() % 10
```

Exercise. Verify that the expression

```
5 + rand() % 10
```

does give you numbers in the range of 5, ..., 20. .

Exercise -1.6.4. How do you generate random integers from 9,...142?
Verify with a C++ program.

Exercise -1.6.5. How do you generate random integers from -5,...,5?
Verify with a C++ program.

Exercise -1.6.6. How do you generate random `double` from 0,...,2.0?
Verify with a C++ program.

Exercise -1.6.7. How do you generate random double from -1.0,...,1.0?
Verify with a C++ program.

Exercise -1.6.8. How do you generate random double from 0,...,3.5?
Verify with a C++ program.

Exercise -1.6.9. How do you generate random double from -2.0,...,2.0?
Verify with a C++ program.

-1.7 The important swap “trick”

debug: 7-07-if-part1.tex

Recall the following extremely important “trick” that you must know. It allows you to swap the values in two variables:

```
int a = 42, b = 24;
std::cout << a << " " << b << std::endl;
int t = a;
a = b;
b = t;
std::cout << a << " " << b << std::endl;
```

Exercise -1.7.1. Write a program that declares two doubles, initializing them with 1.234 and 4.567. Print the values of the variables. Swap their values. Print the values of the variables.

-1.8 First sorting example debug: 8-07-if-part1.tex

Here's your first sorting problem:

- Prompt the user for two integer values and assign them to x and y.
- Sort the values so that the value of x is less than or equal to the value of y

Exercise -1.8.1. Write the above program. Here are some test cases:

Test 1

```
1 2
1 2
```

Test 2

```
2 1
1 2
```

Test 3

```
-1 5
-1 5
```

Test 4

```
5 -1
-1 5
```

Test 5

```
2 2
2 2
```

08. `if` and `if-else`: Part 2

This is a continuation of “if and if-else: Part 1”.

Enough said. Move on!!!

if-else statements

Run this twice by entering 42 and then 24 for input:

```
int a = 0;
std::cout << "C++: What's your favorite number?"
          << std::endl << "User: ";
std::cin >> a;

if (a == 42)
    std::cout << "C++: Mine too!" <<std::endl;
if (a != 42)
    std::cout << "C++: boo!" << std::endl;
```

You want to either do one thing or another based on whether a boolean condition is true or false (respectively). The above does work ... *but* it is the **WRONG** way of doing it.

Now change the above to this:

```
int a = 0;
std::cout << "C++: What's your favorite number?"
          << std::endl << "User: ";
std::cin >> a;

if (a == 42)
    std::cout << "C++: Mine too!" << std::endl;
else
    std::cout << "C++: boo!" << std::endl;
```

You should **NEVER** have a program like the earlier one with two if's with opposite conditions next to each other:

```
if (a == 42)
    std::cout << "C++: Mine too!" <<std::endl;
if (a != 42)          // BAD!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
    std::cout << "C++: boo!" << std::endl;
```

In other words if you can rewrite a bunch of ifs with if-else, then you should. Why? Because this avoids evaluating boolean expressions unnecessarily and hence speed up your program. (There are other reasons).

The **second** way (the if-else) is the **right** way:

```
if (a == 42)
    std::cout << "C++: Mine too!" << std::endl;
else
    std::cout << "C++: boo!" << std::endl;
```

The format of the if-else statement is

```
if ([bool expr])
    [stmt1]
else
    [stmt2]
```

If *[bool expr]* evaluates to true, *[stmt1]* executes. When that's completed, the else part is bypassed. If *[bool expr]* evaluates to false *[stmt2]* is executed.

Obviously you can't write an else statement without an if!!!

Exercise -1.8.2. Here's the first if-else warm up exercise. No sweat required to finish it. First try running this program so that you know what it does:

```
int numHeads = 0;
std::cout << "How many heads do you have? ";
std::cin >> numHeads;
if (numHeads == 2)
    std::cout << "Are you Zaphod?" << std::endl;
if (numHeads != 2)
    std::cout << "You are not Zaphod" << std::endl;
```

Next modify it to use if-else instead of two if's. Run the program and test it. Here are some tests:

```
How many heads do you have? 1
You are not Zaphod
```

And here's an execution of the same program:

```
How many heads do you have? 2
Are you Zaphod?
```


Exercise -1.8.3. Run the following program:

```
std::cout << "How much do you have? ";  
double savings = 0.0;  
std::cin >> savings;  
  
if (savings > 100)  
    std::cout << "Can I borrow a ten?" << std::endl;  
if (savings <= 100)  
    std::cout << "Life is tough" << std::endl;
```

Rewrite the program using if-else.

Exercise -1.8.4. Write a program that prompts the user for an integer and tells you if the input is even or odd. Here's an execution:

```
gimme an int 42  
even!!!
```

Here's another:

```
gimme an int 35  
odd!!!
```

Exercise -1.8.5. No skeleton code here. This is a slight modification of a previous program. Write a program that prompts the user for his/her height (in ft) and weight (in lbs) and prints “you will live more than 100 years!!!” if the product of the height and weight is at least 200.45. (This is not supported by any form of research.) Otherwise it prints “you will not live more than 100 years”. Here's an execution of the program:

```
6.1 180.180  
you will live more than 100 years!!!
```

Here's another

```
3.4 23.4  
you will live more than 100 years!!!
```

Exercise.

Exercise -1.8.6. Write a program that prompts the user for his employee code which is either 0 or 1. If the employee code is 0, his/her hourly rate is \$123.45. If the employee code is 1, his/her hourly rate is \$7.99. Prompt the user for the number of hours he/she worked this week. Finally print the total amount he/she made this week. Here's an informal description of the program that you should follow (obviously you have to translate this to C++!!!):

```
Declare constant HOURLY_RATE_0 and initialize it to 123.45.
Declare constant HOURLY_RATE_1 and initialize it to 7.99.

Declare variable employee_code.
Get employee_code from user.

Declare variable hourly_rate.
If employee_code is 0:
    set hourly_rate to HOURLY_RATE_0
Otherwise:
    set hourly_rate to HOURLY_RATE_1

Get number_of_hours from user.
Print total pay for this week.
```

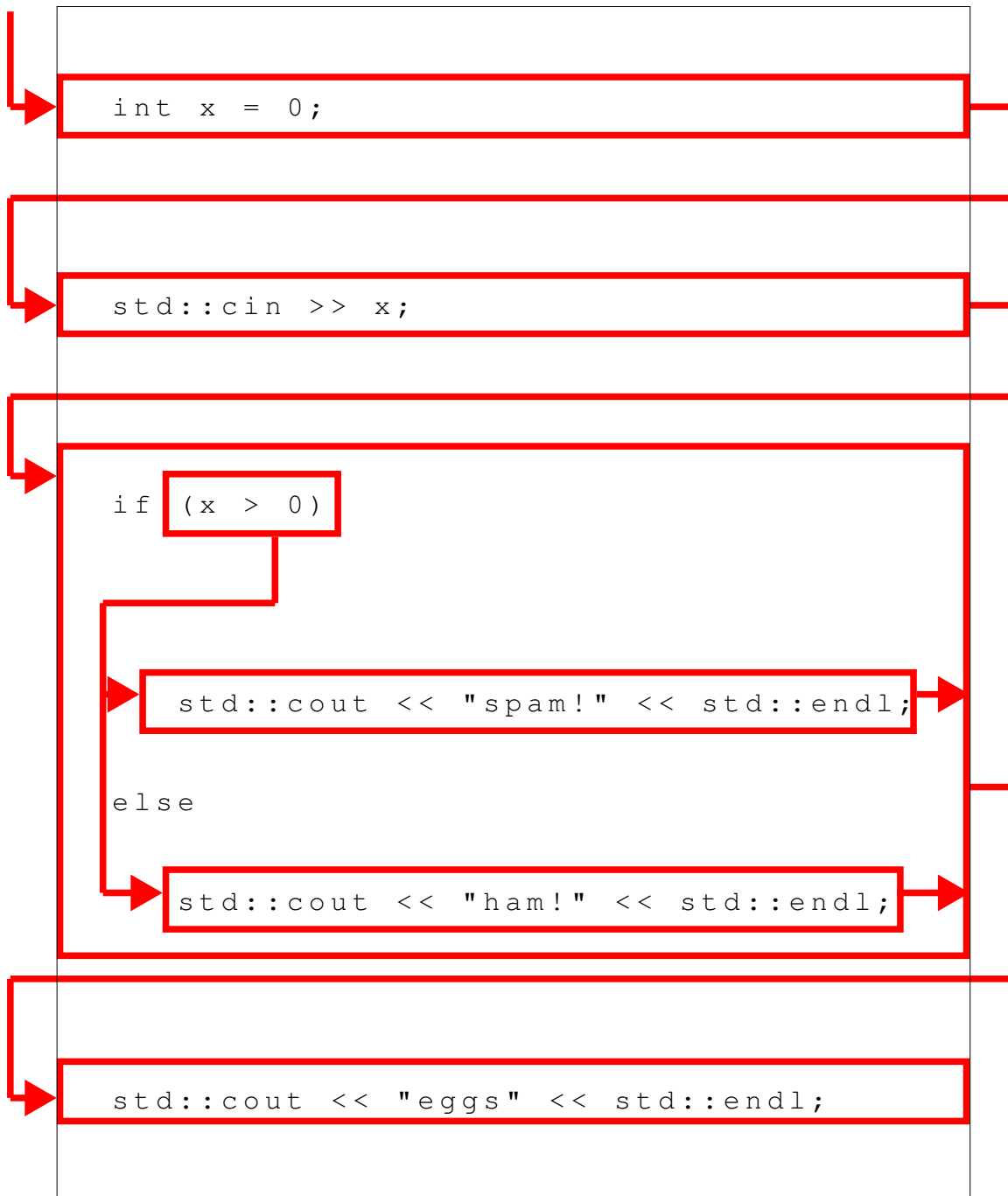
Mental picture: flow of execution for `if-else`

The following picture shows you the flow of execution of a simple program containing the `if-else` statement.

```
int x = 0;
std::cin >> x;

if (x > 0)
    std::cout << "spam!" << std::endl;
else
    std::cout << "ham!" << std::endl;

std::cout << "eggs" << std::endl;
```



Multiplication game: Part 2

Recall our multiplication game that tests if the user can multiply 97 and 94. Here's an execution of the program:

What is the product of 97 and 94? **1**

Here's another:

What is the product of 97 and 94? **9118**
You smart dawg!

Here's the pseudocode:

```
Declare integer variable guess
Print a prompt string to user
Prompt user for integer value for guess
If guess is 9118
    print congratulatory message
```

Now improve the program so that it does the following:

```
Declare integer variable guess
Print a prompt string to user
Prompt user for integer value for guess
If guess is 9118
    print congratulatory message
Otherwise
    print the correct answer
```

Here's an execution:

What is the product of 97 and 94? **1**
Nope ... it's 9118.

Here's another:

What is the product of 97 and 94? **9118**
You smart dawg!

Multiplication game: Part 3

We're going to sell our multiplication game. But no one is going to buy it if all it does is to ask the same question again and again!

Modify your program so that the program prompts the user for the product of two integers between 90 and 99. The integers should be randomly generated.

The pseudocode becomes:

```
Generate two random integers between 90 and 99
    and give the values to variables x and y
Print a prompt string to user to guess the
    product of x and y
```

```
Declare integer variable guess
Prompt user for integer value for guess
```

```
If guess is the product of x and y
    print congratulatory message
Otherwise
    print the product of x and y
```

Note the value of the product of x and y occurs twice. We can com-

pute the product just once and store it in a variable:

```
Generate two random integers between 90 and 99
    and give the values to variables x and y
Print a prompt string to user to guess the
    product of x and y
```

```
Declare integer variable guess
Prompt user for integer value for guess
```

```
If guess is answer
    print congratulatory message
Otherwise
    print answer
```

To test your program you need to know the answer. So just get the program to print the answer temporarily before prompting the user. You can get rid of revealing the answer when you have fully tested your code.

Here's an execution of the program:

```
What is the product of 91 and 94? (answer: 8554) 1
Nope ... it's 8554
```

Here's another execution of the same program:

```
What is the product of 96 and 94? (answer: 9024) 9024
You smart dawg!
```

Of course after it works you get rid of the (answer: ...) output and the execution becomes:

```
What is the product of 91 and 94? (answer: 8554) 1
Nope ... it's 8554
```

```
What is the product of 96 and 94? (answer: 9024) 9024
You smart dawg!
```

if-else blocks

It should not be too surprising that you can execute blocks for if-else. You can have a block controlled by the if-part, or a block controlled by the else-part, or both.

```
std::cout << "Head Surgery Cost Planner \n"
           << "First 3: \ $100 each. \n"
           << "After that: \ $2 each. \n"
           << "Less than 3: extra 500 charge"
           << std::endl;

int extraHeads = 0;
int cost = 0;
std::cout << "How many extra heads do you want? ";
std::cin >> extraHeads;
if (extraHeads <= 3)
{
    cost = extraHeads * 100 + 500;
    std::cout << "Please go to Cheapskate Room."
              << std::endl;
}
else
{
    cost = 3 * 100 + (extraHeads - 3) * 2;
    std::cout << "Please go to Deluxe Room."
              << std::endl;
}
```

Exercise -1.8.7. Once you see code like the above you immediately ask yourself this question: How many constants do you see in the above program? (I can see four). Rewrite it with constants instead of hard-coding integer values.

Exercise -1.8.8. Write a program that computes the area of either a circle or a right-angle triangle. Recall that you can declare character variables. It turns out that you can compare character values too, using the == operator. The type for the character values is `char`. Here's a code skeleton for you:


```
std::cout << "Area Calculator!!! \n";
std::cout << "I can do circles or triangles! \n"
           << "Option (c or t): ";
char >> option;
std::cin >> option;
if (option == 'c')
{
    // Prompt the user for radius and display the
    // area of the circle with the given radius.
}
else // if it's not 'c', assume it is 't'
{
    // Prompt the user for the base and height of
    // the right-angled triangle and display the
    // area.
}
```

Coding style for `if` and `if-else`

ONE SPACE AFTER `if`: BAD (but practiced by some)

```
if(x < 0)
    foo = 1;
```

GOOD

```
if (x < 0)
    foo = 1;
```

ONE-LINE `if`:

It's OK to write a one-line `if` statement if the body is a short statement and not a block and it improves readability.

OK BUT YUCKY:

```
x = 0;

if (x < 0)
    y = 0;
```

BETTER

```
x = 0;
if (x < 0) y = 0;
```

Maximum/minimum computation

Trace this (very very very important program by hand). Understand what it does. Finally rewrite it using the one-line if format.

```
int w = 1, x = 3, y = 0, z = 5;
int max;
max = w;
if (max < x)
    max = x;
if (max < y)
    max = y;
if (max < z)
    max = z;
```

Note that you can also use the bubblesort algorithm to compute the maximum value in w, x, y, z. However the above maximum computation algorithm does not require swapping values (the values in w, x, y, z are not changed) and therefore it uses less computation.

Exercise -1.8.9. Write a program that prompts the user for 6 integer values, prints the 6 values, and then prints the maximum of the 6 values.

Exercise -1.8.10. Refer to the previous exercise. What happens when you change all the < to the > operator? (This is also very very important).

Coding style for blocks

ALIGN BLOCK DELIMITERS TO HEADER:

BAD (but used by some and is common in Java):

```
int main() {  
    return 0;  
}
```

and

```
if (x < 0) {  
    foo = 42;  
    bar = 43;  
}
```

HORRIFYING:

```
if (x < 0)  
{  
    foo = 42;  
    bar = 43;  
}
```

ABOMINABLE:

```
if (x < 0)  
    {  
        foo = 42;  
        bar = 43;  
    }
```

GOOD:

```
int main()  
{  
    return 0;  
}
```

and

```
if (x < 0)  
{  
    foo = 42;  
    bar = 43;  
}
```

The same applies to if-else:

BAD

```
if (x < 0) {  
    foo = 42;  
    bar = 43;  
}  
else {  
    foo = 43;  
    bar = 42;  
}
```

GOOD

```
if (x < 0)  
{  
    foo = 42;  
    bar = 43;  
}  
else  
{  
    foo = 43;  
    bar = 42;  
}
```

ALWAYS USE BLOCKS IF NOT ONE-LINERS:

C/C++ programmers tend to use blocks even when the block has only one statement except when using the one-line if style. This creates a unified look and prevents difficult-to-find bugs caused by “dangling else” (see later).

OK BUT NOT COMMON:

```
if (x < 0)  
    foo = 42;  
else  
    foo = 24;
```

COMMON:

```
if (x < 0)
{
    foo = 42;
}
else
{
    foo = 42;
}
```

Boolean expressions again

We started with boolean expressions such as

$$x > 0$$

and went on to expressions such as

$$x > y$$

and then to

$$x * x + y * y! = 0$$

and finally to

$$a * a + b * b * c * c + d * d$$

i.e.

$$[\text{expr1}] \quad [\text{boolop}] \quad [\text{expr2}]$$

where `[expr1]` and `[expr2]` are boolean expressions and `[boolop]` is a binary boolean operator accepting two numeric values. Let's recall a few more binary operators.

First of all let's not forget that we also have the unary `!` operator. The following prints `x` if **it is not true that `x` is 0**.

```
int x = 0;
std::cin >> x;
if (! (x == 0))
    std::cout << x << std::endl;
```

Exercise -1.8.11. Rewrite the above program so that the `!` operator is not used. Your resulting program therefore has to evaluate one boolean operator and not two.

Exercise -1.8.12. Simplify the boolean expression in the program so that the program behaves the same after your modification:

```
int x = 0;
std::cin >> x;
if (!(x <= 42) || !(x >= 42))
    std::cout << x << std::endl;
```

But note you should not simplify expressions (boolean or numeric) if by doing so it makes your program harder to read. Furthermore, modern-day compilers are actually smart enough to simplify it for you automatically!

Of course we also have the boolean operators `&&` and `||`. So we can have the following boolean expression:

$$(x < y) \ \&\& \ (y < z)$$

Note that $(x < y)$ and $(y < z)$ are boolean expressions. So in general we can have

[boolexpr1] [boolop] [boolexpr2]

where [boolexpr1] and [boolexpr2] are boolean expressions and [boolop] is either `&&` or `||`.

This should be a DIY review:

Exercise -1.8.13. Complete the truth table for `&&`:

true && true is ____
true && false is ____
false && true is ____
false && false is ____

Exercise -1.8.14. Complete the truth table for `||`:


```
true || true is ____
true || false is ____
false || true is ____
false || false is ____
```

You should check with your previous notes.

Exercise -1.8.15. You qualify for free head surgery if you have one head, less than three arms, more fingers on your left than you right hand, and finally either you are from Earth or the product of your highest level reached on Pacman and Galaxian is less than 6 (obviously you need an extra head in that case ...) Of course if you have not played Pacman, your high level is 0; likewise for Galaxian. Complete this program:

```
int numHeads = 0;
int numArms = 0;
int numLeftFingers = 0;
int numRightFingers = 0;
bool earthling = true;
int pacManLevel = 0;
int galaxianLevel = 0;

// prompt for values for all variables
// For boolean variable enter 1 for true and 0 for false

if (
)
std::cout << "You qualify for our free head surgery!"
          << std::endl;
else
std::cout << "Try again at newheads.com" << std::endl;
```

You should create some test cases by hand and then test your code against your test cases.

Nested if-else

Now the plot thickens. Since we can put any statement inside an `if` or an `else`, and `if` and `if-else` are themselves statements, that means that we can nest `if` and `if-else` statements.

That's mind-boggling. Let's have an example.

Run this with input 1. Run it again with input 2. Repeat that with 3. Finally try it with -1.

```
int numHeads = 0;
std::cout << "How many heads do you have? " << std::endl;
std::cout << numHeads;

if (numHeads == 1)
{
    std::cout << "Let me introduce you to our surgeons."
              << std::endl;
}
else
{
    if (numHeads == 2)
    {
        std::cout << "Are you Zaphod?" << std::endl;
    }
    else
    {
        std::cout << "Hmm ... neither 1 nor 2 ..."
                  << std::endl;
    }
}
```

Exercise -1.8.16. Draw a picture showing the flow of execution of the above code segment.

Here's another example. Run this with inputs 1, 2, 3, -1.

```
int numHeads = 0;
std::cout << "How many heads do you have? ";
std::cin >> numHeads;
if (numHeads < 1)
{
    std::cout << "Huh?" << std::endl;
}
else
{
    if (numHeads == 1)
    {
        std::cout << "Let me introduce you to our "
                    << "surgeons." << std::endl;
    }
    else
    {
        if (numHeads == 2)
        {
            std::cout << "Are you Zaphod?";
        }
        else
        {
            std::cout << "You have way too many."
                        << std::endl;
        }
    }
}
}
```

Exercise -1.8.17. What is the output when x has value 0.

```
if (x < 5)
{
    std::cout << "1" << std::endl;
}
else
{
    if (x == 0)
    {
        std::cout << "2" << std::endl;
    }
    else
    {
        if (x > 42)
        {
            std::cout << "3" << std::endl;
        }
    }
}
}
```

By the way look at this program:

```
if (score < 100)
{
    std::cout << "lousy gamer" << std::endl;
}
else
{
    if (100 <= score && score < 200)
    {
        std::cout << "not bad" << std::endl;
    }
    else
    {
        if (200 <= score)
        {
            std::cout << "great!" << std::endl;
        }
    }
}
```

Do you see why the above is the same as:

```
if (score < 100)
{
    std::cout << "lousy gamer" << std::endl;
}
else
{
    if (score < 200)
    {
        std::cout << "not bad" << std::endl;
    }
    else
    {
        if (200 <= score)
        {
            std::cout << "great!" << std::endl;
        }
    }
}
```

Exercise -1.8.18. Write a program that prompts the user for his/her age. If the age is less than 10, print “read the hobbit”, if it’s between 11 and 15, print “read lord of the ring”, if it’s 16, print “read hitchhiker’s guide to the galaxy”, and otherwise, print “read chica chica boom boom”.

Exercise -1.8.19. Write a program that converts a numeric grade to a letter grade. The program prompts the user for a double, and if the double is less than 60, it prints 'F'; if it is at least 60 but less than 70, it prints 'D'; if it is at least 70 but less than 80, it prints 'C'; if it is at least 80 but less than 90, it prints 'B'; and if it's at least 90, it prints 'A'.

The dangling else problem

Here's a trick question (and it's another reason why C/C++/Java programmers use `if` even when the block has only one statement). So ...

PAY ATTENTION

First figure out the output (without a compiler) when `x` has a value of 0.

```
if (x < 0)
    if (x == -2) std::cout << "1" << std::endl;
else
    std::cout << "2" << std::endl;
```

Now run the program.

Mamamia ... ?!?

This is called the “dangling else” problem.

What actually happens is that an `else` always follows the closest `if`. This means that the above code is better indented this way:

```
if (x < 0)
    if (x == -2) std::cout << "1" << std::endl;
    else
        std::cout << "2" << std::endl;
```

Of course the C++ compiler doesn't care about whitespaces. However this version of indentation makes it easier for human beings to follow the logic.

Now if what I intended was actually something like the previous version, then I would have to do this:

```
if (x < 0)
{
    if (x == -2) std::cout << "1" << std::endl;
}
else
    std::cout << "2" << std::endl;
```

In this case, the closest `if`-part that the `else`-part can attach to is the `if`-part on the first line since the `else`-part cannot “go inside” the block.

This is the reason why it's a good practice to ALWAYS use blocks even when the if-part or the else-part has only one line to execute. It prevents the misleading dangling else problem. So if I write this:

```
if (x < 0)
{
    if (x == -2)
    {
        std::cout << "1" << std::endl;
    }
}
else
{
    std::cout << "2" << std::endl;
}
```

there will be no confusion.

Coding style for nested if-else

NOT SO GOOD:

```
if (x < 5)
{
    foo = 42;
}
else
{
    if (x < 8)
    {
        foo = 43;
    }
    else
    {
        if (x == 9)
        {
            foo = 44;
        }
        else
        {
            foo = 45;
        }
    }
}
```

GOOD:

```
if (x < 5)
{
    foo = 42;
}
else if (x < 8)
{
    foo = 43;
}
else if (x == 9)
{
    foo = 44;
}
else
{
    foo = 45;
}
```

Exercise -1.8.20. Rewrite your previous program:


```
int numHeads = 0;
std::cout << "How many heads do you have? ";
std::cin >> numHeads;
if (numHeads < 1)
{
    std::cout << "Huh?" << std::endl;
}
else
{
    if (numHeads == 1)
    {
        std::cout << "Let me introduce you to our "
                    << "surgeons." << std::endl;
    }
    else
    {
        if (numHeads == 2)
        {
            std::cout << "Are you Zaphod?";
        }
        else
        {
            std::cout << "You have way too many."
                        << std::endl;
        }
    }
}
```

using the above format.

Multiplication game: Part 4

Here comes one more improvement to our multiplication game. We're going to give the user a second chance. Furthermore we'll give a hint. We will tell him/her is his/her answer is too high or too low. If the user gets the correct answer the first time, we'll give 2 points. If the user gets it right the second time, we'll give 1 point. Otherwise we'll give zero. (What else can you give anyway?)

Here's the pseudocode:

```
Generate two random integers between 90 and 99
    and give the values to variables x and y
Store the product of x and y in variable answer
Print a prompt string to user to guess the
    product of x and y
```

```
Declare integer variable score and initialize it
    to 0
```

```
Declare integer variable guess
Prompt user for integer value for guess
```

```
If guess is answer
    Print congratulatory message
    score = 2
```

```
Otherwise if guess is less than the answer:
    Tell user the guess is too low
    Print a prompt string to user to guess
        the product of x and y
    If guess is answer
        print congratulatory message
        score = 1
    Otherwise
        print answer
```

```
Otherwise if guess is greater than the answer:
    Tell user the guess is too high
    Print a prompt string to user to guess
        the product of x and y
    If guess is answer
        print congratulatory message
        score = 1
    Otherwise
        print answer
```

```
Print user's score
```

Here are several executions to give you a feel for what the program should do:

Case: correct

```
What is the product of 91 and 94? (answer: 8554) 8554  
You smart dawg!  
Score: 2
```

Case: low-low

```
What is the product of 91 and 94? (answer: 8554) 1  
Too low!  
What is the product of 91 and 94? (answer: 8554) 1  
Nope ... it's 8554  
Score: 0
```

Case: low-correct

```
What is the product of 91 and 94? (answer: 8554) 1  
Too low!  
What is the product of 91 and 94? (answer: 8554) 8554  
You smart dawg!  
Score: 1
```

Case: low-high

```
What is the product of 91 and 94? (answer: 8554) 1  
Too low!  
What is the product of 91 and 94? (answer: 8554) 9000  
Nope ... it's 8554  
Score: 0
```

Case: high-low

```
What is the product of 91 and 94? (answer: 8554) 9000  
Too high!  
What is the product of 91 and 94? (answer: 8554) 1  
Nope ... it's 8554  
Score: 0
```

Case: high-correct

```
What is the product of 91 and 94? (answer: 8554) 9000  
Too high!  
What is the product of 91 and 94? (answer: 8554) 8554  
You smart dawg!  
Score: 1
```

Cleanup – refactoring

In writing software, even when you're done, you should look at your code to see if you can improve on it. You do not want to change the behavior of the code. Rather you want to reorganize the code to make it easier to read or to make it more efficient. Such an activity is called **refactoring**.

This is something that all problem-solving processes involve, not just software development. In particular, this occurs a lot in fundamental research as well. For instance mathematicians always rewrite their proofs to improve the readability. Sometimes mathematicians will even derive different proofs of the same result in order to see things in a totally different way. Anyway, let's get back to programming.

You realized that there is a lot of duplicate code. There is a lot of duplication when the program gives the user a second try:

```
Generate two random integers between 90 and 99
    and give the values to variables x and y
Store the product of x and y in variable answer
Print a prompt string to user to guess the
    product of x and y
```

```
Declare integer variable score and initialize it
    to 0
```

```
Declare integer variable guess
Prompt user for integer value for guess
```

```
If guess is answer
    Print congratulatory message
    score = 2
Otherwise if guess is less than the answer:
    Tell user the guess is too low
    Print a prompt string to user to guess
        the product of x and y
    If guess is answer
        print congratulatory message
        score = 1
    Otherwise
        print answer
Otherwise if guess is greater than the answer:
    Tell user the guess is too high
    Print a prompt string to user to guess
        the product of x and y
    If guess is answer
        print congratulatory message
    score = 1
    Otherwise
        print answer
```

```
Print user's score
```

Not everything is similar between the two chunks of code. But you can see that the similarities is around this spot:

```
If guess is answer
    print congratulatory message
    score = 2
Otherwise if guess is less than the answer:
    Tell user the guess is too low
    Print a prompt string to user to guess
        the product of x and y
    If guess is answer
        print congratulatory message
        score = 1
    Otherwise
        print answer
Otherwise if guess is greater than the answer:
    Tell the user the guess is too low
    Print a prompt string to user to guess
        the product of x and y
    If guess is answer
        print congratulatory message
        score = 1
    Otherwise
        print answer
```

We want to write one chunk of code and not two. To combine the chunks (they reside under two different boolean conditions right now) we need to have a common condition for both – and that's when the

first guess is incorrect:

```
If guess is answer
    Print congratulatory message
    score = 2
Otherwise
    If guess is less than the answer:
    Tell user the guess is too low
    Otherwise
        Tell user the guess is too high
    Print a prompt string to user to guess
        the product of x and y
    If guess is answer
        print congratulatory message
        score = 1
    Otherwise
        print answer
Print user's score
```

Now using this pseudocode, clean up your program and see how many lines of code shorter your program gets.

Find computation

Suppose you're a bunch of integer values in variables, say

a, b, c, d

and you're interested in who has a particular value. If the target value is in a, you want to print 0. If it's in b, you want to print 1, etc. In other words, we think of a has having position 0 and if the target value is found in a, we think of the target at position 0. I will save the position in the variable `index`. The code would look like this:

```
if (target == a)
{
    index = 0;
}
else if (target == b)
{
    index = 1;
}
else if (target == c)
{
    index = 2;
}
else if (target == d)
{
    index = 3;
}
std::cout << index << '\n';
```

If the target is not found in a, b, c, d, we will set `index` to -1:

```
if (target == a)
{
    index = 0;
}
else if (target == b)
{
    index = 1;
}
else if (target == c)
{
    index = 2;
}
else if (target == d)
{
    index = 3;
}
else
{
    index = -1;
}
std::cout << index << '\n';
```

Exercise -1.8.21. Complete the program by prompting the user for a, b, c, d and target and print index.

Exercise -1.8.22. Do the same as above but prompt the user for 5 integer values a, b, c, d, e.

Exercise -1.8.23. Note that the above finds the position of a value by searching from the first variable to the last (a to d). What if you search backwards? For instance if the target value is found in both a and d, your program should print 3 and not 0.

Make sure you understand and remember this algorithm.

Count computation

Suppose you have 4 integer variables (say `a`, `b`, `c`, `d`) and you want to count the number of times a value (say in `target`) occurs in these integer variables. The pseudocode would look like this:

```
count = 0
if target == a:
    increment count by 1
if target == b:
    increment count by 1
if target == c:
    increment count by 1
if target == d:
    increment count by 1
print count
```

Exercise -1.8.24. Write a program that prompts the user for 4 integers, stores them in variables, `a`, `b`, `c`, `d`. The program then prompts the user for an integer value and stores it in `target`. Convert the above pseudocode into C++ and print `count`.

Make sure you understand and remember this algorithm.

Typecast between bool and ints

And don't forget that, when needed, C++ will automatically type cast boolean values to integers by casting the value true to 1 and false to 0. C++ will also can also type cast numeric values (ints, doubles, floats) to boolean values by casting nonzero values to true and zero to false:

bool $\Rightarrow$	int
true	1
false	0

numeric (int/double/float) $\Rightarrow$	bool
nonzero (example: 1, 2, 3)	true
zero (example: 0, 0.0)	false

Try this:

```
if (1)
std::cout << "i'm the queen of england"
    << std::endl;
```

Change 1 to 4 and run the program again.

(The above is not a random example. It's important because we're going into loops very soon.)

Exercise -1.8.25. What is the boolean value of 1 && 2? What about 1 && 0? Do it "by hand". Next verify your computation by running this:

```
std::cout << (1 && 2) << std::endl;
std::cout << (1 && 0) << std::endl;
```

I also warn you again that type casting does not change the value of a variable:

```
int x = 42;
std::cout << (x && 1) << std::endl;
std::cout << x << std::endl;
```

(The value of `x` is type cast to `true`. `x` is still 42.)

This is really the same as calling the `sqrt()` (square root) function with a variable. The value of the variable is not changed.

```
double x = 2.0;
std::cout << x << std::endl;
std::cout << sqrt(x) << std::endl;
std::cout << x << std::endl;
```

Assignment based on conditions

Here's a trick to simplify an assignment via an `if-else` statement. Suppose you have this program:

```
bool newMember = true;
double loanRate = 0.0;

if (newMember)
    loanRate = 0.01;
else
    loanRate = 0.20;

// Yeah that's what credit cards companies do ...
```

You can rewrite this as:

```
bool newMember = true;
double loanRate = 0.01;
if (!newMember)
    loanRate = 0.20;
```

Yet another way to do this is:

```
bool newMember = true;
double loanRate = newMember * 0.01
                  + (!newMember) * 0.20;
```

There is however a **problem** with this technique and some disapprove of it. That is, you're using a boolean value (true or false) as a numeric value (1 or 0). Some people believes that automatic type casting like this is dangerous and misleading and that **types should be kept as separate as possible**.

To make such people happy (example: your tech leader / manager might be one) and also not to mislead the reader, you should always **explicitly type cast**. For instance above should be written:

```
bool newMember = true;
double loanRate = int(newMember) * 0.01
                  + int(!newMember) * 0.20;
```

Exercise -1.8.26. Write a program that gives the variable `heartAttackRate`

0.01, 0.02, 0.03, and 0.94 depending on whether `numHeads` is 1, 2, 3 or at least 4 (respectively). Complete this:

```
int numHeads = 0;
std::cin >> numHeads;
double heartAttackRate = _____
std::cout << heartAttackRate << std::endl;}
```


The ternary operator

So far our operators accept one value, for example:

$-y$
 $!b$

or two values, for example:

$x + y$
 $b1 \ \&\& \ b2$

There is a curious operator that accepts **three**. Try this program:

```
int x = 5;
std::cout << (x == 5 ? 42 : 0) << std::endl;
```

Run this with different values for variable x.

Get the point of the `(?:)` operator yet? The format looks like this:

`([bool expr] ? [expr1] : [expr2])`

Basically, if the `[bool expr]` (a boolean expression) evaluates to `true`, C++ will evaluate `[expr1]` and give you its value; otherwise it gives you the value from `[expr2]`.

Here's another example. First try to guess what the program does. Next run it to verify.

```
int x = 0, y = 1, z = 2;

int a = (x + y * z ? x : y + z);
std::cout << a << std::endl;
```

Do you see that it does the same thing as this:

```
int x = 0, y = 1, z = 2;

int a = 0;
if (x + y < z)
{
    a = x;
}
else
{
    a = y + z;
}
std::cout << a << std::endl;
```

Here's another example. Try this die game:

```
int die_roll = 1 + rand() % 6 ;
std::cout << "value of die roll: " <<
    die_roll << '\n';
std::cout << "You "
    << (die_roll == 6 ? "win!" : "lose")
    << '\n';
```

Get it?

Exercise -1.8.27. Write a dice game. Roll two dice. If the dice both give you the same number, print “win”. Otherwise your program prints “lose”. Your program should also print the values of the dice roll. Use the ternary operator.

Exercise -1.8.28. Write a dice-coin game. First, you roll the dice. If the dice values are the same you gain \$1.50 and the game ends. Otherwise you pay \$1. However you have a second chance: you toss a coin. If you get a head (use random integer value of 0) you gain \$1, otherwise (i.e., for a tail), you gain \$0.50. Your program prints all the dice values and the result of the coin toss and finally your gain (which can be positive or negative). Run this program 10 times. What is your average gain? Use the ternary operator whenever possible.

Exercise -1.8.29. Rewrite the following code fragment so that it's cleaner

```
int x, y;
std::cin >> x >> y;
if (x >= 42)
{
    std::cout << x << ' ';
    if (y < 10)
    {
        std::cout << y << ' ';
    }
    else
    {
        std::cout << '-' << ' ';
    }
}
else
{
    std::cout << '*' << ' ';
    if (y < 10)
    {
        std::cout << y << ' ';
    }
    else
    {
        std::cout << '-' << ' ';
    }
}
```

Summary

The if and if-else statement looks like these:

```
if ([bool expr])  
    [stmt]
```

```
if ([bool expr])  
    [stmt1]  
else  
    [stmt2]
```

where `[bool expr]` is a boolean expression and `[stmt]`, `[stmt1]`, and `[stmt2]` are either statements or blocks of statements.

The else statement will always be associated with the closest if statement. In other words

```
if (x < 0)  
    if (x == -2) std::cout << "1" << std::endl;  
else  
    std::cout << "2" << std::endl;
```

is better written this way:

```
if (x < 0)  
    if (x == -2)  
        std::cout << "1" << std::endl;  
    else  
        std::cout << "2" << std::endl;
```

The `rand()` function returns a “random” integer between 0 and `RAND_MAX`. Number. The `srand()` function “seeds” the `rand()` function. You might need to “`#<include cstdlib>`” to use the `srand()` and `rand()` functions. One way to seed `rand()` is to call

```
srand((unsigned) time(0))
```

in which case you need to “`#<include ctime>`”.

```
double(rand()) / RAND_MAX
```

will generate “random” doubles between 0 and 1. To generate a random integer between a and b (two integers with a ≤ b), you can use

```
int (double (rand()) / RAND_MAX * (b - a)) + a
```

or

```
rand() % (b - a + 1) + a
```

(The second method is not as general nor as random as the first. For instance you cannot use it to generate random number between 0 and 1000000 in MS .NET STUDIO).

The following code segment swaps the values in a and b (and t has the value of a):

```
t = a;      a = b;      b = t;
```

Using the “swapping trick”, you can always ensure that a pair of variables are in ascending (or descending order):

```
if (a > b)
{
    t = a;
    a = b;
    b = t;
}
```

Given for instance four variables a, b, c, and d, you can apply the above to the pairs a,b and b,c and c,d to ensure that the largest value among a, b, c, d is in d:

```
if (a > b) { swap values of a and b }
if (b > c) { swap values of b and c }
if (c > d) { swap values of c and d }
```

This is called one pass of the sorting process. You can then apply the process to a, b, c for a second pass, and then to a, b for a third pass. This will sort the values of a, b, c, d into ascending order.

Just as there is automatic type casting from int to double, there is

also automatic type casting between bool and int or double:

bool $\Rightarrow$	int
true	1
false	0

numeric (int/double/float) $\Rightarrow$	bool
nonzero (example: 1, 2, 3)	true
zero (example: 0, 0.0)	false

For instance the following is a valid C++ statement: if (1)
`std::cout << "that's all folks";`

The ternary operator looks like this:

`([bool expr] ? [expr1] : [expr2])`

This whole expression evaluates to `[expr1]` if `[bool expr]` is true; otherwise it evaluates to `[expr2]`.

Repetition code and breaking deeply nested code and deep expressions

(There are some solutions at the end of this pdf.)

Here's one way to compute sum from 1 to 5 is

```
int s = 1 + 2 + 3 + 4 + 5;
```

Another way to do this is to do this:

```
int s = 0;
s = s + 1;
s = s + 2;
s = s + 3;
s = s + 4;
s = s + 5;
```

And to make all repetition code exactly the same, you do this:

```
int i = 1;
int s = 0;
s = s + i;
i = i + 1;
s = s + i;
i = i + 1;
s = s + i;
i = i + 1;
s = s + i;
i = i + 1;
s = s + i;
i = i + 1;
```

You have already seen this before.

The reason why you want to think/write your code using repeating code is because later we can use loop controls to tell C++ to repeat a block of code by giving the loop structure just one block of the code.

Exercise -1.8.30. Compute the sum of squares from 1 to 25, i.e., $s = 1*1 + 2*2 + 3*3 + 4*4 + 5*5$ using the same idea as above.

Exercise -1.8.31. Get an integer n from the user and then compute the sum of the rightmost 4 digits of n . Do this with repetition code.

Maximum/minimum computation

Recall I have already talked about the max/min computation. There are two ways of doing it.

Here's the max computation that you have already seen:

```
int w = 1, x = 3, y = 0, z = 5;
int max;

max = w;
if (max < x)
    max = x;
if (max < y)
    max = y;
if (max < z)
    max = z;
```

Notice that other than the first statement, the rest of the code is more or less a repetition of the same kind of code:

```
max = w;
if (max < [variable])
    max = [variable];
```

There's another **TERRIBLE** way to do the above:

```
int w = 1, x = 3, y = 0, z = 5;
if (w >= x && w >= y && w >= z)
    max = w;
else
{
    if (x >= w && x >= y && x >= z)
        max = x;
    else
    {
        if (y >= w && y >= x && y >= z)
            max = y;
        else
            max = z;
    }
}
```

Of course you can try to make it “neater”:


```
int w = 1, x = 3, y = 0, z = 5;

if (w >= x && w >= y && w >= z)
    max = w;
else if (x >= w && x >= y && x >= z)
    max = x;
else if (y >= w && y >= x && y >= z)
    max = y;
else
    max = z;
```

But you are computing way too many boolean conditions. If *z* is the maximum value, how many boolean operators are you evaluating? Compare this to the first max computation.

Secondly, and this is probably even more important:

Exercise -1.8.32. Suppose you need to add another variable to your *w*, *x*, *y*, *z* for max computation:

```
int w = 1, x = 3, y = 0, z = 5, a = 7;
int max;

// ... max computation ...
```

Complete this using the first method and then using the second method. Do I need to explain anymore that the first method is way better?

Part of the problem is that the boolean expression in

```
if (w >= x && w >= y && w >= z)
    max = w;
```

depends on a huge number of variables.

The earlier better solution is to consider one variable at a time.

In general if the boolean is made up of expressions which **do not depend on each other**, it's usually best to process them **separately**.

Count computation

Here's the pseudocode for the count computation again:

```
count = 0
if target == a:
    increment count by 1
if target == b:
    increment count by 1
if target == c:
    increment count by 1
if target == d:
    increment count by 1
print count
```

Again, you see repetition of code.

Another way to do the above, a **TERRIBLE** way, is to do this:

```
count = 0
if target != a && target != b && target != c && target != d
    count = 0
else if target == a && target != b && target != c && target != d
    count 1
else if target != a && target == b && target != c && target != d
    count 1
else if target != a && target != b && target == c && target != d
    count 1
else if target != a && target != b && target == c && target == d
    count 1
else if target == a && target == b && target != c && target != d
    count 2
....
```

I don't even want to complete this!!! This is clearly a really TERRIBLE algorithm. There is a total of $2^4 = 16$ boolean expressions. (Why?) YIKES!

And furthermore, if I add another variable *e* to the above, I would have to change a lot of code, as well as adding some more code.

Exercise -1.8.33. Get integer *x*, *y* from the user and print *x*, *y*, except that if *x* is negative you print 0 instead of *x* and if *y* is negative you print 0 instead of *y*.

Here's a test case

```
2 3
2 3
```

Here's another test case

```
-1 3
0 3
```

and another test:

```
2 -2
2 0
```

and finally

```
-5 -2
0 0
```

You can write your program with boolean conditions where each boolean condition involves **both** x and y . But instead of that, write your program so that you have boolean expressions where each boolean expression involves only **one** variable.

Multiplication game: Part 4

Here's the pseudocode:

```
Generate two random integers between 90 and 99
    and give the values to variables x and y
Store the product of x and y in variable answer
Print a prompt string to user to guess the
    product of x and y
```

```
Declare integer variable score and initialize it
    to 0
```

```
Declare integer variable guess
```

```
Prompt user for integer value for guess
```

```
If guess is answer
```

```
    Print congratulatory message
    score = 2
```

```
Otherwise if guess is less than the answer:
```

```
    Tell user the guess is too low
    Print a prompt string to user to guess
        the product of x and y
```

```
    If guess is answer
        print congratulatory message
        score = 1
```

```
    Otherwise
```

```
        print answer
```

```
Otherwise if guess is greater than the answer:
```

```
    Tell user the guess is too low
    Print a prompt string to user to guess
        the product of x and y
```

```
    If guess is answer
        print congratulatory message
        score = 1
```

```
    Otherwise
```

```
        print answer
```

```
Print user's score
```

Sometimes, you have deeply nested code. What if the above, where you give the user at most 2 chances to answer a question, is changed to allowing the user to have **3** chances or even **4** chances? If you use the above method, you would have a very deeply nested code structure.

Another way to allow the user 3 chances is to avoid deeply nested code as follows. First, since you do not want to nest the 3 cases, you want something like this:

```
ask user for an answer
ask user for an answer
ask user for an answer
```

But of course you want the second “ask user for an answer” to know if the first “ask user for an answer” gave rise to a correct answer or not.

```
ask user for an answer
if first answer is not correct:
    ask user for an answer
ask user for an answer
```

Likewise the third “ask user for an answer” also need to know if a correct answer was given. So instead of “first answer is not correct”, we probably want “answer is not correct” that can be used on the second and third “ask user for an answer”.

```
ask user for an answer
if answer is not correct
    ask user for an answer
if answer is not correct
    ask user for an answer
```

Let's make "answer is not correct" into a boolean variable:

```
bool answer_is_correct

ask user for an answer
if answer from user is correct,
    answer_is_correct = true

if !answer_is_correct:
    ask user for an answer
    if answer from user is correct,
        answer_is_correct = true

if !answer_is_correct:
    ask user for an answer
    if answer from user is correct,
        answer_is_correct = true
```

You want to think of `answer_is_correct` as a **communication device** from the first "ask user for an answer" with the second and third, and also from the second to the third. For this example, the repeated blocks of code are **not completely independent**. Other than the first block, subsequent blocks need to know if the user has given a correct answer.

Get it?

Exercise -1.8.34. Write the multiplication game that allows the user to have at most 3 tries.

Exercise -1.8.35. Get an integer n from the user and then compute the sum of the rightmost 4 digits of n . Once a digit 0 is added, you should stop the summation. Do this with repetition code. Here's a test case

1534315 13

Here's another:

1534015 6

(after adding 5 and 1, on seeing a 0, the summation stops. Therefore the sum is 6) and another

1534205 5

Solutions

Exercise -1.8.36. Compute the sum of squares from 1 to 25, i.e., $s = 1*1 + 2*2 + 3*3 + 4*4 + 5*5$ using the same idea as above.

Solution.

```
#include <iostream>

int main()
{
    int s = 0;
    int i = 1;

    s = s + i;
    i = i + 1;

    s = s + i;
    i = i + 1;

    s = s + i;
    i = i + 1;

    s = s + i;
    i = i + 1;

    s = s + i;
    i = i + 1;

    std::cout << s << '\n';

    return 0;
}
```

Exercise -1.8.37. Get an integer n from the user and then compute the sum of the leftmost 4 digits of n . Do this with repetition code.

Solution.

```
#include <iostream>

int main()
{
    int n;
    std::cin >> n;
```

```
int s = 0;
s = s + n % 10;
n = n / 10;
s = s + n % 10;
n = n / 10;
s = s + n % 10;
n = n / 10;
s = s + n % 10;
n = n / 10;
std::cout << s << ", " << n;
return 0;
}
```

\begin{ex} Get integer \texttt{x}, \texttt{y} from the user and print \texttt{x}, \texttt{y}, except that if \texttt{x} is negative you print \texttt{0} instead of \texttt{x} and if \texttt{y} is negative you print \texttt{0} instead of \texttt{y}. Here's a test case

```
\begin{console}
2 3
```

```
2 3
\end{console}
```

Here's another test case

```
\begin{console}
-1 3
```

```
0 3
\end{console}
```

and another test:

```
\begin{console}
2 -2
```

```
2 0
\end{console}
```

and finally

```
\begin{console}
-5 -2
```

```
0 0
\end{console}
```

You can write your program with boolean conditions where

```

    each boolean condition involves both \texttt{x} and \
    texttt{y}.
\end{ex}

\protect\hypertarget{anchor-2}Solution.

\begin{ex}
But instead of that, write your program so that you have
    boolean expressions where each boolean expression
    involves only one variable.
\begin{console}
#include <iostream>

int main()
{
    int x, y;
    std::cin >> x >> y;
    if (x < 0)
    {
        x = 0;
    }

    std::cout << x;

    if (y < 0)
    {
        y = 0;
    }

    std::cout << y;

    return 0;
}
\end{console}
\end{ex}

\begin{ex}
Get an integer n from the user and then compute the
sum of the rightmost 4 digits of n. Once a digit 0 is added
    , you should
stop the summation. Do this with repetition code. Here's a
    test case

\begin{consolethree}[escapeinside=||]
|\textbf{1534315}|

13

```

Here's another:

```
1534015
```

6

(after adding 5 and 1, on seeing a 0, the summation stops. Therefore the sum is 6) and another

1534205

5

Here's the idea. First here's the code to sum the 4 rightmost digits:

```

#include <iostream>

int main()
{
    int n;
    std::cin >> n;

    int s = 0;

    s = s + n % 10;

    n = n / 10;

    s = s + n % 10;

    n = n / 10;

    s = s + n % 10;

    n = n / 10;

    s = s + n % 10;

    n = n / 10;

    std::cout << s << n;

    return 0;
}

```

Each block needs to know if a zero was encountered. So \ldots

```

\begin{console}
#include <iostream>

int main()
{
    int n;
    std::cin >> n;

    int s = 0;
    bool zero_not_found = true;

    if (zero_not_found)
    {
        s = s + n % 10;
        n = n / 10;
    }

    if (zero_not_found)
    {
        s = s + n % 10;
        n = n / 10;
    }

```

```

    if (zero_not_found)
    {
        s = s + n % 10;
        n = n / 10;
    }

```

Later when I talk about loops and all the repeating blocks of code are placed in a loop structure, you'll see that you can control a loop to make it stop "prematurely".

Exercise

You have more than enough exercises in these notes already. Make sure you go over all of them on your own.

Some Mental Math (DIY)

This section has nothing to do with programming. It's just for fun.

Suppose I told you that it's possible to multiply 98 and 97 very quickly without paper and pencil or calculators or C++. Would you believe me? It does require some practice, but there is a way to multiply two values a little less than 100 very quickly.

Let me show you how with the 98 x 97 example. You subtract the numbers from 100. So $100 - 98 = 2$ and $100 - 97 = 3$. Write this down on a piece of paper:

98	97
2	3

Now look at 98 and 3. Compute $98 - 3$. This is 95. You will need this number. Note that $97 - 2$ is also 95. So there are two different ways of getting 95 with the two diagonals. Write this down:

98	97	95
2	3	

Now look at the 2 and 3. Multiply them together. You get 6. Write this down:

98	97	95	06
2	3		

That's it. Here's another example: 96 x 95. First you have:

96	95
4	5

Next you get

96	95	91
4	5	

and finally

96	95	91	20
4	5		

The answer is 9120. With some practice you will be able to do the computation in your head. (Using algebra, you can actually prove that the method does work.) So now you can show it off to your math teacher.

09. switch

OBJECTIVES

1. Write switch statements
2. Write switch statements where cases do not have breaks
3. Write switch statements where the default case is missing

The if and if-else statements allow us to decide if we want to execute a statement. It allows us to write programs that can make decisions. There is another way of doing that.

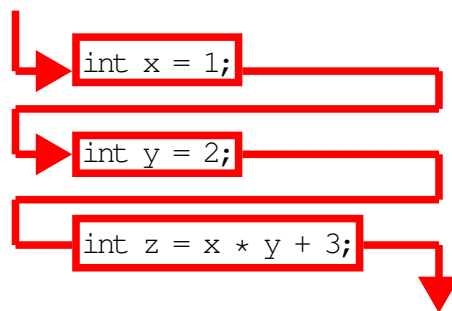
This is the switch statement.

Diagram of flow of execution of *if* statements

Here's the big picture for if.

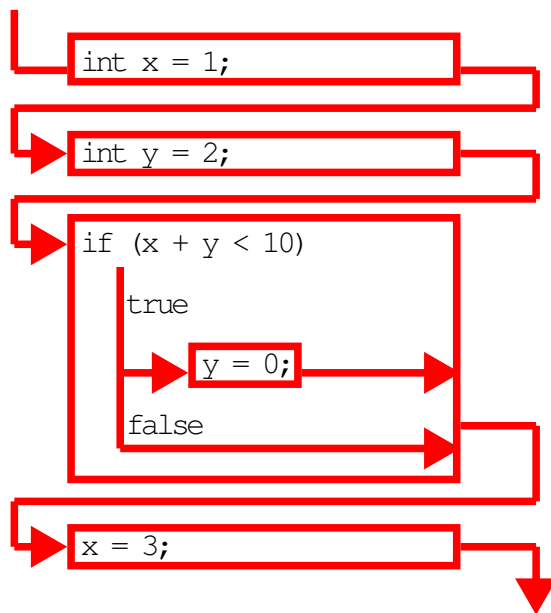
Before the if statement, the code in main executes one statement at a time. For example:

```
int x = 1;  
int y = 2;  
int z = x * y + 3;
```



Now with the if statement we have:

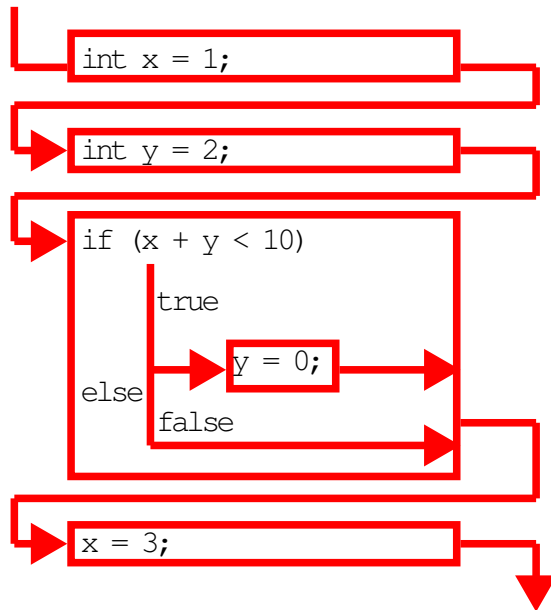
```
int x = 1;  
int y = 2;  
if (x + y < 10)  
    y = 0;  
x = 3;
```



Of course it's just as easy to visualize the flow of execution for if-else. If you have:

```
int x = 1;
int y = 2;
int z = 0;
if (x + y < 10)
{
    y = 0;
}
else
{
    y = 1;
}
x = 3;
```

then



Note that the path of execution is determined by the **boolean value** of a boolean expression.

switch

Now I'll show you something that's very similar to the if and if-else statement. Watch out! There are **subtle differences**

OK. First run this:

```
int x = 0;
std::cin >> x;
if (x == 1)
{
    std::cout << "yo" << std::endl;
}
else if (x == 2)
{
    std::cout << "right ..." << std::endl;
}
else if (x == 3)
{
    std::cout << "oh?" << std::endl;
}
else
{
    std::cout << "huh?" << std::endl;
}
std::cout << "out of if-else" << std::endl;
```

Run this with $x = 0$, $x = 1$, $x = 2$, $x = 3$, $x = 4$. No surprises right?

Note that all the conditions uses the `==` boolean operator and it is used to compare integer values.

Modify the above:

```
int x = 0;
std::cin >> x;
switch (x)
{
    case 1:
        std::cout << "yo" << std::endl;
        break;

    case 2:
        std::cout << "right ..." << std::endl;
        break;

    case 3:
        std::cout << "oh?" << std::endl;
        break;

    default:
        std::cout << "huh?" << std::endl;
}
std::cout << "out of switch" << std::endl;
```

Run it again.

The `switch` statement is like a switchboard, directing the flow of execution based on the integer value of a variable. The flow of execution is passed to the case whose value matches the value of the variable.

Now modify it again by getting rid of the `break` statement in case 2:

```
int x = 0;
std::cin >> x;
switch (x)
{
    case 1:
        std::cout << "yo" << std::endl;
        break;

    case 2:
        std::cout << "right ..." << std::endl;

    case 3:
        std::cout << "oh?" << std::endl;
        break;

    default:
        std::cout << "huh?" << std::endl;
}
std::cout << "out of switch" << std::endl;
```

What does the `break` statement do? If you do **not** have a `break` statement, execution will actually **go to the statements of the next case without checking the value for that case!!!**

There are **two** ways to get **out** of a `switch`: either through a `break` statement or when execution has reached the **end of the switch block**.

By now, I don't have to tell you that you can put any statement in the switch statement. You can put an if or an if-else into the switch. You can put a switch into a switch. You can put a switch into an if-else, etc. Isn't life ever so colorful?

Why? Why? Why?

So why do we need the switch when it's clear that a switch can be written as a nested if-else?

The reason is because the switch statement is extremely fast when there are many cases when compared to the if-else statement.

Note that for a nested if statement:

```
int x = 0;
std::cin >> x;
if (x == 1)
{
    std::cout << "yo" << std::endl;
}
else if (x == 2)
{
    std::cout << "right ..." << std::endl;
}
else if (x == 3)
{
    std::cout << "oh?" << std::endl;
}
else
{
    std::cout << "huh?" << std::endl;
}
std::cout << "out of if-else" << std::endl;
```

each time a boolean condition is false you have to go to the nested level in the if-else stack. This means that for instance to go into the final else block, C++ must evaluate three boolean expressions.

(One optimization trick used by C/C++ programmers is therefore to always put the most likely case first in the stack of if-else statements.)

This is however not the case for the switch statement. If the above is converted to the switch statement, the time taken to get to any case is the same. Even if you have 100 cases, the time to get to the first case is usually the same as the time to get to the last.

Combining cases

Now run this:

```
int x = 0;
std::cin >> x;
switch (x)
{
    case 1:
        std::cout << "what?" << std::endl;
        break;

    case 2:
    case 3:
        std::cout << "oh?" << std::endl;
        break;

    default:
        std::cout << "huh?" << std::endl;
}
std::cout << "out of switch" << std::endl;
```

The above is a useful trick when two cases execute the same statements. The above has the same effect as the following (although the following is probably slower):

```
int x = 0;
std::cin >> x;

if (x == 1)
{
    std::cout << "what?" << std::endl;
}
else if (x == 2 || x == 3)
{
    std::cout << "oh?" << std::endl;
}
else
{
    std::cout << "huh?" << std::endl;
}
std::cout << "out of nested if-else" << std::endl;
```

One more example to make sure you get it:

```
int x = 0;
std::cin >> x;
switch (x)
{
    case 1:
        std::cout << "what?" << std::endl;
        break;

    case 2:
    case 3:
    case 4:
    case 5:
        std::cout << "oh?" << std::endl;
        break;

    default:
        std::cout << "huh?" << std::endl;
}
std::cout << "out of switch" << std::endl;
```

Run it with several values for `x`.

No default

You can leave out the default case:

```
int x = 0;
std::cin >> x;

switch (x)
{
    case 1:
        std::cout << "yo" << std::endl;
        break;

    case 2:
        std::cout << "right ..." << std::endl;

    case 3:
        std::cout << "oh?" << std::endl;
        break;
}
std::cout << "out of switch" << std::endl;
```

Run this with several values for x including for instance x = 0 and x = 100.

Gotchas

Here are some gotchas ... and good source of trick questions for the instructor :)

Each of the following exercises shows you a gotcha for beginning C/C++ programmer. Make sure you find it.

Exercise -1.8.38. Find the bug:

```
int x = 0;
std::cin >> x;
switch x
{
    case 1:
        std::cout << "what?" << std::endl;
        break;

    case 2:
        std::cout << "oh?" << std::endl;

    case 3:
        std::cout << "huh?" << std::endl;
        break;
}
std::cout << "out of switch" << std::endl;
```

Exercise -1.8.39. Find the bug:

```
int x = 0;
std::cin >> x;
switch (x)
{
    case < 1 :
        std::cout << "what?" << std::endl;
        break;

    case 2:
        std::cout << "oh?" << std::endl;

    case 3:
        std::cout << "huh?" << std::endl;
        break;
}
std::cout << "out of switch" << std::endl;
```

Exercise -1.8.40. Bug, bug, bug! Find it! ...

```
double x = 0;
std::cin >> x;
switch (x)
{
    case 1.1:
        std::cout << "what?" << std::endl;
        break;

    case 1.2:
        std::cout << "oh?" << std::endl;

    case 1.3:
        std::cout << "huh?" << std::endl;
        break;
}
std::cout << "out of switch" << std::endl;
```

Exercise -1.8.41. Where's that sneaky bug??

```
int x = 0;
std::cin >> x;
int y = 1;
switch (x)
{
    case y:
        std::cout << "what?" << std::endl;
        break;

    case 2:
        std::cout << "oh?" << std::endl;

    case default:
        std::cout << "huh?" << std::endl;
        break;
}
std::cout << "out of switch" << std::endl;
```

Exercise -1.8.42. Recall that a character is the “atom” of a string. The name of the character type is `char`. You can declare character variables, get character values from the keyboard, and print the character value of a character variable. Try this:

```
char x = ' ';
std::cin >> x;
std::cout << x << std::endl;
```

Now here's the question: Can you use characters for the cases?

```
int x, y;
char op;
std::cout << "x: ";
std::cin >> x;
std::cout << "y: ";
std::cin >> y;
std::cout << "operator (+ or -): ";
std::cin >> op;
switch (op)
{
    case '+':
        std::cout<< x + y<< std::endl;
        break;

    case '-':
        std::cout<< x - y<< std::endl;
        break;

    default:
        std::cout<< "no such op<<
        std::endl;
        break;
}
std::cout<< "out of switch << std::endl;
```

Add all the binary integer operators you know so far to the above program.

Coding style

Good::

```
switch (x)
{
    case 1:
        std::cout << "what?" << std::endl;
        break;

    case 2:
    case 3:
        std::cout << "oh?" << std::endl;
        break;

    default:
        std::cout << "huh?" << std::endl;
}
```

This is also perfectly OK: :

```
switch (x)
{
    case 1:
    {
        std::cout << "what?" << std::endl;
        break;
    }

    case 2:
    case 3:
    {
        std::cout << "oh?" << std::endl;
        break;
    }

    default:
    {
        std::cout << "huh?" << std::endl;
    }
}
```


More information

Actually you can use the value of an **integer expression** and not just a variable to determine the flow of execution:

```
int x = 0;
std::cin >> x;
int y = 1;
switch (x + y)
{
    case 1:
        std::cout << "yo" << std::endl;
        break;

    case 2:
        std::cout << "right ..." << std::endl;

    case 3:
        std::cout << "oh?" << std::endl;
        break;
}
std::cout << "out of switch" << std::endl;
```

And although you cannot use variables for the case values, you can use **integer constants** or even **constant integer expressions**:

```
int x = 0;
std::cin >> x;
const int y = 1;
const int z = 2;
switch (x)
{
    case y + z + 1:
        std::cout << "yo" << std::endl;
        break;

    case 2:
        std::cout << "right ..." << std::endl;

    case 3:
        std::cout << "oh?" << std::endl;
        break;
}
std::cout << "out of switch" << std::endl;
```

Summary

The statement looks like this:

```
switch ([int expr])  
{  
  case [const int expr]:  
    [stmt]  
  ...  
}
```

or

```
switch ([int expr])  
{  
  case [const int expr]:  
    [stmt]  
  ...  
  default:  
    [stmt2]  
}
```

where `[stmt]` and `[stmt2]` are either statements or blocks of statements. `[int expr]` is an expression that can be evaluated to an integer and `[const int expr]` is a constant integer expression.

Once the flow of execution enters a case, it will execute all the statements in that case until a `break` or end of switch block is reached. If there is no `break`, execution will continue with the statements of the next case without checking the value for that case, if there is one.

On executing the `break` statement in a `switch` statement, flow of control will resume to the statement immediately after the `switch` statement.

Exercise.

Exercise -1.8.43. Q1. Read this program carefully. Determine which part of the program can be rewritten using the switch statement. Rewrite it and test it.

```
int numHeads = 0;
std::cout<< "How many heads do you have? ";
std::cin >> numHeads;

if (numHeads< 1)
{
    std::cout<< "Huh" << std::endl;
}
else
{
    if (numHeads == 1)
    {
        std::cout << "Let me introduce you to our << "surgeons"
        << std::endl;
    }
    else
    {
        if (numHeads == 2)
        {
            std::cout << "Are you Zaphod?";
        }
        else
        {
            std::cout << "You have way too many"
            << std::endl;
        }
    }
}
```

Exercise -1.8.44. Q2. Rewrite the nested if-else statement using the switch statement:

```
int x = 0;
std::cin >> x;

int y = 0;

if (x == 0)
{
    y = 5;
}
else if (x == 1 || x == 2 || x == 3)
{
    y = 7;
}
else if (x == 6 || x == 7 || x == 42)
{
    y = 0;
}
std::cout << y << std::endl;
```

Avoid code duplication!

Exercise -1.8.45. Q3. Write a program that declares four integer constants N, S, E, W. with values 0, 1, 2, 3 respectively. Generates a random integer from 0 to 3. If the random integer is N, print “go north”. If the random integer is S, print “go south”. If the random integer is E, print “go east”. If the random integer is W, print “go west”. Use a `switch` statement.

10. for Loops: Part 1

OBJECTIVES

- Use increment, decrement and augmented assignment operators
- Write `for`-loops
- Write block of statements for the body of the `for` loops

We'll now begin our study of loops. I've already many times that a loop allows you to repeat the execution of a chunk of code.

I'll also talk a little more about the important concept of scopes

Why loops?

What are loops? Well, a loop is just something that will **repeat** a statement or a bunch of statements (i.e., a block). Why do we need loops? Because we are too lazy and **way too smart** to do redundant work.

For instance, look at the following multiplication game. Instead of asking a single multiplication question (I'm using 2 spaces for indentation for the notes but in your work you should still use 4 spaces):

```
#include <iostream>
#include <ctime>
int main()
{
    srand((unsigned int) time(0));
    int x = rand() % 10 + 90;
    int y = rand() % 10 + 90;
    int guess = 0;
    std::cout << "What is " << x << " * " << y << " ?";
    std::cin >> guess;
    int product = x * y;
    if (guess < product)
    {
        std::cout << "Incorrect! Too low!"
        << "The answer is " << product
        << std::endl;
    }
    else if (guess == product)
    {
        std::cout << "Correct!" << std::endl;
    }
    else
    {
        std::cout << "Incorrect! Too high!"
        << "The answer is" << product
        << std::endl;
    }
}
```

what if you want to ask two questions? You can copy and paste your code:

```
include < iostream>
include < ctime>
int main()
{
    srand((unsigned int) time(0));
    int x = rand() % 10 + 90;
    int y = rand() % 10 + 90;
    int guess = 0;
    std::cout << "What is " << x << " * " << y << "?";
    std::cin >> guess;
    int product = x * y;
    if (guess < product)
    {
        std::cout << "Incorrect! Too low!"
        << "The answer is " << product
        << std::endl;
    }
    else if (guess == product)
    {
        std::cout << "Correct!" << std::endl;
    }
    else
    {
        std::cout << "Incorrect! Too high!"
        << "The answer is" << product
        << std::endl;
    }
    x = rand() % 10 + 90;
    y = rand() % 10 + 90;
    std::cout << "What is " << x << " * " << y << "?";
    std::cin >> guess;
    product = x * y;
    if (guess < product)
    {
        std::cout << "Incorrect! Too low!"
        << "The answer is " << product
        << std::endl;
    }
    else if (guess == product)
    {
        std::cout << "Correct!" << std::endl;
    }
    else
    {
        std::cout << "Incorrect! Too high!"
        << "The answer is" << product
        << std::endl;
    }
}
```

(Of course variables `x`, `y`, `guess`, and `product` are already de-

clared, so you don't declare them again!)

Great ...

But ... what if you want to ask **10** number of questions he/she wants to answer?

And ... what if you want to modify the program? Each time you modify one part of the program you have to modify all the other copies!!! That's not smart!!!

That's when you need to know how to write loops.

Or, take for instance, a computer game. Most games with graphics involve continually moving an image by a small amount to simulate smooth motion. In this case you want to repeat each small change until the user stops the game or when the game ends.

Similarly, in business applications, for instance in a payroll program, the program will process the pay for each employee kept in a file until all records are read. (Never mind if you don't understand what a file is. The point is that you need repetitions.)

Anyway, you will need a way to tell C++ to continually execute a statement or a block of statements until a condition is reached.

C++ understands three different types of loops: the **for-loop**, the **while-loop** and the **do-while-loop**.

We'll talk about the `for`-loop first. But before we talk about the `for`-loop, we will need to talk about some operators which are frequently used with the `for`-loop.

In this set of notes, because of the way C++ (and Java) programmers write their loops, I also want to talk about "scope".

Increment and decrement operators

You have already seen the +, -, *, /, % operators for numeric values. Now for a few more.

Exercise -1.8.46. Run this:

```
{
int i = 1;

i++;
std::cout << i << std::endl;

i = 1;
++i;
std::cout << i << std::endl;

i = 1;
i--;
std::cout << i << std::endl;

i = 1;
--i;
std::cout << i << std::endl;
}
```

Change the initial value of **i** to any other number and run the program again.

The ++ and – are the increment and decrement operators respectively. When you apply ++ **in front** of the variable, you are using the **pre-increment** operator:

```
++i;
```

And when you use the ++ **after** the variable, you are using the **post-increment** operator.

```
i++;
```

Although they both add 1 to **i**, there is a difference. First of all, when you apply ++ to a variable, not only is the value of the variable changed, a copy of the value is returned. The next two exercises will make everything clear.

Exercise -1.8.47. Run this:

```
int i = 5;
int j = ++i; // i is incremented and the new value of
             // i is given to j
std::cout << i << " " << j << std::endl;
```

Exercise -1.8.48. Now run this:

```
int i = 5;
int j = i++; // The value of i is given to j and
             // *then* i is incremented
std::cout << i << " " << j << std::endl;
```

Make sure to compare the above two programs!!! In other words, the difference is in the **order** of incrementing and giving the value of the variable

The same applies to the pre- and the post-decrement operators.

Exercise -1.8.49. Run this.

```
int i = 5;
int j = --i;
std::cout << i << " " << j << std::endl;
```

Exercise -1.8.50. And this:

```
int i = 5;
int j = i--;
std::cout << i << " " << j << std::endl;
```

So now you know three different ways to add 1 to variable i:

```
i = i + 1; i++; ++i;
```

Choices are nice, right? Well, except for people who can't make up their minds. Here's a hint: the increment operators are faster. Another thing. Do not try to confuse yourself (or others) by using the increment and decrement operators in an expression. For instance the following is **BADDDDDDD** although it does run:

```
a = (++i) + (k--) * (++j);
```

This is the same as:

```
++i;  
++j;  
a = i + k * j;  
--k;
```

which is much easier to read.

Exercise -1.8.51. First try to figure out the output without your C++ compiler:

```
int i = 0;  
int j = (++i) + (++i);  
std::cout << i << ' ' << j << '\n';
```

Now verify with your C++ compiler. Explain to yourself what happens when the pre-increment operator is applied multiple times to the same variable in an expression. Now do the same with this:

```
int i = 0;  
int j = (i++) + (i++);  
std::cout << i << ' ' << j << '\n';
```

Exercise -1.8.52. We have been using the pre-++ and post-++ on *int* variables. Can you do that to *double* variables?

Let me summarize what we now know about the pre- and post-increment operators:

++i	increments the value of i
i++	increments the value of i
j=(++i)	increments the value of i and then assign new value of i to j
j=(i++)	give the value of i to j and then increments the value of i

and here's the summary for pre- and post-decrement operators:

--i	decrements the value of i
i--	decrements the value of i
j=(--i)	decrements the value of i and then assign new value of i to j
j=(i--)	give the value of i to j and then decrements the value of i

Gotchas

Try all these gotchas and remember them.

```
int i = 5;
(i++)++;
std::cout << i + ' ' + j << '\n';
```

```
int i = 5;
++(i++);
std::cout << i << ' ' << j << '\n';
```

```
int i = 5;
--(i++);
std::cout << i << ' ' << j << '\n';
```

```
std::cout << (++2) << '\n';
```

Augmented assignment operators

Exercise -1.8.53. Run this

```
int i = 1, j = 5;

i += 2;
std::cout << i << std::endl;

i = 1;
i += j;
std::cout << i << std::endl;
```

Change the initial values of `i` and `j` and run the program again.

Exercise -1.8.54. Run this

```
int i = 1, j = 5;

i -= 2;
std::cout << i << std::endl;

i = 1;
i -= j;
std::cout << i << std::endl;
```

Run the program again with different initial values for `i` and `j`.

Exercise -1.8.55. Run this

```
int i = 1, j = 5;

i *= 2;
std::cout << i << std::endl;

i = 1;
i *= j;
std::cout << i << std::endl;
```

In general, suppose `[op]` is an operator such as `+`, `-`, `*`, `/`, `%`, then

$$x \text{ [op]} = y$$

is the same as

$$x = x \text{ [op]} y$$

For instance,

$x \ \% = \ y \ + \ 5$	is the same as	$x = x \ \% \ (y \ + \ 5)$
$y \ /\ = \ c$	is the same as	$y = y \ /\ c$

etc. Note that the left hand version (using the augmented assignment operators) is faster than using an operator and then assignment.

Exercise -1.8.56. First figure out the output on your own:

```
int i = 10, j = 5;

i /= 2;
std::cout << i << std::endl;

i = 1;
i /= j;
std::cout << i << std::endl;
```

Now verify with your C++ compiler.

Exercise -1.8.57. Figure out the output without your compiler:

```
int i = 9, j = 5;

i %= 2;
std::cout << i << std::endl;

i = 1;
i %= j;
std::cout << i << std::endl;
```

Verify with your compiler.

Exercise -1.8.58. Figure out the output without your compiler:

```
int i = 2, j = -3, k = 5;

i %= j + k;
std::cout << i << std::endl;

i = 1;
i *= j;
std::cout << i << std::endl;
```

Verify with your compiler.

Exercise -1.8.59. One more exercise ...

```
int i = 9, j = 5;

i += i + j;
std::cout << i << std::endl;
```

By the way, you now have **four** different ways to add one to variable *i*:

```
++i;      i++;      i += 1;      i = i + 1;
```

(C++ programming is terrible for people who can't decide. Right?)
In general, the operators on the left are faster than the ones to their right.

Exercise -1.8.60. Rewrite the following using an augmented assignment operator.

```
int heads = 0, extraHeads = 0;
std::cin >> heads;
std::cin >> extraHeads;
heads = heads + extraHeads;
std::cout << heads << std::endl;
```

Exercise -1.8.61. Rewrite the following using an augmented assignment operator.

```
double princ = 0.0, rate = 0.0;
std::cin >> princ;
std::cin >> rate;
princ = princ * (1 + rate);
std::cout << princ << std::endl;
```

Exercise -1.8.62. In fact, the augmented operators actually return a value, i.e., besides modifying the value of a variable, they evaluate to a value. Try this experiment:

```
int i = 0;
int j = (i += 1); // i is incremented *AND* (i += 1)
// gives the new value of i
std::cout << i << ' ' << j << std::endl;
```

Exercise -1.8.63. What is the output (without using your compiler of course)?

```
int i = 5;
int j = (i += 5);
int k = (j += i);
i = (k -= j + 1);
std::cout << i << ' ' << j << ' ' << k << std::endl;
```

Exercise -1.8.64. Here's a horrifying program. First try to figure out the output on your own.

```
int i = 0, j = 1;
int k = (++i) * (i += 1) + (j *= i) * (i--);
std::cout << i << ' ' << j << ' ' << k << '\n';
```

Now verify with your C++ compiler. You should **never** write such an expression in actual programs. And if you join a company where programmers write such code ... well ... good luck to you!!!

Exercise -1.8.65. Can we increment an `int` by a `double`?

```
int i = 5;
double j = 1.5;
i += j;
std::cout << i << ' ' << j << std::endl;
std::cout << (i += j) << ' ' << j << std::endl;
```


Exercise -1.8.66. Can we increment a double by a double?

```
double i = 5.5;
int j = 1;
i += j;
std::cout << i << ' ' << j << std::endl;
std::cout << (i += j) << ' ' << j << std::endl;
```

Exercise -1.8.67. How about incrementing a double by an int?

```
double i = 5.5;
int j = 1;
i += j;
std::cout << i << ' ' << j << std::endl;
std::cout << (i += j) << ' ' << j << std::endl;
```

Exercise -1.8.68. Test if `-=`, `*=`, `/=` work with doubles. What about `%=?` for-loop

for-loop

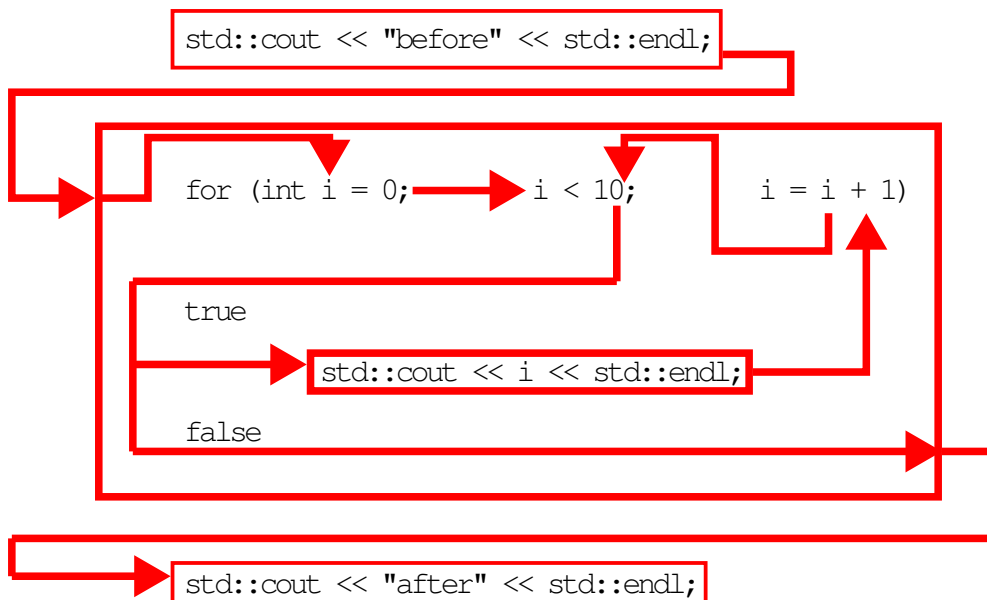
Exercise -1.8.69. Run this

```
std::cout << "before" << std::endl;

for (int i = 0; i < 10; i = i + 1)
    std::cout << i << std::endl;

std::cout << "after" << std::endl;
```

Note that the statement in bold, the `for`-loop, is a single statement.



Here's how the `for`-loop controls the execution:

```
std::cout << "before" << std::endl;
for (int i = 0; i < 10; i = i + 1)
true
std::cout << i << std::endl;
false
std::cout << "after" << std::endl;
```

Exercise -1.8.70. Rewrite the `for`-loop using the `++` operator (either pre-increment or post-increment).

More examples

The for-loop is a brand new concept. So let's do more drills before your first warm-up.

Try this:

```
std::cout << "before" << std::endl;

for (int i = 3; i < 7; i++)
    std::cout << i << std::endl;

std::cout << "after" << std::endl;
```

Try this:

```
std::cout << "before" << std::endl;

for (int i = 3; i <= 7; i++)
    std::cout << i << std::endl;

std::cout << "after" << std::endl;
```

Try this:

```
std::cout << "before" << std::endl;

for (int i = 3; i <= 7; i += 2)
    std::cout << i << std::endl;

std::cout << "after" << std::endl;
```

Try this:

```
std::cout << "before" << std::endl;

for (int i = 5; i >= 0; i--)
    std::cout << i << std::endl;

std::cout << "after" << std::endl;
```

Exercise -1.8.71. Warm-up time!!! Modify the previous program so that the output is

```
10
8
6
4
```

Exercise -1.8.72. Write a program that prints 1000 "hello world!"s ... oh ... and you have to do it in 1 minute. So either you are a killer typist or you have to use the `for`-loop.

Blocks

Of course it's not too surprising that you can have **blocks** with the for-loop:

```
std::cout << "entering the for-loop ...\n";
for (int i = 10; i > 0; i--)
{
    std::cout << "entered the block ...\n";
    std::cout << "i: " << i;
    std::cout << "exiting the block ...\n";
}
std::cout << "done with the for-loop ...\n";
```

It shouldn't be surprising that you can do this:

```
std::cout << "entering the for-loop ...\n";
int x = 10, y = 0;
for (int i = x; i > y; i--)
{
    std::cout << "entered the block ...\n";
    std::cout << "i: " << i;
    std::cout << "exiting the block ...\n";
}
std::cout << "done with the for-loop ...\n";
```

By the way, just like in the case of `if-` and `if-else` statements, C/C++ programming tend to use blocks for the for-loop even when the body of the for-loop has only one statement:

```
for (int i = 0; i < 10; i++)
{
    std::cout << i << std::endl;
}
```

Exercise -1.8.73. Remember our program that prints a table of squares?

n	n ²
0	0
1	1
2	4

Write a program that prints the squares of all positive integers from 0 to 100. Your program must be less than 10 lines long and you have 3 minutes to do that - so I don't mean you have a very long

`std::cout`!!! Here's some pseudocode that might help:

```
Print "n    n^2\n"
Print "---- ---\n"
For n running from 0 to 100 (inclusive):
    print n, spaces, square of n, and newline
```

Exercise -1.8.74. Modify your program so that it prompts the user for integers and assign them to integer variables `start` and `end`. Print a table of squares from `start` to `end`.

Exercise -1.8.75. Can you use a `double` variable to control the for-loop? Try this

```
std::cout << "before" << std::endl;

for (double x = 3.1; x < 7.2; x += 0.1)
{
    std::cout << x << std::endl;
}

std::cout << "after" << std::endl;
```

Exercise -1.8.76. Write a program that prints a table of cubes (like the program on squares above).

Scope

The scope of a variable refers to where you can reference the variable. In the following program:

```
for (int i = 10; i > 0; i--)
{
    std::cout << i;
    std::cout << " ... ";
}
std::cout << "blast off!" << std::endl;
```

The variable `i` is created in the `for`-loop. Once you step out of the `for`-loop, the variable `i` goes out of scope and is **destroyed**.

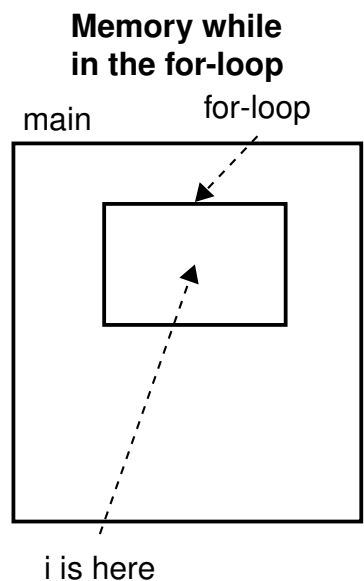
THIS IS VERY IMPORTANT!!!

Try this:

```
#include <iostream>
int main()
{
    for (int i = 10; i > 0; i--)
    {
        std::cout << i;
        std::cout << " ... ";
    }

    std::cout << "blast off!" << std::endl;
    std::cout << i << std::endl;

    return 0;
}
```



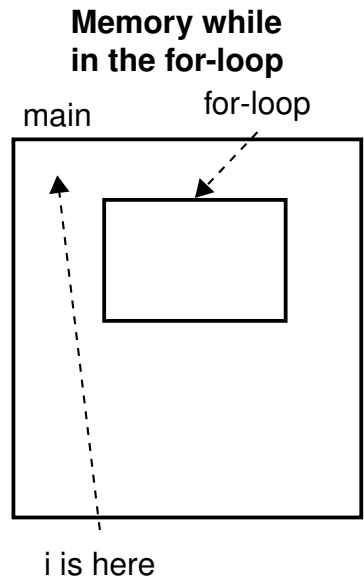
(WARNING: If you're using Microsoft Visual Studio, you MUST Disable Language Extensions. Some versions of MSVS do NOT destroy variables created inside a `for`-loop.)

Now try this:

```
#include <iostream>
int main()
{
    int i = 0;
    for (int i = 10; i > 0; i--)
    {
        std::cout << i;
        std::cout << " ... ";
    }

    std::cout << "blast off!" << std::endl;
    std::cout << i << std::endl;

    return 0;
}
```



There are other scenarios and other scope rules but this is enough for the time being.

It's really important to understand the scope of a for-loop. The presence of a block (and not just a statement) that is controlled by the for-loop visually makes things slightly more complicated. So let's look at the basic for-loop again.

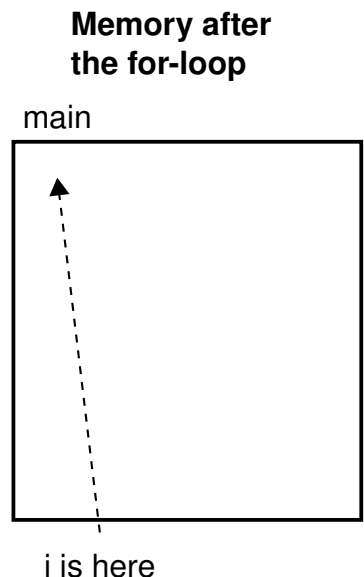
Here's a simple for-loop:

```
for (int i = 0; i < 10; i++)
    std::cout << i << std::endl;

std::cout << i << std::endl;
```

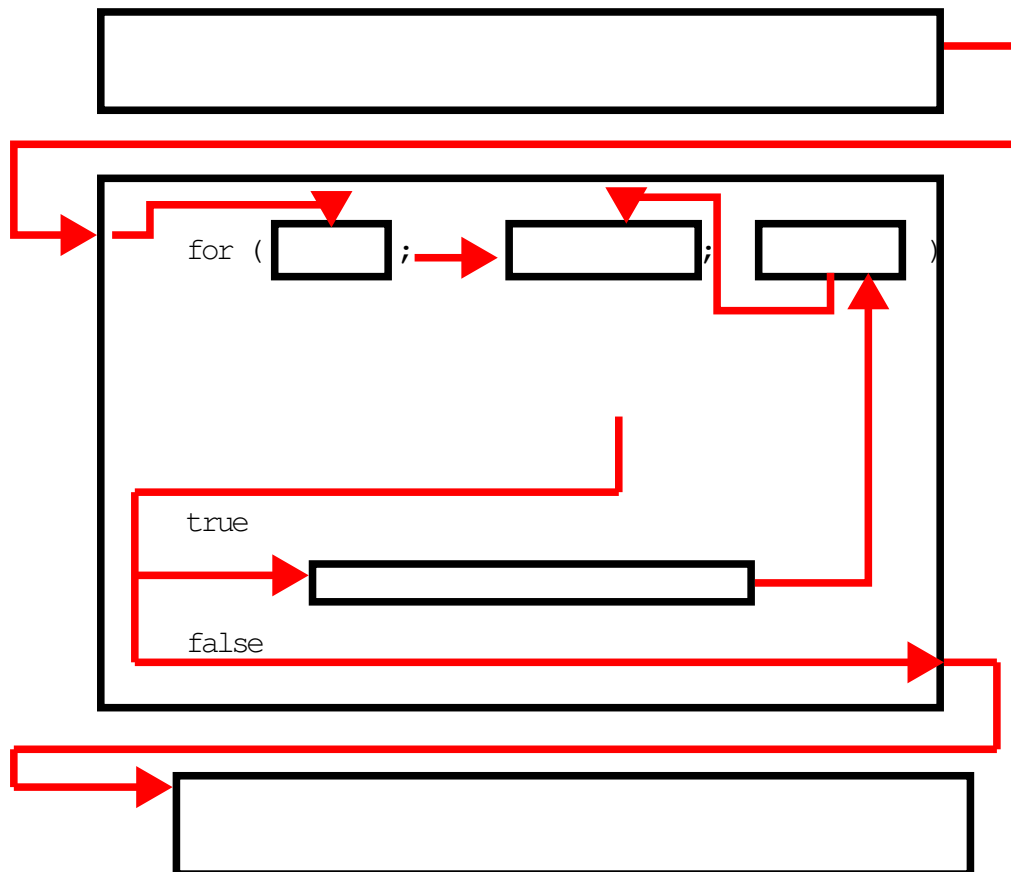
Again you see that the `i` lives inside the `for`-loop and dies and once the `for`-loop is finished, the `i` is destroyed.

There are other scope rules but this is enough for the time being.



Mental picture of flow of execution

Recall that the flow of execution for the for-loop looks like this:

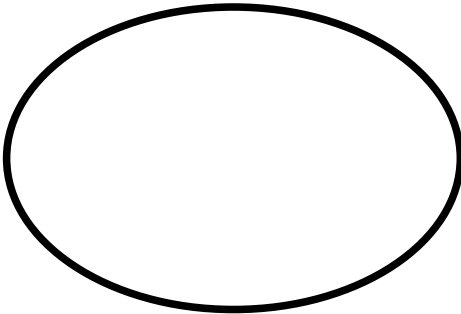


For you to be a competent programmer, it's extremely important to "visualize" the flow of execution and roughly how the computer thinks. So I'm going to simulate the execution of a simple for-loop.

```
int sum = 0;
    for ( int i = 3;    i < 5;    i = i + 1 )

        sum = sum + i;
```

CPU



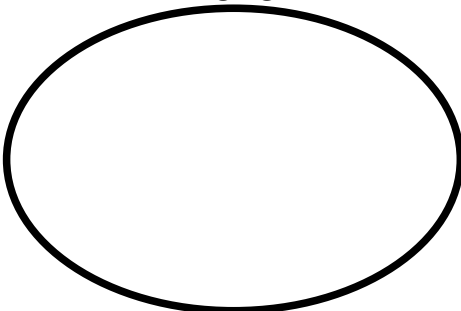
main



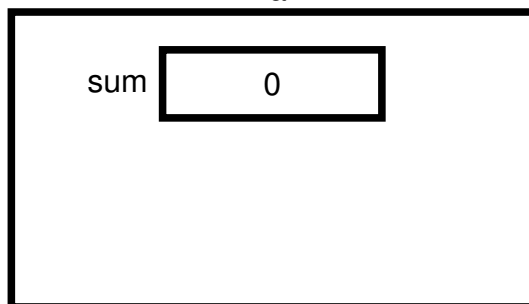
```
int sum = 0;
    for ( int i = 3;    i < 5;    i = i + 1 )

        sum = sum + i;
```

CPU

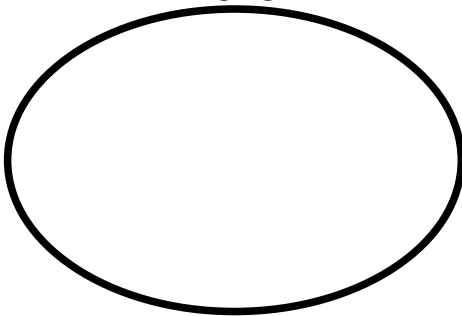


main

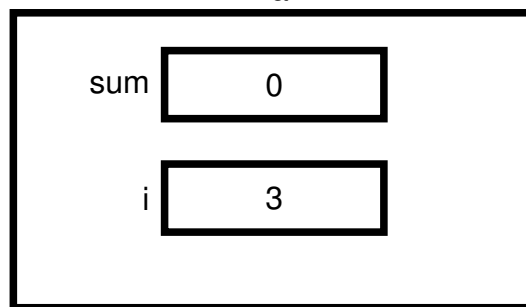


```
int sum = 0;  
for ( int i = 3;    i < 5;    i = i + 1 )  
  
    sum = sum + i;
```

CPU

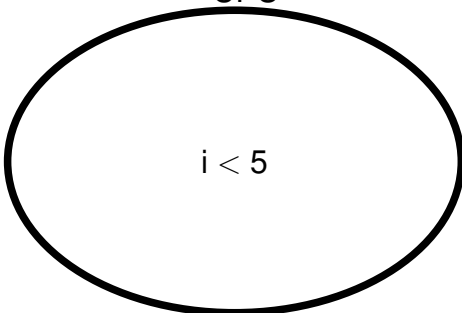


main

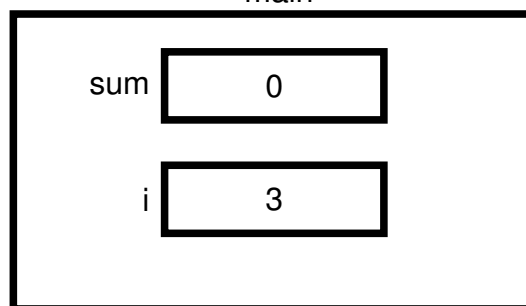


```
int sum = 0;  
for ( int i = 3;    i < 5;    i = i + 1 )  
  
    sum = sum + i;
```

CPU



main



```
int sum = 0;  
for ( int i = 3; i < 5; i = i + 1 )  
    sum = sum + i;
```

CPU

 $3 < 5$

main

sum 0

i 3

```
int sum = 0;  
for ( int i = 3; i < 5; i = i + 1 )  
    sum = sum + i;
```

CPU

true

main

sum 0

i 3

```
int sum = 0;  
for ( int i = 3;    i < 5;    i = i + 1 )  
    sum = sum + i;
```

CPU

$$\text{sum} = \text{sum} + i$$

main

sum 0

i 3

```
int sum = 0;  
for ( int i = 3;    i < 5;    i = i + 1 )  
    sum = sum + i;
```

CPU

$$\text{sum} = 0 + i$$

main

sum 0

i 3

```
int sum = 0;  
for ( int i = 3;    i < 5;    i = i + 1 )  
    sum = sum + i;
```

CPU

 $sum = 0 + 3$

main

sum 0

i 3

```
int sum = 0;  
for ( int i = 3;    i < 5;    i = i + 1 )  
    sum = sum + i;
```

CPU

 $sum = 3$

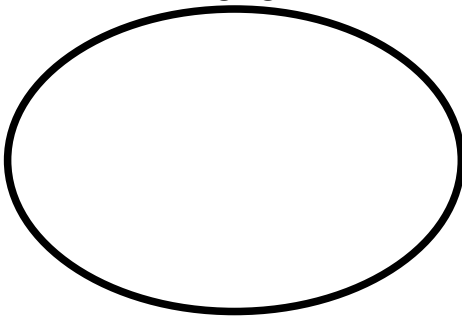
main

sum 0

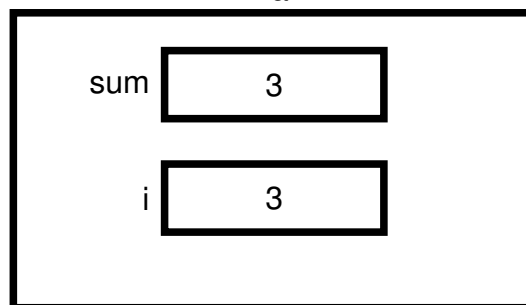
i 3

```
int sum = 0;  
for ( int i = 3;    i < 5;    i = i + 1 )  
    sum = sum + i;
```

CPU

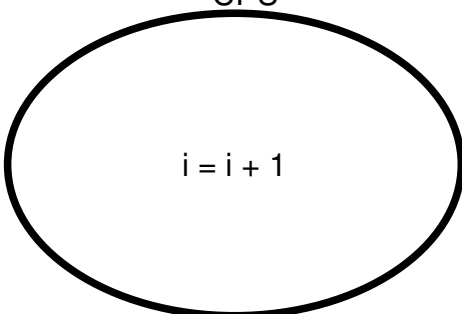


main

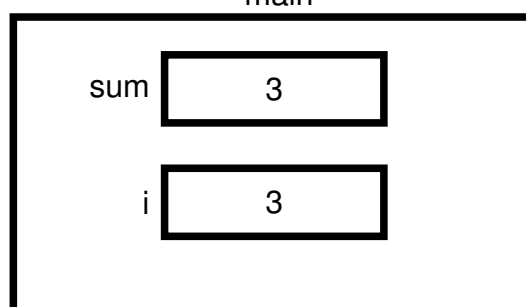


```
int sum = 0;  
for ( int i = 3;    i < 5;    i = i + 1 )  
    sum = sum + i;
```

CPU



main



```
int sum = 0;
for ( int i = 3;    i < 5;    i = i + 1 )
    sum = sum + i;
```

CPU

 $i = 3 + 1$

main

sum 3

i 3

```
int sum = 0;
for ( int i = 3;    i < 5;    i = i + 1 )
    sum = sum + i;
```

CPU

 $i = 4$

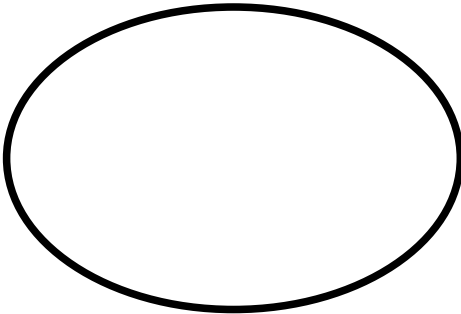
main

sum 3

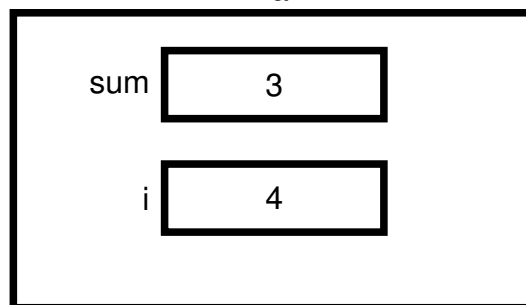
i 3


```
int sum = 0;  
for ( int i = 3;    i < 5;    i = i + 1 )  
  
    sum = sum + i;
```

CPU

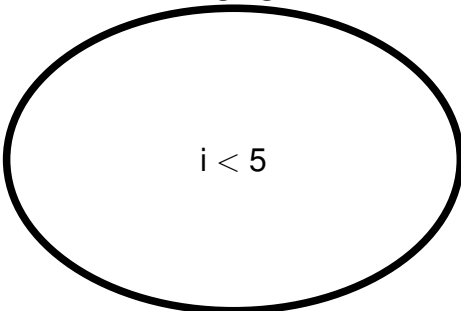


main

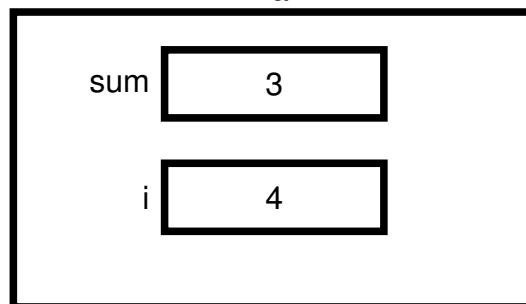


```
int sum = 0;  
for ( int i = 3;    i < 5;    i = i + 1 )  
  
    sum = sum + i;
```

CPU



main



```
int sum = 0;  
for ( int i = 3; i < 5; i = i + 1 )  
  
    sum = sum + i;
```

CPU

 $4 < 5$

main

sum 3

i 4

```
int sum = 0;  
for ( int i = 3; i < 5; i = i + 1 )  
  
    sum = sum + i;
```

CPU

true

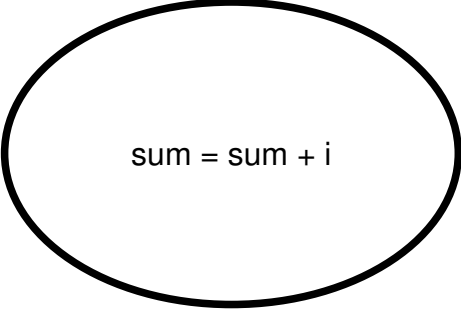
main

sum 3

i 4

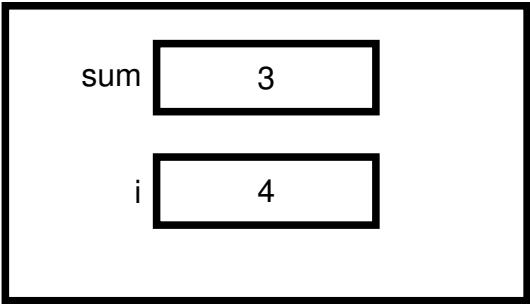
```
int sum = 0;
for ( int i = 3; i < 5; i = i + 1 )
    sum = sum + i;
```

CPU



sum = sum + i

main

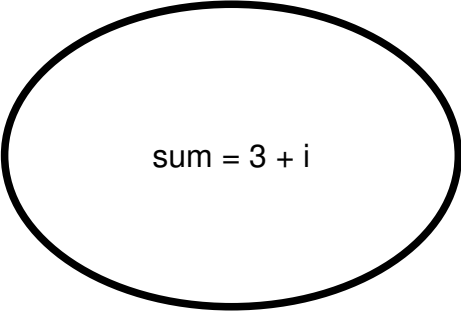


sum 3

i 4

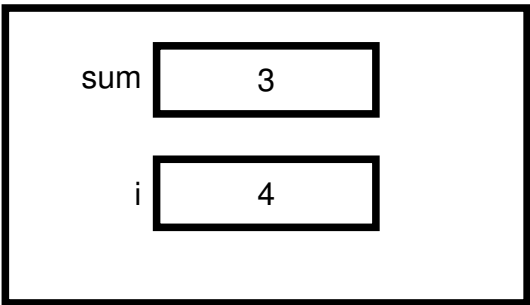
```
int sum = 0;
for ( int i = 3; i < 5; i = i + 1 )
    sum = sum + i;
```

CPU



sum = 3 + i

main



sum 3

i 4

```
int sum = 0;  
for ( int i = 3;    i < 5;    i = i + 1 )  
    sum = sum + i;
```

CPU

 $sum = 3 + 4$

main

sum 3

i 4

```
int sum = 0;  
for ( int i = 3;    i < 5;    i = i + 1 )  
    sum = sum + i;
```

CPU

 $sum = 7$

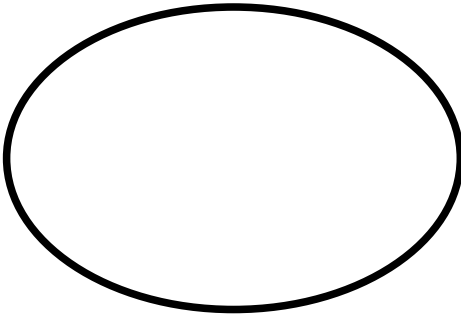
main

sum 3

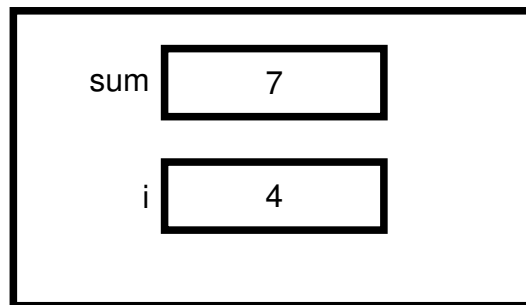
i 4

```
int sum = 0;  
for ( int i = 3;    i < 5;    i = i + 1 )  
    sum = sum + i;
```

CPU

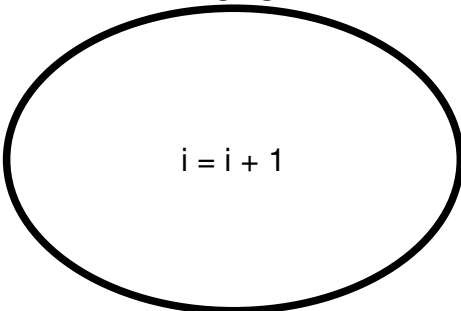


main

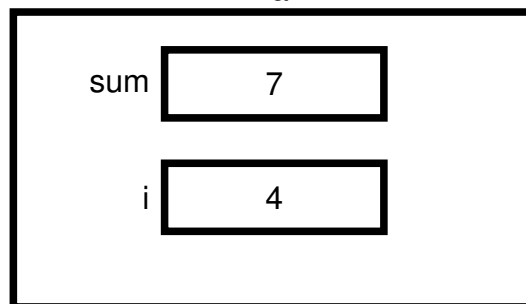


```
int sum = 0;  
for ( int i = 3;    i < 5;    i = i + 1 )  
    sum = sum + i;
```

CPU



main



```
int sum = 0;  
for ( int i = 3;    i < 5;    i = i + 1 )  
  
    sum = sum + i;
```

CPU

 $i = 4 + 1$

main

sum 7

i 4

```
int sum = 0;  
for ( int i = 3;    i < 5;    i = i + 1 )  
  
    sum = sum + i;
```

CPU

 $i = 5$

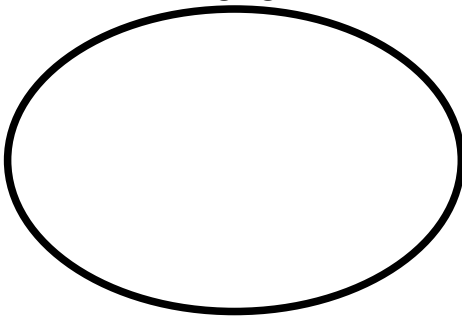
main

sum 7

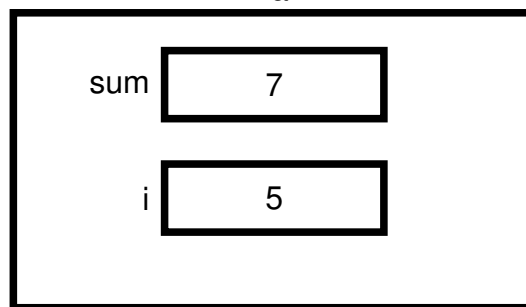
i 4

```
int sum = 0;  
for ( int i = 3;    i < 5;    i = i + 1 )  
  
    sum = sum + i;
```

CPU

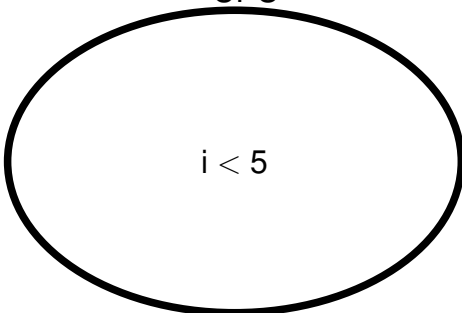


main

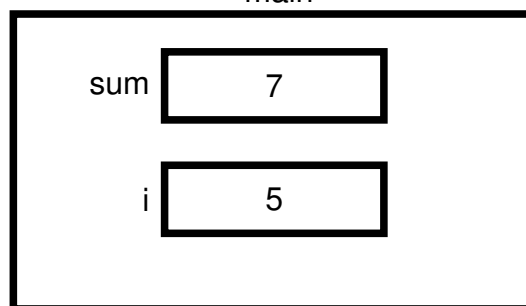


```
int sum = 0;  
for ( int i = 3;    i < 5;    i = i + 1 )  
  
    sum = sum + i;
```

CPU



main



```
int sum = 0;  
for ( int i = 3; i < 5; i = i + 1 )  
    sum = sum + i;
```

CPU

 $5 < 5$

main

sum 7

i 5

```
int sum = 0;  
for ( int i = 3; i < 5; i = i + 1 )  
    sum = sum + i;
```

CPU

false

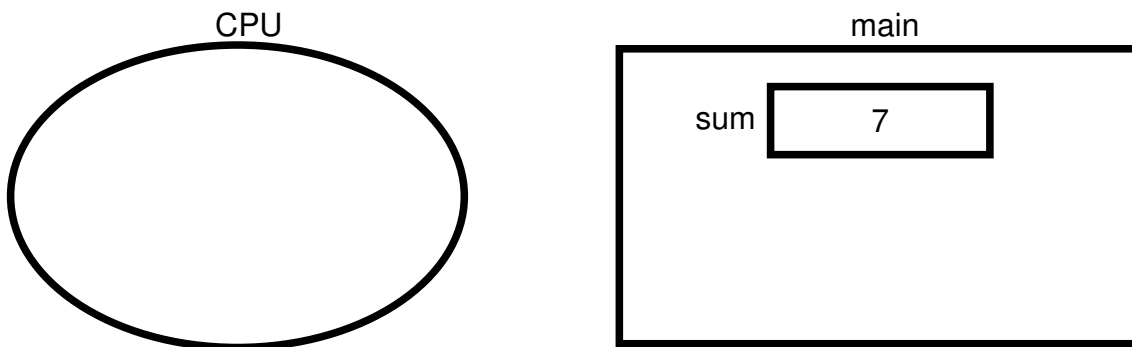
main

sum 7

i 5


```
int sum = 0;
    for ( int i = 3;    i < 5;    i = i + 1 )

        sum = sum + i;
```



Note that the above simulation is only a **model**. Furthermore, it does not explain how the computer knew that $4 < 5$ was true. And what does “true” really mean to a computer anyway? In fact, does a computer “know” what’s a “4” (as in four)??? For details like these you will have to take Assembly Language (CISS360) and Computer Architecture (CISS420).

When you leave the for-loop, variable i is destroyed

Exercise -1.8.77. There’s one place where you can use the pre-increment operator and another where you can use `+=`. Modify the above code.

Exercise -1.8.78. Do the same trace with this program:

```
int s = 10;
for (int i = 10; i < 14; i = i + 1)
{
    s = s + i;
}
std::cout << s << std::endl;
```

What is the output? Now run the program with your C++ compiler and verify.

Exercise -1.8.79. Do the same trace with this program:

```
int s = 0;
for (int i = 3; i < 10; i += 2)
{
    s = s + i;
}
std::cout << s << std::endl;
```

What is the output? Now run the program with your C++ compiler and verify.

Exercise -1.8.80. Do the same trace with this program:

```
int s = 0;
for (int i = 0; i < 10; i = i + 1);
    s = s + i;
std::cout << s << std::endl;
```

What is the output (or is there an error)? Now run the program with your C++ compiler and verify.

Using `for`-loop to compute sums

Look at the following program:

```
int sum = 0;
int i = 1;

sum = sum + i;
i = i + 1;

sum = sum + i;
i = i + 1;

sum = sum + i;
i = i + 1;

sum = sum + i;
i = i + 1;

sum = sum + i;
i = i + 1;

sum = sum + i;
i = i + 1;

sum = sum + i;
i = i + 1;

sum = sum + i;
i = i + 1;

std::cout << sum << std::endl;
```

OK. ... looks like a long program ... but most of it repeats.

Now look at this one:

```
int sum = 0;
for (int i = 1; i <= 10; i = i + 1)
{
    sum = sum + i;
}
std::cout << sum << std::endl;
```

Do you see that they do the same thing? Make sure you trace both programs by hand and see that the actual computations are the same.

Exercise -1.8.81. Modify the `i = i + 1` so that the plus augmented operator is used. (Don't peek ahead!)

Exercise -1.8.82. You can get C++ to tell us what's happening in the for-loop by pausing the loop like this:

```
int sum = 0;
for (int i = 1; i <= 3; i++)
{
    char c;
    std::cout << "enter a char to continue ...";
    std::cin >> c;

    sum = sum + i;
    std::cout << i << ' ' << sum << std::endl;
}
std::cout << sum << std::endl;
```

Make sure you try this. This is a **very basic technique** for slowing down a loop so that you can follow what's happening and do **debugging**.

Exercise -1.8.83. Rewrite the program to use an augmented operator. (Yes, there's one other place where you can use an augmented operator.)

Exercise -1.8.84. Modify the above program to compute the sum of all integers from 1 to 100. You want to remove the pause in the middle of the program:

```
int sum = 0;
for (int i = 1; i <= 3; i++)
{
    char c;
    std::cout << "enter something to continue ...";
    std::cin >> c;

    sum = sum + i;
    std::cout << i << ' ' << sum << std::endl;
}
std::cout << sum << std::endl;
```

Exercise -1.8.85. Modify the above program to compute the sum of squares from 1^2 to 5^2 . Verify your program by calculating this sum with a calculator. Modify your program to compute the sum of squares from 1^2 to 100^2 . Of course the number is going to be really huge ... you do not want to check this one with your calculator!

Exercise -1.8.86. Modify the above program to compute the sum of reciprocals of squares from $1/1^2$ to $1/5^2$. (What type of values are you adding to `sum`? What should the type of `sum` be?) Verify your program by calculating this `sum` with a calculator. Modify your program to compute the sum of reciprocals of squares from $1/1^2$ to $1/100^2$. Compare to the previous exercise and note that the sum stabilizes quickly since the terms you're adding to the sum shrink very quickly. In this case we say that the sum **converges**. You can try to modify your program to add up to $1/1000000^2$. You'll find that it won't change the sum much.

Exercise -1.8.87. Modify the above program to compute

$$1/1 + 1/2 + 1/3 + \dots + 1/100$$

In this case, the sum seems to stabilize since each term you add to the sum shrinks. However the sum is actually NOT stabilizing. It is actually growing ... but very slowly. You can try to compute the sum up to $1/1000000$.

In general the computation of a sum using a loop looks like this:

```
sum = 0
for i running from a, a+1, a+2, ..., b:
    sum = sum + (some term)
```

For instance:

```
sum = 0;
for (int i = 1; i <= 100; ++i)
{
    sum += i;
}
```

In this case the term to add to `sum` is `i`. If you want to sum squares you can do this:

```
sum = 0;
for (int i = 1; i <= 100; ++i)
{
    sum += i * i;
}
```

or this:

```
sum = 0;
for (int i = 1; i <= 100; ++i)
{
    int term = i * i;
    sum += term;
}
```

Of course for such cases, the term can be computed from `i`. There's no reason why the term cannot be supplied by, for instance, a user. Here's one that adds integers specified by a user:

```
int num_accts;
std::cout << "how many bank accounts do you have? ";
std::cin >> num_accts;

double sum = 0;
for (int i = 1; i <= num_accts; ++i)
{
    std::cout << "enter amt in bank acct #"
               << i << "? $";
    double amt;
    std::cin >> amt;
    sum += amt;
    std::cout << "so far ... you have $"
               << sum << std::endl;
}
std::cout << "total ... you have $"
           << sum << std::endl;
```

Exercise -1.8.88. Write a program that first asks a user how many houses he/she has. Next the program asks the user for the number of rooms in each house. Finally, the program prints the total number of rooms he/she has for all the properties owned by the user.

Exercise -1.8.89. Write a program that first asks a user how many houses he/she has. Next the program asks the user for the number of rooms in each house and the average square footage for the rooms in that house. Finally, the program prints the total square footage of all the rooms across all the properties owned by the user.

Using for-loop to compute products

Exercise -1.8.90. Trace this by hand:

```
int product = 1;
for (int i = 1; i <= 5; i++)
{
    product *= i;
    std::cout << i << ' ' << product << std::endl;
}
std::cout << product << std::endl;
```

What is the output (or is there an error)? Now run the program with your C++ compiler and verify.

By the way, the product of all positive integer from 1 to 5 (i.e. $1 \times 2 \times 3 \times 4 \times 5$) is called the **factorial** of 5. The factorial of 10 is just

$$1 \times 2 \times 3 \times 4 \times 5 \times 6 \times 7 \times 8 \times 9 \times 10$$

In general, the factorial of n is such an important quantity that we have a shorthand notation for it: The factorial of n is written $n!$. So

$$10! = 1 \times 2 \times 3 \times 4 \times 5 \times 6 \times 7 \times 8 \times 9 \times 10$$

Note that the factorial of 0 is defined to be 1.

$$0! = 1$$

Study the above code. Do you see that it computes the factorial of 5?

The factorial appears in Computer Science, Math, Physics, Chemistry, Finance, etc. So you bet it's important.

Exercise -1.8.91. Compute by hand the factorial of n for $n = 0, 1, 2, 3, 4, 5, 6$. Use your calculator to verify your work. Using C++, compute the factorial of 10.

Exercise -1.8.92. Why did I initialize `product` with 1 and not 0? What if you run this program instead?


```
int product = 0;
for (int i = 1; i <= 5; i++)
{
    product *= i;
    std::cout << i << ' ' << product << std::endl;
}
```

And what if I initialize the `product` to 2? (See why it has to be initialized to 1 now?)

Exercise -1.8.93. Write a program that prompts the user for `n` and prints the product of **odd** integers greater than 0 and less than `n`.

Exercise -1.8.94. Write a program that prompts the user for `n` (an integer value at least 0) and prints 2-to-the-power of `n`. For instance, if the user enters 0, the program prints 1; if the user enters 3, the program prints 8; if the user entered 10, the program prints 1024; Etc.

Exercise -1.8.95. Write a program that prompts the user for `x` (a `double`) and then `n` (an `int` value at least 0) and prints `x`-to-the-power of `n`. Check your program with several cases, testing it either by hand or by using a calculator.

Exercise -1.8.96. Write a program that prompts the user for `x` (a `double`) and then `n` (an `int` value ***which can be negative***) and prints `x`-to-the-power of `n`. Check your program with several cases, testing it either by hand or by using a calculator.

Exercise -1.8.97. Besides the factorial function `n!`, there's another quantity that is extremely important in Computer Science, Math, Physics, and you-name-it. It's called the `n`-choose-`r` function. It's sometimes written `C(n,r)`. It depends on the factorial. Let `n` and `r` be positive integers where `r` is at most `n`. Then `n`-choose-`r` is given by

$$n! / (r! (n - r)!)$$

`n`-choose-`r` is the total numbers of ways to choose `r` things from `n` distinct things. For instance, suppose you are an IT manager and have a team of 40 programmers, and you have to form a team of 5

software engineers. The total numbers of ways to form this team is 40-choose-5, i.e.,

$$40! / (5! \times (40 - 5)!) = 40! / (5! \times 35!)$$

However 40! is too huge to compute. Use your program to compute 5-choose-2. Verify the correctness of your program by doing this computation by hand. Here are 5 symbols:

a, b, c, d, e

Write down all possible selections of 2 symbols. Let me begin for you:

a, b

a, c

Note that the selection of a, b is considered the same as the selection of b, a. So do not include b, a. Likewise only include b, e and not e, b.

Using for-Loop to compute min/max

Recall that if you have int variables a, b, c, d, and you want to compute the minimum, you can do this (pseudocode only!):

```
min = a
if b < min:
    min = b
if c < min:
    min = c
if d < min:
    min = d
```

In this case min is an integer variable. (Re-study your notes on the if statement if you don't recall this!!!)

The idea of course works for doubles too. Of course the min variable must be declared to be a double variable. Make sure you understand the above!

The idea for the computation of the maximum is similar.

Now the above sure looks like a loop. If the values are not stored in variables a, b, c, d but for instance is supplied by the user (so we need to do `std::cin`), then we have a very compact way to compute the minimum of 4 values supplied by the user. Instead of doing

```
prompt user for a
min = a

prompt user for b
if b < min:
    min = b

prompt user for c
if c < min:
    min = c

prompt user for d
if d < min:
    min = d

print min
```

we can do this:

```
prompt user for x
min = x

// 3 more values
for i = 0, 1, 2:
    prompt user for x
    if x < min:
        min = x

print min
```

(user inputs are all stored using x).

The C++ code is then:

```
int x;
int min;

std::cin >> x;
min = x;

// 3 more values, so i = 0, 1, 2 works.
for (int i = 0; i < 3; i++)
{
    std::cin >> x;
    if (x < min)
    {
        min = x;
    }
}
std::cout << min << std::endl;
```

Not only that, this code can be EASILY modified to compute the minimum of 10 values:

```
int x;
int min;

std::cin >> x;
min = x;

// i = 0, 1, 2, 3, 4, 5, 6, 7, 8 --- 9 more values
for (int i = 0; i < 9; i++)
{
    std::cin >> x;
    if (x < min)
    {
        min = x;
    }
}
std::cout << min << std::endl;
```

The old method without loops would take a lot more time to write!

Not only that, the above method works when you allow the user to specify how many values the user wants to input!!

```
int x;
int min;
int n; // number of values for the min computation

std::cin >> n;
std::cin >> x;
min = x;

for (int i = 0; i < n - 1; i++)
{
    std::cin >> x;
    if (x < min)
    {
        min = x;
    }
}

std::cout << min << std::endl;
```

As you can see, the loop is indispensable!!!

Exercise -1.8.98. Write a program that prompts the user for n and then prompts the user for n double values and then prints the maxi-

mum of the values enters by the user.

Exercise -1.8.99. Write a program that prompt the user for n and then prompts the user for n double values and then prints the minimum of the **absolute values** of the values entered by the user. Recall from MATH104 that the absolute value of x is just x without the negative sign. So for instance

$$|42| = 42$$

$$|-24| = 24$$

The absolute value functions were mentioned in the set of notes on doubles. For your convenience, here's the information again. For C++, if you need the absolute value of an integer value this is how you do it:

```
#include <iostream>
#include <cstdlib>

int main()
{
    int x = 42, y = -24;
    std::cout << abs(42) << '\n';
    std::cout << abs(-42) << '\n';
    std::cout << abs(x) << '\n';
    std::cout << abs(y) << '\n';
    return 0;
}
```

If you need to compute the absolute value of doubles or floats you do this:

```
#include <iostream>
#include <cmath>

int main()
{
    int x = 4.2, y = -2.4;
    std::cout << fabs(42) << '\n';
    std::cout << fabs(-42) << '\n';
    std::cout << fabs(x) << '\n';
    std::cout << fabs(y) << '\n';
}
return 0;
```

The boolean condition

Here's one of our first few programs:

```
std::cout << "entering the for-loop ...\n";
for (int i = 10; i > 0; i--)
{
    std::cout << "entered the block ...\n";
    std::cout << "i: " << i;
    std::cout << "exiting the block ...\n";
}
std::cout << "done with the for-loop ...\n";
```

Look at this boolean expression:

```
std::cout << "entering the for-loop ...\n";
for (int i = 10; i > 0; i--)
{
    std::cout << "entered the block ...\n";
    std::cout << "i: " << i;
    std::cout << "exiting the block ...\n";
}
std::cout << "done with the for-loop ...\n";
```

One of the most common misconceptions about the for-loop (and actually applies to all loops) is that many first-time programmers think that if the boolean expression

$$i > 0$$

is false at **any point** in the loop, you will break you out of the for-loop immediately. That's **NOT TRUE**. Look at the picture on page 12 again. The check on whether to break out of the loop occurs at a specific point. If I do this instead:

```
std::cout << "entering the for-loop ...\n";
for (int i = 10; i > 0; i--)
{
    std::cout << "entered the block ...\n";
    std::cout << "i: " << i;
    i = 0;
    std::cout << "exiting the block ...\n";
}
std::cout << "done with the for-loop ...\n";
```

The program would still execute the last statement in the body of the for-loop:

```
std::cout << "exiting the block ... n";
```

because the boolean check comes after this print statement.

MAKE SURE YOU REMEMBER THAT!

Generating Equally Spaced Points on a Line

In the next two sections, we will generate points in a range which are equally spaced. This is what I mean:

Suppose I tell you that I want 5 equally spaced points on the interval $[0, 4]$ that includes the end points 0 and 4. You would say:

0, 1, 2, 3, 4

There are 5 values. They are equally spaced: consecutive values differ by 1. The first value 0 is the left endpoint of $[0, 4]$ and the last value 4 is the right endpoint of $[0, 4]$.

What if I ask you for 3 equally spaced points on $[0, 4]$ including both end points? Well this means that you want 3 values, say we call them

x_0, x_1, x_2

where x_0 is 0 and x_2 is 4. What about x_1 ? x_1 is right in the middle. So it must be

$$x_1 = (x_2 - x_0) / 2 = 2$$

So the values are

0, 2, 4

Great! Done!

What if I ask you for 4 equally points in $[0, 4]$? Let's call them

x_0, x_1, x_2, x_3

Of course

$$x_0 = 0 \quad x_3 = 4$$

What about x_1 and x_2 ? Well since they are equally spaced, let's say the gap between x_0 and x_1 is d . Since the points are equally spaced, the gap is of course also the gap between x_1, x_2 and also x_2, x_3 . There are altogether 3 gaps and they add up to 4. So

$$d = 4/3$$

(as a real number, i.e., double. The above is math, not C++) In other words:

$$d = 1.3333.... \text{ (up to 4 decimal places)}$$

This mean that

$$\begin{aligned}x_0 &= 0 \\x_1 &= 0 + d = 1.3333.... \\x_2 &= x_1 + d = 2.6666... \\x_3 &= x_2 + d = 4\end{aligned}$$

What if I want **7** points in the same interval? Say the points are

$$x_0, x_1, x_2, x_3, x_4, x_5, x_6$$

There are **6** gaps. Call the gap d . The gaps add up to 4. Therefore

$$6d = 4$$

i.e.,

$$d = 4/6 = 0.6666...$$

and the values of the points are

$$\begin{aligned}x_0 &= 0 \\x_1 &= x_0 + d \\x_2 &= x_1 + d \\x_3 &= x_2 + d \\x_4 &= x_3 + d \\x_5 &= x_4 + d \\x_6 &= x_5 + d\end{aligned}$$

Writing this as a C++ program I get

```
double x = 0;
double d = double(4) / 6
for (int i = 0; i < 7; ++i)
{
    std::cout << x << '\n';
    x += d
}
```

Run this and check that you do get 7 points which are equally spaced.

Exercise -1.8.100. Modify the following code to generate equally spaced points in $[0, 4]$ if the number of points is n .

```
int n;
std::cin >> n;
double x = 0;
double d = double(4) / 6;
for (int i = 0; i < 7; ++i)
{
    std::cout << x << '\n';
    x += d
}
```

Test your program for different values of n .

Exercise -1.8.101. If the interval is not $[0, 4]$ but $[0, b]$ and the number of points is n , then modify the following code as appropriate:

```
int n = 0;
double b = 0.0;
std::cin >> b;
std::cin >> n;

double x = 0;
double d = double(4) / 6;
for (int i = 0; i < 7; ++i)
{
    std::cout << x << '\n';
    x += d
}
```

Test your code thoroughly.

Exercise -1.8.102. If the interval is $[a, b]$ and the number of points is n , then modify the following code as appropriate:

```
int n = 0;
double a = 0.0, b = 0.0;
std::cin >> a >> b;
std::cin >> n;

double x = 0;
double d = double(4) / 6;
for (int i = 0; i < 7; ++i)
{
    std::cout << x << '\n';
    x += d
}
```

Test your code thoroughly. [Hint: Move the interval so that the left endpoint is 0. The interval becomes $[0, b - a]$. Get the points using the previous exercise. Now move the points back to $[a, b]$.

WARNING ... INCOMING SPOILER!!!

Here's the answer to the above. If you are generating n equally spaced points in the interval $a, b]$ (and the left and right end points are included), the code is

```
double x = a;
double d = double(b - a) / (n - 1);
for (int i = 0; i < n; ++i)
{
    std::cout << x << '\n';
    x += d
}
```

Note that because of rounding errors (remember that doubles and floats are not exact!), the last point might not be exactly the value b . You can stop the for-loop earlier and manually use b for the last point:

```
double x = a;
double d = double(b - a) / (n - 1);
for (int i = 0; i < n - 1; ++i)
{
    std::cout << x << '\n';
    x += d;
}

x = b;
std::cout << x << '\n';
```

In the next couple of sections, we will be using these equally spaced

points to do some computations. So in the general case, the code will look something like this:

```
double x = a;
double d = double(b - a) / (n - 1);
for (int i = 0; i < n - 1; ++i)
{
    {... do something with x ...}
    x += d;
}
x = b;
... do something with x ...
```

Sums again: Area computation

I'm going to use the `for`-loop to compute a certain sum. This time it's going to be the area under a curve. The computation of areas is extremely important in many areas of science.

The idea is actually very simple: you need to know how to compute the area of rectangles and you need to be able to add lots of these areas.

The idea is pretty simple and illustrates the power of being able to do perform repetitions quickly.

First you draw the graph of the function, for example, $y = f(x) = x^2$, the standard parabola):

Suppose you want to compute the area under this curve and above the x-axis and from $x = 2$ to $x = 7$:

You can make an approximation by computing the areas of the rectangles with

$x=2$ to $x=3$ with height $f(2)$
 $x=3$ to $x=4$ with height $f(3)$
 $x=4$ to $x=5$ with height $f(4)$
 $x=5$ to $x=6$ with height $f(5)$
 $x=6$ to $x=7$ with height $f(6)$

There are altogether 5 rectangles. Draw these rectangles into the graph above.

Note that for each rectangle, the base has width 1. Also, for each rectangle, I am using the left endpoint on the width to determine the height. The areas of these rectangles is smaller than the area under the parabola from $x = 2$ to $x = 7$.

Exercise -1.8.103. Write a `for`-loop to compute the sum of the above rectangles. Write down the sum of areas. We'll need it for comparison later.

Now we "move" the sum of areas of our rectangles towards the parabola

in the following way: we will use **more** rectangles. Previously the base of the rectangles had a width of 1. Now I'm going to use rectangles with base of length 0.5. Altogether we now have 10 rectangles:

$x=2.0$ to $x=2.5$ with height $f(2.0)$

$x=2.5$ to $x=3.0$ with height $f(2.5)$

$x=3.0$ to $x=3.5$ with height $f(3.0)$

$x=3.5$ to $x=4.0$ with height $f(3.5)$

$x=4.0$ to $x=4.5$ with height $f(4.0)$

$x=4.5$ to $x=5.0$ with height $f(4.5)$

$x=5.0$ to $x=5.5$ with height $f(5.0)$

$x=5.5$ to $x=6.0$ with height $f(5.5)$

$x=6.0$ to $x=6.5$ with height $f(6.0)$

$x=6.5$ to $x=7.0$ with height $f(6.5)$

Draw these 10 rectangles into the above graph.

Do you see that the total area of these rectangles are closer to the actual area under the parabola than the one from our first approximation?

Do you see that, intuitively, the total area of the rectangles gets closer and closer to the area under the parabola from $x = 2$ to $x = 7$ as the number of rectangles used increases? Here's the case where the length of the base of the rectangles is 0.25:

and here's the case where the length is 0.125

Using the code that computes equally spaced point, the sum of the areas of these rectangles can thus be computed as follows:

```
double a = 2;
double b = 7;
int n = 11; // 10 rects mean 11 points
double sum = 0;
double x = 0;
double d = double(b - a) / (n - 1);
for (int i = 0; i < n - 1; ++i)
{
    sum += d * (x * x);
    x += d;
}

// Why do we NOT need to execute the following?
// x = b;
// sum += d * (x * x);
```

Exercise -1.8.104. Compute area when 20 rectangles are used.

Exercise -1.8.105. Now do the same where you use 100 rectangles.

Now write a program that prompts the user for the number of rectangles! Say we call this variable n . Run the program with $n = 1000$ rectangles!!! You bet the value computed by your program is extremely close to the real area of the parabola. Of course you wouldn't add the areas of 1000 rectangles by hand – it would take too long! That's what a program is for – if you know C++!!!

Exercise -1.8.106. If you know some Calculus 1, you know that the area under the parabola from $x = 2$ to $x = 7$ is in fact exactly

$$(1/3)(7 * 7 * 7 - 2 * 2 * 2)$$

Use your calculator (or C++) to compute this value and compare against the value obtained by your program with $n = 1000$ rectangles.

Exercise -1.8.107. The above prompts the user for n , the number of rectangles to use for the area computation using a for-loop. Write a nested for-loop where the outer for-loop supplies values of n running from 1000 to 1000000 by increments of 1000. The pseudocode looks

like this:

```
for n = 1000, 2000, 3000, ..., 1000000:  
compute and print the area using n rectangles
```

Exercise -1.8.108. Modify the above program so that instead of computing the area under the parabola from $x = 2$ to $x = 7$, the program prompts the user for a , b , n and computes the area under the parabola from $x = a$ to $x = b$ using n rectangles.

Exercise -1.8.109. What is the area under the curve $y = x^2$ from $x=0$ to $x=10$?

The computation of areas under a curve is important. For instance in Physics, the work done by a force F is the area under the force-displacement graph where you plot the force acting on an object and the displacement of the object. By Newton's law of universal gravitation, we know the gravitational force between two objects which are apart from each other at a fixed distance. So if a rocket wants to move away from earth, you can compute the work done moving the rocket to a distance from the surface of earth to a far away point in space. Knowing the work to be done moving the rocket from earth through a required distance, you can then compute the amount of chemical fuel that is needed to combust in order to convert the chemical energy into required work done. Of course there are many other applications of the computation of areas.

Root finding: An inefficient method

Another very common problem in Math and Sciences is finding solution(s) to an equation such as finding solutions to quadratic equations:

$$2 * x * x + 4 * x - 3 = 0$$

This is easy since we have the quadratic equation formula. But this is highly non-trivial if the equation is more complex such as a degree 5 equation:

$$2 * x * x * x * x * x + 4 * x * x - 3 * x + 10 = 0$$

or even one with trigonometric, logarithmic, and/or exponential functions!!!

For instance, what if I want to find a solution to

$$3 * x * x - 10 * x * \sin(x) - 500 = 0$$

in the range [0, 20]? You don't recall a solution to such an equation from your math classes right?!?

Since we don't have a formula to use, we just have to plot the graph and visually see where the function touches the x-axis. In the same way, we can compute the values of the above expression

$$3 * x * x - 10 * x * \sin(x) - 500$$

for many many many many many values in the range [0, 20] and see which value of x will give a value of

$$3 * x * x - 10 * x * \sin(x) - 500$$

that is very close to 0. For instance we can test 1,000,000 values in the range [0,20]. If we do this by hand, we would go crazy. But ...

We know C++!!!!

All we need to do is to run a for-loop of 1,000,000 values in the range [0,20] and find the value of x that will get use as close as possible to the y-axis. Close to the y-axis means

$$|f(x)|$$

is close to zero. In other words, if you're finding the point closest to the y-axis it means that you need to find the smallest $|f(x)|$ for the 1,000,000 x values in the range.

Exercise -1.8.110. Using a `for`-loop, find a value for x from 1 to 5 such that $x \sin(x) + 1$ is as close as possible to the x-axis.

Maximum and minimum of functions

Using the idea of generating lots and lots of equally spaced points on an interval, you can very quickly compute the approximate maximum and minimum of functions. Note that the max or min is only an approximation since your points might miss the actual maximum and minimum.

Exercise -1.8.111. Write a program to compute and print the maximum value of the sine function on the interval $[0, 10]$ using 1,000,000 equally spaced points. The sine function is already mentioned in the notes on floating point values.

Exercise -1.8.112. Modify the above program so that it prints the value of x where the maximum occurs.

Exercise -1.8.113. Compute and print the maximum and minimum value of the function

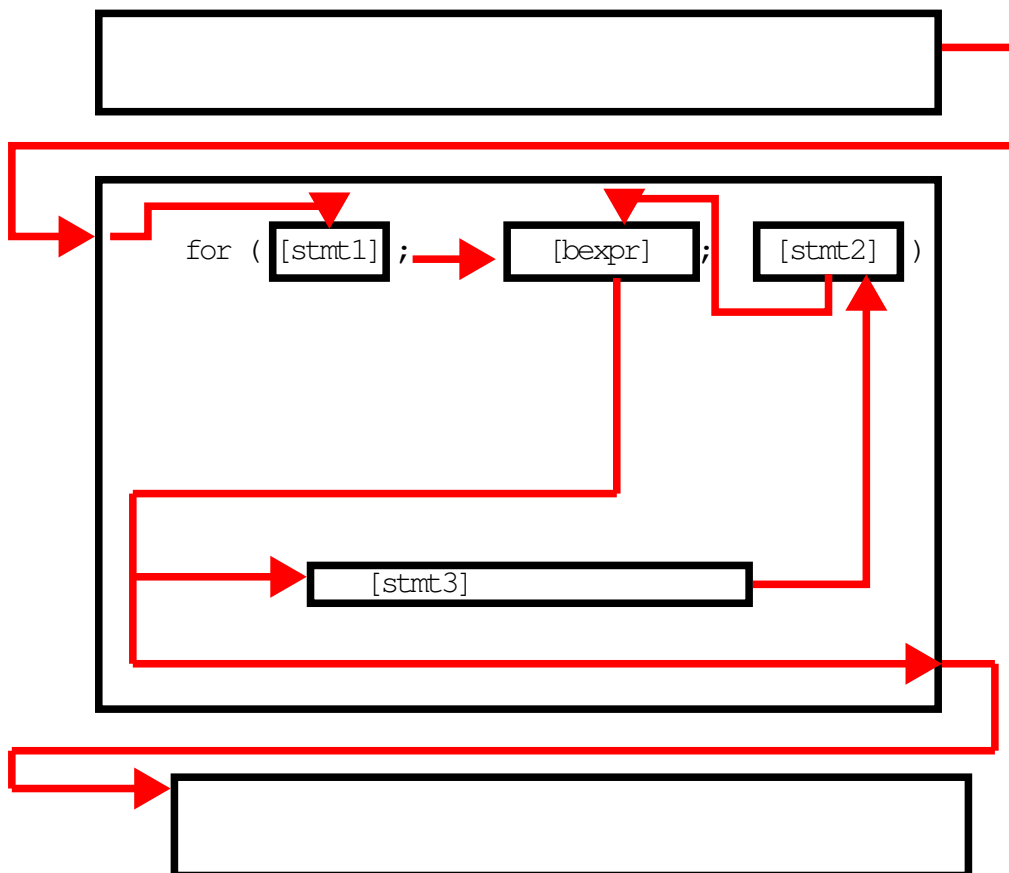
$$f(x) = x \sin(x)$$

on the interval $[0, 20]$ using 1,000,000 points. When you're done, modify the program to also print the x value where the maximum and minimum occurs.

11. for Loops: Part 2

This is a continuation of our discussion of for-loops ...

The three parts of the for-loop



Look at the structure of the for-loop:

The statement looks like this:

```
for ([stmt1]; [bexpr]; [stmt2])
    [stmt3]
```

where [stmt1], [stmt2], [stmt3] are statements; [stmt1] and [stmt2] are written without a ';' since it's included in the above notation.

[stmt1] is executed **first** and it's executed exactly **once**.

After executing `[stmt1]`, `[bexpr]` is evaluated.

If the resulting value is `true`, `[stmt3]` is executed. After that `[stmt2]` is executed. And after that we repeat by evaluating `[bexpr]`.

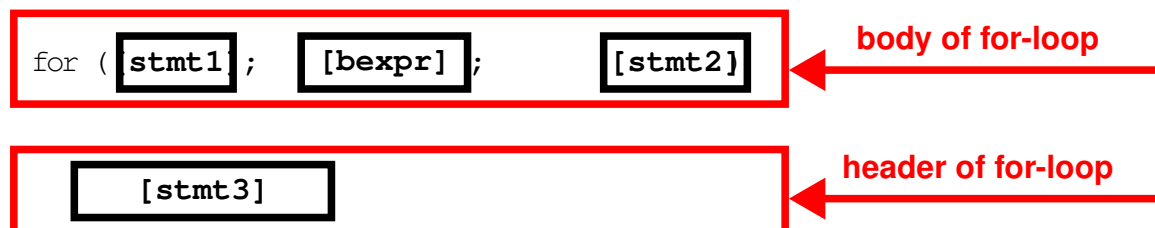
If the resulting value of the `[bexpr]` is `false`, the program exits the `for`-loop and executes the statement after the `for`-loop.

By the way, sometimes `[stmt1]` is sometimes called the **initialization** statement of the `for`-loop while `[stmt2]` is called the **update** statement. For the following code:

```
for (int i = 0; i < 100; i++)
{
    std::cout << i << std::endl;
}
```

`i` is called the **index or counter variable** of the `for`-loop.

Two final terms:



Leaving things out

Refer to the diagram from the previous section.

Actually it turns out that you can leave out `[stmt1]`. Try this

```
for (int i = 0; i < 5; i++)
{
    std::cout << "i have " << i
               << " head(s) "
               << std::endl;
}
```

And now modify it as follows:

```
int i = 0;
for (; i < 5; i++)
{
    std::cout << "i have " << i
               << " head(s) "
               << std::endl;
}
```

You can also leave the third part out as well!!!

```
int i = 0;
for (; i < 5;)
{
    std::cout << "i have " << i
               << " head(s) "
               << std::endl;

    i++;
}
```

What if the second part is blank? Try this:

```
int i = 0;
for (;;)
{
    std::cout << "i have " << i
               << " head(s) "
               << std::endl;

    i++;
}
```

(Pretty gross, isn't it? ...)

This tells you that if the boolean expression is missing, C++ will assume you want the boolean value to be **true**. Remember what I said before: you have to pay attention whenever C++ does something for you automatically.

By the way, the above program is executing an **infinite loop** (because it doesn't stop). This format of the for-loop is sometimes used in writing games, although we'll see that there's another type of loop called the **while-loop** that is frequently used instead. Although in the above

```
for (;;)
...
```

it seems like you can't get out of the loop. But, in fact, **you can get out of the loop in another way**. We'll talk about it later.

Why scopes?

Compare the following programs:

```
int i = 0;
int sum = 0;

for (i = 0; i < 10; i++)
{
    sum += i;
}

std::cout << sum << std::endl;
```

```
int sum = 0;

for (int i = 1; i < 11; i++)
{
    sum += i;
}

std::cout << sum << std::endl;
```

Their goals are the same: To compute and print the sum of integers from 1 to 10.

The only difference is in the scope of variable `i`. The scope of `i` is a lot smaller in the second program.

If you think about it, the variable `i` is sort of like a “scratch” variable when compared to `sum`. You don’t really need it once the computation of `sum` is completed. When you read the second program, you can very easily deduce the fact that `i` is a “scratch” variable from its scope: it exists only in the `for`-loop.

A complex problem is solved by breaking the problem into smaller and simpler subproblems. Once that’s done you solve the simpler subproblems and put these subsolutions together to solve the big problem. One should be careful of the unintended effects of a sub-solution – a small piece of the solution overall – on another.

Here’s an example. This program prints the sum from 1 to 10 and then the sum from 1 to 100:

```
int i;
int sum = 0;

for (i = 1; i < 11; i++)
{
    sum += i;
}

// Print sum from 1 to 10
std::cout << sum << std::endl;

// Continue the summing process
for (; i < 101; i++)
{
    sum += i;
}

std::cout << sum << std::endl;
```

If you're working on a project with someone and you both have agreed that *i* is a scratch variable, he might use it like this in a computation of a formula involving *a*, *b*, *c*, *d*, *e*, *f*

```
i = a + b;
result = i + c + d + e + f;
```

He might just insert this into your code:

```
int i;
int sum = 0;

for (i = 1; i < 11; i++)
{
    sum += i;
}

std::cout << sum << std::endl;

i = a + b;
result = i + c + d + e + f;

for (; i < 101; i++)
{
    sum += i;
}

std::cout << sum << std::endl;
```

Do you see how his work has now destroyed your computation?

In general you want to have as little “communication” as possible between your subsolutions when they are meant to be independent.

One way to achieve that is to use small scopes. Look at this:

```
int i;
int sum = 0;

for (int i = 1; i < 11; i++)
{
    sum += i;
}

std::cout << sum << std::endl;

i = a + b;
result = i + c + d + e + f;

for (int i = 1; i < 101; i++)
{
    sum += i;
}

std::cout << sum << std::endl;
```

There are really 3 i's. The first is outside both for-loops. The second is created only when you enter the first for-loop; this is destroyed when you exit the first for-loop. The third is created only when you enter the second for-loop; this is destroyed when you exit the second for-loop. They all live in different “places”.

Even more examples

Now for examples where the body of the `for`-loop is more complicated.

Of course you can put `if`-statements in the `for`-loop. Run this:

```
int sum = 0;
for (int i = 1; i <= 10; ++i)
{
    if (i == 3 || i > 7)
    {
        sum += i;
        std::cout << i << " " << sum << std::endl;
    }
}
```

Exercise -1.8.114. Modify the above program by adding an `else` statement to the `if` statement that prints `i` and " is not included".

Exercise -1.8.115. Write a program that randomly generates 20 integers between 0 and 3.

Exercise -1.8.116. Write a program that randomly generates 20 integers between 10 and 20 and continually sums up all of them, printing the sum in each iteration.

Mixing two for-loops

Of course you can put a `for`-loop in a `for`-loop to get a double `for`-loop:

```
for (int i = 0; i < 5; i++)
{
    std::cout << "i:" << i << std::endl;
    for (int j = 0; j < 3; j++)
    {
        std::cout << " j:" << j << std::endl;
    }
}
```

Read the output very carefully!!!

Exercise -1.8.117. Write a program that has a double `for`-loop and prints this

```
3
  1 2 3
4
  1 2 3
5
  1 2 3
```

Exercise -1.8.118. Write a program that has a double `for`-loop and prints this

```
3
  0 1 2 3
4
  0 1 2 3 4
5
  0 1 2 3 4 5
```

Exercise -1.8.119. Write a program that has a double `for`-loop and prints this

```
3
  0 1 2
4
  0 1 2 3
5
  0 1 2 3 4
```

Exercise -1.8.120. Write a program that has a double for-loop and prints this

```
3
  0 1 2 3 4 5 6
4
  0 1 2 3 4 5 6 7 8
5
  0 1 2 3 4 5 6 7 8 9 10
```

Exercise -1.8.121. Write a program that has a double for-loop and prints this

```
3
  3 4 5 6
4
  4 5 6 7 8
5
  5 6 7 8 9 10
```

Exercise -1.8.122. Write the following program that prints a calendar month. The program prompts the user for three integers: the month, the year, and the day-of-week for the first day of the month (with 0 representing Sunday, 1 representing Monday, etc.),. For instance, if the user entered

```
3 2008 6
```

It means that he/she wants the calendar for March 2008 and the first day of the month is a Saturday. The output is

```
March 2008
-----
Su Mo Tu We Th Fr Sa
                1
 2  3  4  5  6  7  8
 9 10 11 12 13 14 15
16 17 18 19 20 21 22
23 24 25 26 27 28 29
30 31
```

Here's another test case:

3 2008 4

March 2008

```
-----  
  
Su Mo Tu We Th Fr Sa  
          1 2 3  
 4  5  6  7  8  9 10  
11 12 13 14 15 16 17  
18 19 20 21 22 23 24  
25 26 27 28 29 30 31
```

Of course your program should be smart enough to compute the number of days in the month (including leap year cases for the month of February.)

Some ASCII art

Now for some (silly) ASCII art.

Try this:

```
int numSpaces = 0;
std::cout << "How far away is the star? ";
std::cin >> numSpaces;

for (int i = 0; i < numSpaces; i++)
{
    std::cout << ' ';
}
std::cout << '*' << std::endl;
```

And this:

```
int numStars = 0;
std::cout << "How many stars? ";
std::cin >> numStars;

for (int i = 0; i < numStars; i++)
{
    std::cout << '*';
}
std::cout << std::endl;
```


And more ASCII art

It can only get worse ...

I want to draw a square of *. The square has width 5 and height 3.
First I draw a horizontal line:

```
// Draw horizontal line
for (int i = 0; i < 5; i++)
{
    std::cout << '*';
}
```

OK. Now, I just draw the horizontal line 3 times:

```
for (int j = 0; j < 3; j++)
{
    // Draw horizontal line
    for (int i = 0; i < 5; i++)
    {
        std::cout << '*';
    }
}
```

Oops! How come I get a straight line??? Well, of course after drawing a horizontal line, I need to skip my cursor to the next line:

```
for (int j = 0; j < 3; j++)
{
    // Draw horizontal line
    for (int i = 0; i < 5; i++)
    {
        std::cout << '*';
    }
    std::cout << std::endl;
}
```

The boolean condition in the for-loop can be any boolean expression. So instead of drawing 3 lines, you can prompt the user for number of lines to draw instead.

```
int height = 0;
std::cin >> height;
for (int j = 0; j < height; j++)
{
    // Draw horizontal line
    for (int i = 0; i < 5; i++)
    {
        std::cout << '*';
    }
    std::cout << std::endl;
}
```

Exercise -1.8.123. Now modify the program so that it prompts the user for the width of the square. (Obviously you must draw the square with that width).

Unrolling the `for`-loop: How to see the `for`-loop

Sometimes it's hard to come up with the appropriate `for`-loop. In that case it's always easier to **“unroll”** the `for`-loop for several special cases by hand. Unrolling a `for`-loop is just writing down what the program executes without using the `for`-loop. For instance, unrolling the following:

```
for (int i = 0; i < 3; i++)
{
    std::cout << i << std::endl;
}
```

gives us

```
\begin{console}
std::cout << 0 << std::endl;
std::cout << 1 << std::endl;
std::cout << 2 << std::endl;
```

For instance, suppose you want the program to draw this:

```
*
**
***
****
```

if the user enters 4 (this is a triangle). And if he enters 3, the program draws:

```
*
**
***
```

So just take a special case, say when the user enters 3, and write the pseudocode **without the `for`-loop**:

```
draw a line of length 1
draw a line of length 2
draw a line of length 3
```

The pseudocode for height 4 is

```
draw a line of length 1
draw a line of length 2
draw a line of length 3
draw a line of length 4
```

Of course you want to write one program and not two! The program must work for both cases. We must find a way to “combine” them.

The first pseudocode ... **NOT C++!!!** ... can be written:

```
for i = 1, 2, 3:  
    draw a line of length i
```

and the second can be written:

```
for i = 1, 2, 3, 4:  
    draw a line of length i
```

Now they are almost the same! If the height entered by the user is kept in variable *h*, then the pseudocode is

```
for i = 1, 2, ..., h:  
    draw a line of length i
```

AHA! This pseudocode works for both!!!

Of course you already know how to draw a line (of stars) of length *i*. So altogether the pseudocode becomes:

```
for i = 1, 2, ..., h:  
    for j = 1, 2, ..., i:  
        draw a star  
    go to the next line
```

And finally a straightforward translation to C++ gives:

```
for (int i = 1; i <= h; i++)  
{  
    for (int j = 1; j <= i; j++)  
    {  
        std::cout << '*';  
    }  
    std::cout << std::endl;  
}
```

Now we add the icing by prompting the user for *h*:

```
int h = 0;  
std::cin >> h;  
for (int i = 1; i <= h; i++)  
{  
    for (int j = 1; j <= i; j++)  
    {  
        std::cout << '*';  
    }  
    std::cout << std::endl;  
}
```

Unrolling the for-loop: Another example

What about this? If the user enters 3 your program has to draw:

```
***
***
***
```

If the user enters 4 your program has to draw

```
****
****
****
****
```

The pseudocode for the case when the user enters 3 is (without using the for-loop):

```
draw 2 spaces
draw 3 stars
draw 1 space
draw 3 stars
draw 3 stars
```

The trick is to make everything as uniform as possible. So write this:

```
draw 2 spaces
draw 3 stars
draw 1 space
draw 3 stars
draw 0 space
draw 3 stars
```

See the pattern yet? No?

The interleaving of two different things is confusing. Rewrite this as:

```
draw 2 spaces; draw 3 stars
draw 1 space; draw 3 stars
draw 0 space; draw 3 stars
```

What if the user enters 4? The pseudocode becomes

```
draw 3 spaces; draw 4 stars
draw 2 spaces; draw 4 stars
draw 1 space; draw 4 stars
draw 0 space; draw 4 stars
```

How do you combine the above cases into one common pseudocode?

The first is

```
for i = 2, 1, 0:  
    draw i spaces; draw 3 stars
```

The second is

```
for i = 3, 2, 1, 0:  
    draw i spaces; draw 4 stars
```

Now both pseudocodes are almost the same!!! Suppose the value entered by the user is kept in a variable *n*. Then you can (finally!) combine both pseudocodes into one:

```
for i = n-1, ..., 0:  
    draw i spaces; draw n stars
```

Now you add in the details:

```
for i = n-1, ..., 0:  
    // draw i spaces  
    for j = 1, ..., i  
        draw a space  
    // draw n stars  
    for j = 1, ..., n  
        draw a star  
    go to next line
```

And finally a straightforward translation to C++ yields:

```
for (int i = n - 1; i >= 0; i--)  
{  
    for (int j = 1; j <= i; j++)  
    {  
        std::cout << ' '  
    }  
    for (int j = 1; j <= n; j++)  
    {  
        std::cout << '*';  
    }  
    std::cout << std::endl;  
}
```

Adding the prompt (and some comments) we get:

```
int n = 0;
std::cin >> n;
for (int i = n-1; i >= 0; i--)
{
    // Print i spaces
    for (int j = 1; j <= i; j++)
    {
        std::cout << ' ';
    }
    // Print n stars
    for (int j = 1; j <= n; j++)
    {
        std::cout << '*';
    }
    std::cout << std::endl;
}
```

Exercise -1.8.124. Write a program that draws the following figure when the user enters 3

```
***
* *
***
```

and when the user enters 4 it draws

```
****
* *
* *
****
```

[See next page for hints.]

Exercise -1.8.125. Write a program that draws the following figure when the user enters 3

```
 *
***
*****
```

and this when the user enters 4

```
 *
***
*****
*****
```

[See next page for hints.]

WARNING: SPOILERS!!!

Here are the pseudocode (with explanations) for the last two ASCII art problems from the previous section.

Here's the first problem . . .

Exercise -1.8.126. Write a program that draws the following figure when the user enters 3

```
***
* *
***
```

and when the user enters 4 it draws

```
****
*  *
*  *
****
```

and here's the pseudocode with explanation:

```
For n = 4:
    4 stars
    1 star, 2 spaces, 1 star
    1 star, 2 spaces, 1 star
    4 stars
For n = 5:
    5 stars
    1 star, 3 spaces, 1 star
    1 star, 3 spaces, 1 star
    1 star, 3 spaces, 1 star
    5 stars
```

In general, we have:

```
n stars
n-2 of [1 star, n-2 stars, 1 star]
n stars
```

Therefore, the pseudocode is:

```
print n stars
print newline
```

```
for i = 1,..., n-2
    print 1 star
    print n-2 stars
    print 1 star
    print newline
print n stars
```

And after expanding some of the print statements we get:

```
//print n stars
for i=1 ,..., n: print '*'
print newline
for i = 1, ..., n-2
    print '*'
    //print n-2 stars
    for j=1,..., n-2: print '*'
    print '*'
    print newline
//print n stars
for i=1, ..., n: print '*'
```

Exercise -1.8.127. Write a program that draws the following figure when the user enters 3:

```
  *
 ***
*****
```

and this when the user enters 4:

```
  *
 ***
*****
*****
```

When $n = 3$:

```
2 spaces, 1 star
1 space, 3 stars
0 space, 5 stars
```

When $n = 4$:

```
3 spaces, 1 star
2 spaces, 3 stars
1 space, 5 stars
0 space, 7 stars
```

There are two columns of numbers to handle. In other words, each line (which is the body of a loop) has two numbers. The first column is easy - it's just from $n-1$ to 0. So the pseudocode looks like this:

```
for i = n-1 to 0
    i spaces, ??? stars
```

What about the second? The relationship between the second number and i is not so direct. But note that it starts with 1 and always goes up by 2 after each iteration. So the pseudocode looks like

```
numStars = 1
for i = n-1 to 0
    i spaces, numStars stars
    numStars = numStars + 2
```

In more details:

```
numStars = 1
for i = n-1 to 0
    print i spaces
    print numStars stars
    print newline
    numStars = numStars + 2
```

And after expanding some print statements we get

```
numStars = 1
for i = n-1 to 0
    //print i spaces
    for j=1, ..., i: print "
    // print numStars stars
    for j=1, ..., numStars: print '*'
```

```
print newline  
numStars = numStars + 2
```

break

Remember `break` from the `switch` statement? If you execute `break`, you immediately go out of the `switch` block. `break` can also be used in a `for`-loop.

```
First run this:
for (int i = 0; i < 1000; i += 2)
{
    std::cout << "entered body of for-loop \n";
    std::cout << i << std::endl;
    std::cout << "exiting body of for-loop \n";
}
```

Now run this:

```
for (int i = 0; i < 1000; i += 2)
{
    std::cout << "entered body of for-loop\n";
    std::cout << i << std::endl;
    if (i == 100) break;
    std::cout << "exiting body of for-loop\n";
}
```

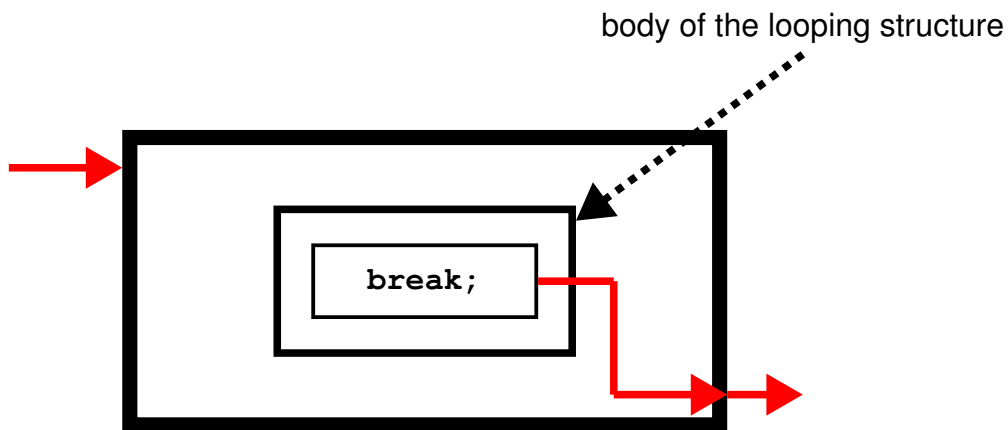
See the point of `break`?

The `break` when executed in a `for`-loop will get out of that `for`-loop. That's all there is to it.

Of course some times you can avoid the `break`. For the above program you can rewrite it as

```
for (int i = 0; i < 101; i += 2)
{
    std::cout << i << std::endl;
}
```

There's a general principle (or good practice) that you should **avoid having too many exit points out of a loop**. Too many exit points make the code difficult to trace.



It's very important to note that the execution of break only exits the current for-loop. Make sure you trace and run try this:

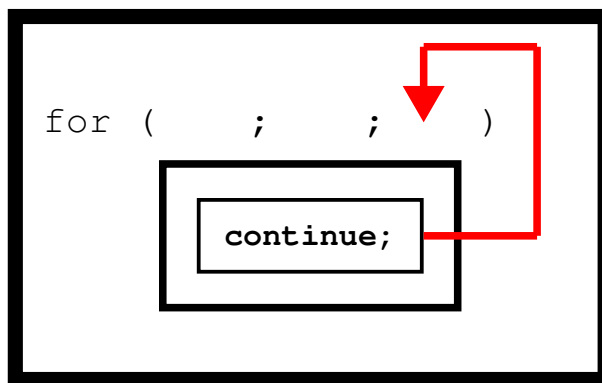
```
for (int i = 0; i < 5; i++)
{
    std::cout << "A: " << i << std::endl;
    for (int j = 0; j < 5; j++)
    {
        std::cout << "B: " << j << std::endl;
        if (j == 1) break;
        std::cout << "C: " << j << std::endl;
    }
    std::cout << "D: " << i << std::endl;
}
```

continue

Another way to control the flow of execution in a for-loop is by using the `continue` statement. Try this and tell me what it does:

```
for (int i = 0; i < 101; i += 2)
{
    std::cout << "top of loop" << std::endl;
    std::cout << i << std::endl;
    if (i > 10) continue;
    std::cout << "bottom of loop" << std::endl;
}
```

A picture tells a thousand words:



Exercise -1.8.128. Write a program that prompts the user for `start`, `end`, and `skip` (altogether there are three integers) and prints the sum of all integers from `start` to `end` except for `skip`. Here's the skeleton

```
int start = 0, skip = 0, end = 0;
std::cin >> start >> end >> skip;
int sum = 0;
for (      ;      ;      )
{
    if (      )      ;
    sum +=      ;
}
std::cout << sum << std::endl;
```

For instance, if `start` is -5, `end` is 10, and `skip` is 7, then your program should compute the sum of integers from -5 to 10 (inclusive) except for 7.

Divisors and primes

Quick review of divisors and primes (See previous notes on integers for more information.)

Let n be a positive integer at least 1 and let d be a positive integer greater than 0. We say that d **divides** n or that d is a **divisor** of n if you can find an integer x such that $dx = n$. (Yes, we've seen this before, in earlier notes and also in the prerequisite math courses.) You already know that the statement

“ d divides n ”

in C++ is

$$n \% d == 0$$

Of course for d to be a divisor of n , d must be at most n .

Exercise -1.8.129. Write a program that prompts the user for n and prints all the (positive) divisors of a given positive integer n . For instance, if the user entered 10 for n , the program should print

```
1
2
5
10
```

If the user entered 11, the program should print

```
1
11
```

Don't forget that when the user entered 1 for n , your program should print only one 1:

```
1
```

Once you're done with your program, modify it so that if the user entered an integer less than or equal to 0, it prints

```
Wrong input. Run again and enter a positive int.
```

A **prime** is a positive integer which is greater than 1 and is divisible by only 1 and itself.

Let's try to get C++ to list primes.

Look at the statement: "A prime is a positive integer which is greater than 1 and is divisible by only 1 and itself."

Let's say the value of the integer is in variable `n`. There are two conditions to check on `n`

- `n` greater than 1
- `n` is divisible by only 1 and itself

The first condition is easy. The second one is tricky. But its not too bad.

Suppose `n` has value 7. We just need to check that 2, 3, 4, 5, and 6 cannot divide 7 to conclude that 7 is a prime.

Hmmm "2, 3, 4, 5, and 6"... sure sounds like a for-loop.

What about "cannot divide"??? Well "`i` divides `n`" is the same as saying `n % i` is zero. So for "`i` does not divide `n`", the remainder when you divide `n` by `i` must be nonzero, i.e. `n % i` is not zero. (Duh.)

Try this:

```
int n = 7;
for (int i = 2; i < n; ++i)
{
    std::cout << i << ' ' << n % i << std::endl;
}
```

Now try this:

```
int n = 25;
for (int i = 2; i < n; i++)
{
    std::cout << i << ' ' << n % i << std::endl;
}
```

AHA! If you do get a nonzero value, it's not a prime and you can stop the check right?

```
int n = 25;
for (int i = 2; i < n; ++i)
{
    std::cout << n % i << std::endl;
    if (n % i == 0) break;
}
```

But ... of course you still need to know if your `n` is a prime!!! You can't

tell from the above code. Why? Because it's possible to get out of the loop in **two** different ways:

- you could have gotten out of the loop because `i` reached `n` (therefore `n` is a prime); or
- `break` was executed (therefore `n` is not a prime)

We need to **disambiguate** the exit condition of the for-loop!!!

To do that, we create a variable that tells us why we are out of the for-loop. Let's call it `flag`. I'll let our program get the value of `n` from us so that it's easier to test the program:

```
int n;
std::cin >> n;
int flag;
for (int i = 2; i < n; ++i)
{
    std::cout << n % i << std::endl;
    if (n % i == 0)
    {
        flag = 1; // flag equals 1 means n not prime
        break;
    }
}
```

But what if `n` is indeed a prime? It would pass all the tests (i.e. the body of the if statement will not execute). In that case `flag` will not be assigned a value!!! We had better give `flag` an initial value:

```
int n = 25;
int flag = 0; // flag equals 0 means n is prime
for (int i = 2; i < n; ++i)
{
    std::cout << n % i << std::endl;
    if (n % i == 0)
    {
        flag = 1; // flag equals 1 means n not prime
        break;
    }
}
// At this point ...
// if flag is 0, then n is prime
// if flag is 1, then n is not prime \\\
```

Let's insert a print statement after the loop:

```
int n = 25;
int flag = 0; // flag equals 0 means n is prime
for (int i = 2; i < n; ++i)
{
    std::cout << n % i << std::endl;
    if (n % i == 0)
    {
        flag = 1; // flag equals 1 means n not prime
        break;
    }
}

std::cout << (flag == 0 ? "prime" : "not prime")
           << std::endl;
```

(Review the ternary operator if you have forgotten about it.)

Go ahead and test your program with a couple of values for `n`.

Note that `flag` only needs to have two possible values because the only purpose of `flag` is to tell us why we exit the loop and there are only two reasons for exiting the for-loop. Although the program works, the variable **name** `flag` can be better. The **purpose** of `flag` is to tell if `n` is a prime. So ...

I will use a **bool** variable **isprime** instead. Why boolean? Because this variable only needs to take on **two possible values**. Of course if there are three exit points in the for-loop, then you should not use a boolean variable.

```
int n = 25;
bool isprime = true;
for (int i = 2; i < n; ++i)
{
    std::cout << n % i << std::endl;
    if (n % i == 0)
    {
        isprime = false;
        break;
    }
}

std::cout << (isprime ? "prime" : "not prime") << std::endl;
```

Let us now remove the print statement in the for-loop:

```
int n = 25;
bool isprime = true;
for (int i = 2; i < n; ++i)
{
    if (n % i == 0)
    {
        isprime = false;
        break;
    }
}

std::cout << (isprime ? "prime" : "not prime")
           << std::endl;
```

Study the above program very carefully. Make sure you see that the choice of the name of the variable `isprime` makes the program easier to understand.

Note that it's possible to remove the `break` statement; study this program very carefully:

```
bool isprime = true;
for (int i = 2; i < n && isprime; i++)
{
    if (n % i == 0)
    {
        isprime = false;
    }
}

std::cout << (isprime ? "prime" : "not prime")
           << std::endl;
```

Note that the loop keeps running as long as

$$i < n \ \&\& \ isprime$$

is `true`. The only way to get out of the for-loop is when the boolean expression

$$i < n \ \&\& \ isprime$$

evaluates to `false` which is when `i` reaches `n` or when `isprime` is `false`.

Exercise -1.8.130. Print all the primes from 2 to 100. (Recall that 1

is not a prime.) The pseudocode is

```
for n running from 2 to 100:
    if n is prime, print n
```

(Don't forget that a C++ int variable can take values up to about 2 billion.) And of course you know how to check "n is prime" using the code I've shown you above. Rework your program so that it prompts the users for int values for int variables a and b and prints the primes from a to b. Run your program, listing the primes in the range from 1,000,000,000 to 2,000,000,000. Observe the speed/performance of your program.

Note that it's obvious that, for instance when you're testing if $n = 87$ is a prime, you need **not** test if 86 is a divisor of $n = 87$. 86 is obviously too big to divide 87!!!

It can be proven that instead of testing i from 2 to $n - 1$, you really only need to test up to the **square root of n** . (I won't prove this. You can take, for instance, a class on cryptography to find out why.)

```
bool isprime = true;
for (int i = 2; i <= sqrt(n) && isprime; ++i)
{
    if (n % i == 0)
    {
        isprime = false;
    }
}

std::cout << (isprime ? "prime" : "not prime")
          << std::endl;
```

(Don't forget: if you want to use the `sqrt` function, you must add `#include <cmath>` at the top of your code. Also don't forget that if you're using MS VS, you might need to convert the value of n to a double, i.e. you should use `sqrt(double(n))` instead of `sqrt(n)`.)

Now, for instance when you test if 1023457 is prime, your program will run i from 2 to 1011 instead of from 2 to 1023456. That's quite a lot of CPU time saved!!!

Exercise -1.8.131. Redo your earlier primes printing program using this improved algorithm. Run your program, listing the primes in the range from 1,000,000,000 to 2,000,000,000. Observe the improved

speed/performance of your program.

But there's still something else you can do. Note that `sqrt(n)` is computed many times, once every time the boolean expression in the for-loop is evaluated. We can compute and store the square root of `n` to avoid recomputation:

```
bool isprime = true;
int sqrtn = sqrt(n);
for (int i = 2; i <= sqrtn && isprime; ++i)
{
    if (n % i == 0)
    {
        isprime = false;
    }
}

std::cout << (isprime ? "prime" : "not prime")
           << std::endl;
```

Exercise -1.8.132. Redo your earlier primes printing program using this improved algorithm. Run your program, listing the primes in the range from 1,000,000,000 to 2,000,000,000. Observe the improved speed/performance of your program.

There's yet another thing we can do. Note that `sqrtn` is used only in the for-loop. We can actually declare `sqrtn` at the initialization part of the for-loop so that the scope of `sqrtn` stays inside the loop:

```
bool isprime = true;
for (int i = 2, sqrtn = sqrt(n); i <= sqrtn && isprime; ++i)
{
    if (n % i == 0)
    {
        isprime = false;
    }
}

std::cout << (isprime ? "prime" : "not prime")
           << std::endl;
```

Note that this does not improve the speed of your program. It simply shrinks the scope of variable `sqrtn`.

Exercise -1.8.133. Write a program that prompts the user for `n` and prints `p` and `p + 2` where `p` and `p + 2` are both primes and at most `n`.

For instance, if the user entered 20 your program should print

```
1. 3 5
2. 5 7
3. 11 13
4. 17 19
```

For instance, note that 5 and 7 are both primes and 7 is $5 + 2$. Note however that 7 and 9 are not printed since 9 is not a prime. These pairs of primes are called “twin primes”. It has been conjectured for more than 150 years that there are infinitely many twin primes. This is one of the most famous open questions in Math. The largest known twin primes (as of 2010) have more than 100000 decimal digits. If you find something bigger than that let me know. Don’t forget that with what you know now, you can’t handle integers significantly larger than 2 billion, i.e. your int variables can’t handle more than 10 digits!!!

Exercise -1.8.134. Write a program that prompts the user for n and prints the longest chain of consecutive composite integers at most n . A composite integer is an integer greater than 1 that is not a prime. For instance, if the user entered 10 for n , then the following are the numbers from 2 to n with the composites underlined:

2, 3, 4, 5, 6, 7, 8, 9, 10

The longest chain of composites is 8, 9, 10. Therefore your program should print

```
8 9 10
```

Brute force search

Computer Science, like Mathematics, seeks to find solutions to problems. One way to look for solutions is through “brute-force” search. This means going over all possible candidates and seeing which ones are the right solutions.

Let’s try one. Suppose you want to find **integer** solution(s) to this equation:

$$x^3 = 729$$

If x is a negative integer, x^3 is negative. So we are definitely only interested in positive integers. Furthermore x must be less than 729 (well ... a lot less!). So we can try $x = 0, 1, 2, \dots, 729$ and see which one satisfies the above equation.

WHOA!!!

That sure looks suspiciously like a for-loop to me ...

Exercise -1.8.135. Write a for-loop with an index variable running from 0 to 729 to find (and print ... of course!) solution(s) to the equation

$$x^3 = 729$$

Well ... the above example is kind of silly since we could have computed the cube root of 729 and see if it’s an integer value ... but what if you have **two** variables in the equation? Suppose you want to solve for positive integers x and y satisfying

$$x^2 + y^2 = 13^2$$

Well first of all each x and y must be at most 13, right? So all you need to do is to check all possible 0, ..., 13 for x and 0, ..., 13 for y . In other words you need to check all the following cases:

$x = 0, y = 0$

$x = 0, y = 1$

$x = 0, y = 2$

...

$x = 0, y = 13$

x = 1, y = 0
x = 1, y = 1
x = 1, y = 2
...
x = 1, y = 13
x = 2, y = 0
x = 2, y = 1
x = 2, y = 2
...
x = 2, y = 13
...
...
...
x = 13, y = 13

That sure looks like a double-nested for-loop to me! Here's the program:

```
for (int x = 0; x <= 13; ++x)
{
    for (int y = 0; y <= 13; ++y)
    {
        if (x * x + y * y == 13 * 13)
        {
            std::cout << x << ', ' << y << '\n';
        }
    }
}
```

Now note that you see some “repetitions”. I’m saying that if $x = a$, $y = b$ is a solution, then $x = b$, $y = a$ is also a solution because in the equation above, you can switch x and y . What if you do not want to include such repetitions? One way would be to ensure $x \leq y$. In other words you try to get your program to run through these values of x, y :

x = 0, y = 0
x = 0, y = 1
x = 0, y = 2
...
x = 0, y = 13
x = 1, y = 1 (note that y starts with 1 and not 0)
x = 1, y = 2
x = 1, y = 3
...

x = 1, y = 13
x = 2, y = 2 (note that y starts with 2 and not 0)
x = 2, y = 3
x = 2, y = 4
...
x = 2, y = 13
...
...
...
x = 13, y = 13

```
for (int x = 0; x <= 13; ++x)
{
    for (int y = x; y <= 13; ++y)
    {
        if (x * x + y * y == 13 * 13)
        {
            std::cout << x << ', ' << y << '\n';
        }
    }
}
```

Of course you realize that $x^2 + y^2 = 13^2$ gives the solution of the right-angle triangle with hypotenuse of length 13 and integer sides. Why not find all such triangles with hypotenuse from 1 to 100??? This means solving

$$x^2 + y^2 = z^2$$

where $1 \leq z \leq 100$.

```
for (int z = 1; z <= 100; ++z)
{
    for (int x = 0; x <= z; ++x)
    {
        for (int y = x; y <= z; ++y)
        {
            if (x * x + y * y == z * z)
            {
                std::cout << x << ', '
                           << y << ', '
                           << z << '\n';
            }
        }
    }
}
```

This prints the sides of all right angle triangles with integer sides where the hypotenuse is between 1 to 100.

Exercise -1.8.136. Can you improve the performance of the above program?

Exercise -1.8.137. Find all integer solutions to the equation

$$x^4 + 2y^4 = 10001$$

Exercise -1.8.138. Here's something from wikipedia.org:

1729 is known as the **Hardy–Ramanujan number**, after a famous anecdote of the British mathematician [G. H. Hardy](#) regarding a hospital visit to the Indian mathematician [Srinivasa Ramanujan](#). In Hardy's words:

“ I remember once going to see him when he was ill at [Putney](#). I had ridden in taxi cab number 1729 and remarked that the number seemed to me rather a dull one, and that I hoped it was not an unfavorable [omen](#). "No," he replied, "it is a very interesting number; **it is the smallest number expressible as the sum of two cubes in two different ways.**"

The quotation is sometimes expressed using the term "positive cubes", as the admission of negative perfect cubes (the cube of a [negative integer](#)) gives the smallest solution as [91](#) (which is a factor of 1729):

$$91 = 6^3 + (-5)^3 = 4^3 + 3^3$$

Of course, equating "smallest" with "most negative", as opposed to "closest to zero" gives rise to solutions like -91, -189, -1729, and further negative numbers. This ambiguity is eliminated by the term "positive cubes".

Numbers such as

$$1729 = 1^3 + 12^3 = 9^3 + 10^3$$

that are the smallest number that can be expressed as the sum of two cubes in n distinct ways have been dubbed [taxicab numbers](#). 1729 is the second taxicab number (the first is $2 = 1^3 + 1^3$). The number was also found in one of Ramanujan's notebooks dated years before the incident.

Find all positive integer solutions to the equation

$$x^3 + y^3 = 1729$$

(Hint: Write a double for-loop.)

Exercise -1.8.139. The above exercise verified that there are exactly two integer solutions. It does not verify that 1729 is the “**smallest number expressible as the sum of two cubes in two different ways**”. To do that you need to solve this:

$$x^3 + y^3 = z$$

Of course you should print out x , y , and z . Look at all the solutions and find the value of z with exactly two integer solutions. Is it 1729? (Or is Ramanujan totally wrong all these years ...)

Exercise -1.8.140. Write a program that prints all positive fractions m/n from 0 to 5 where m and n are positive and at most 10. The fractions need not be reduced or in any particular order or unique.

Brute force search: polynomial factorization

Here's another brute force search program. Let's write a program to factorize a degree 2 polynomial into two polynomials with integer coefficients. For instance

$$x^2 - 1 = (x + 1)(x - 1)$$

Therefore when the user enters 1 0 -1 for polynomial $x^2 - 1$, the program produces $(x + 1)(x - 1)$. Here's an execution of the program:

```
1 0 -1
1x^2 + 0x + -1 = (1x + 1) (1x + -1)
```

Assume that your program will only handle coefficients from -20 to 20.

We know that

$$(ax + b)(cx + d) = acx^2 + (ad + bc)x + bd$$

Suppose the user entered A, B, C for polynomial

$$Ax^2 + Bx + C$$

then we basically want to find a, b, c, d such that

$$(ax + b)(cx + d) = acx^2 + (ad + bc)x + bd = Ax^2 + Bx + C$$

which is the same as saying:

$$\begin{aligned} A &= ac \\ B &= ad + bc \\ C &= bd \end{aligned}$$

For instance in the case of the polynomial $x^2 - 1$, we want to find a, b, c, d such that

$$\begin{aligned} 1 &= a * c \\ 0 &= a * d + b * c \\ -1 &= b * d \end{aligned}$$

Here's the program:

```

int A, B, C;
std::cin >> A >> B >> C;

int a, b, c, d;

bool found = false;

for (a = -20; a <= 20 && !found; ++a)
{
    for (b = -20; b <= 20 && !found; ++b)
    {
        for (c = -20; c <= 20 && !found; ++c)
        {
            for (d = -20; d <= 20 && !found; ++d)
            {
                if (a * c == A
                    && B == a * d + b * c
                    && b * d == C)
                {
                    found = true;
                }
            }
        }
    }
}

if (found)
{
    std::cout << a << ' ' << b
               << ' ' << c << ' ' << d << '\n';
}
}

```

Of course you can change the range of values for a, b, c, d if you like. In fact, it's a good idea to create a constant, say N, for 20 in the above program so that a, b, c, d ranges from -N to N and you can change the value of N easily.

Note that this is a **brute force** search for a factorization of the given polynomial because it's **not smart**. For instance, if the given polynomial is

$$15x^2 + \dots$$

then since you want

$$15x^2 + \dots = (ax + b)(cx + d) = acx^2 + (ad + bc)x + bd$$

then of course

$$15 = ac$$

and the right thing to do is to factorize 15. You would get $15 = 1 \times 15 = 3 \times 5 = 5 \times 3 = 15 \times 1 = -1 \times -15 = -3 \times -5 = -5 \times -3$ and then a and c can only be

a = 1, c = 15
a = 3, c = 5
a = 5, c = 3
a = 15, c = 1
a = -1, c = -15
a = -3, c = -5
a = -5, c = -3
a = -15, c = -1

The above program however tries 41 possible values for a (i.e., from -20 to 20) and also 41 possible values for c. This means that the program actually tries 41 x 41 possible cases for a and c. Including b and d, the program tries

$$41 \times 41 \times 41 \times 41 = 2825761$$

cases!!! This is no big deal for a modern-day computer of course. But if we want to allow more values for a, b, c, d, say from -100 to 100, then there are about

$$201 \times 201 \times 201 \times 201 = 1632240801$$

You can try this range of values for the above program. You'll see that the program will run very slowly. Brute force algorithms are not smart at all.

Exercise -1.8.141. Write a program that prompts the user for n and if n is even, attempts to rewrite n as a sum of at most two primes. For instance if the user entered 2, the program prints:

2
2

If the user entered 4 the program prints

4 $2 + 2$

If the user entered 12, the program prints

12 $5 + 7$

Using your program, attempt to answer this question: Are there even integers which are not a sum of at most two primes? (For more information, google for “Goldbach conjecture”. This is also a very famous conjecture in Math that is as yet unproven.)

Summary

The following summarizes the pre- and post-increment operators:

`++i` increments the value of `i`

`i++` increments the value of `i`

`j = (++i)` increments the value of `i` and then assign new value of `i` to `j`

`j = (i++)` give the value of `i` to `j` and then increments the value of `i`

The following summarizes the pre- and post-decrement operators

`--i` decrements the value of `i`

`i--` decrements the value of `i`

`j = (--i)` decrements the value of `i` and then assign new value of `i` to `j`

`j = (i--)` give the value of `i` to `j` and then decrements the value of `i`

The following summarizes the augmented assignment operators: suppose `[op]` is an operator such as `+`, `-`, `*`, `/`, `%`, then

`x [op] = y` is the same as `x = x [op] y`

For instance `x += y` is the same as `x = x + y`. The augmented assignment operator returns the new augmented value. For instance:

`z = (x += y)`

The for-loop statement looks like this:

for ([`stmt1`]; [`bool expr`]; [`stmt2`])_[`stmt3`]

where _[`stmt3`] can be either a statement of a block of statements. The for-loop statement executes as follows:

1. Execute [`stmt1`], go to 2.
2. If the [`bool expr`] is true, go to 3, otherwise goto 5
3. Execute [`stmt3`], go to 4.
4. Execute [`stmt2`], go to 2.
5. Exit the for-loop statement to the statement after the for-loop.

Of course since _[`stmt3`] can be any statement or a block, a for-

loop can contain an if statement, if-else statement, a switch-case statement, or even a for-loop.

The scope of a variable is from its point of declaration to the end of the smallest block where it was declared.

Exercises

Q1. Find all positive integer solutions to the equation

$$x^3 + y^4 = 1729$$

(Hint: Write a double for-loop.)

Q2. Write a program that draws the following figure when the user enters 3

```
*  
**  
***
```

and when the user enters 4 it draws

```
*  
**  
***  
****
```

Q3. Write a program that draws the following figure when the user enters 3

```
*  
**  
***
```

and this when the user enters 5

```
*  
 **  
***  
 ****  
*****
```

and this when the user enters 6

```
*  
    **  
***  
    ****  
*****  
*****
```

Q4. Write a program that draws the following figure when the user enters 3

```
***  
*  *  
***
```

and when the user enters 5 it draws



Etc.

Q6. Write a program that draws the following figure when the user enters 7 it draws

```

  ** **
*****
*****
  *****
   ***
    *

```

and when the user enters 9 it draws

```

  *** **
*****
*****
  *****
   *****
    *****
     ***
      *

```

and when the user enters 11 it draws

```

  ***** **
*****
*****
  *****
   *****
    *****
     *****
      ***
       *

```

Q7. Write a program that draws the following given 2 and 2:

```

*****
*           *
*  X       *
*           *
*           *
*           *
*           *
*           *
*           *
*           *
*****

```

and this when the user enters 7 and 3:

```

*****
*           *
*           *
*           *
*           *
*           *
*           *
*  X       *
*           *
*           *
*****

```

Q8. Write a program that prompts the user for n and computes

$$1 + 1/(1) + 1/(1 \times 2) + 1/(1 \times 2 \times 3) + 1/(1 \times 2 \times 3 \times 4) + \dots + 1/(1 \times 2 \times 3 \times \dots \times n)$$

Q9. Write a program that prompts the user for a double d and computes the largest n such that $n^2 + n$ is at most d .

Q10. Write a program that prompts the user for an integer n and displays all the non-squares from 1 to n and print the number of these integers. (An integer is a non-square if it is not the square of another integer. For instance 9 is a square since it is the square of 3, however 6 is not a square.)

Q11. A number n is said to be perfect if it is the sum of all its divisors less than n . For instance 6 is perfect since the divisors of 6 are 1, 2, 3, 6 and the sum of the divisors less than 6 is $1 + 2 + 3 = 6$. Write a program that prompts the user for n and prints all perfect numbers from 1 to n .

12. while Loops

OBJECTIVES

- Write while-loops
- Use sentinel values to exit a while-loop

Like the `for`-loop, the purpose of the `while`-loop is to repeat a block of statements (or just a statement). While there is a difference, once you know how the `for`-loop works, the `while`-loop is easy: the `while`-loop is a simplification of the `for`-loop.

while-loop

First run this. I'm not going to explain it since this is old stuff.

```
int sum = 0;
for (int i = 1; i <= 10; i++)
{
    sum += i;
    std::cout << "i:" << i << " sum:" << sum
               << std::endl;
}
std::cout << "sum:" << sum << std::endl;
```

Now try this:

```
int sum = 0;
int i = 1;

for (; i <= 10;)
{
    sum += i;
    std::cout << "i:" << i << " sum:" << sum
               << std::endl;

    i++;
}
std::cout << "sum:" << sum << std::endl;
```

The program works the same right? One final modification:

```
int sum = 0;
int i = 1;
while (i <= 10)
{
    sum += i;
    std::cout << "i:" << i << " sum:" << sum
               << std::endl;

    i++;
}
std::cout << "sum:" << sum << std::endl;
```

That's all there is to it.

The `while`-loop is just the `for`-loop without the initialization and the update part; the `while`-loop only has the boolean condition from the `for`-loop. So if a `while`-loop must perform an initialization and/or an update, you can put them in the following spots:

```
int sum = 0;
int i = 1;           // INITIALIZATION
while (i <= 10)      // BOOLEAN CONDITION
{
    sum += i;
    i++;              // UPDATE
}
```

By the way, there is a slight difference between the `while`-loop and `for`-loop. The difference has nothing to do with the flow of execution. It has to do with where we declare variables in loops. (This was already mentioned earlier in the notes on `for`-loops).

Look at these:

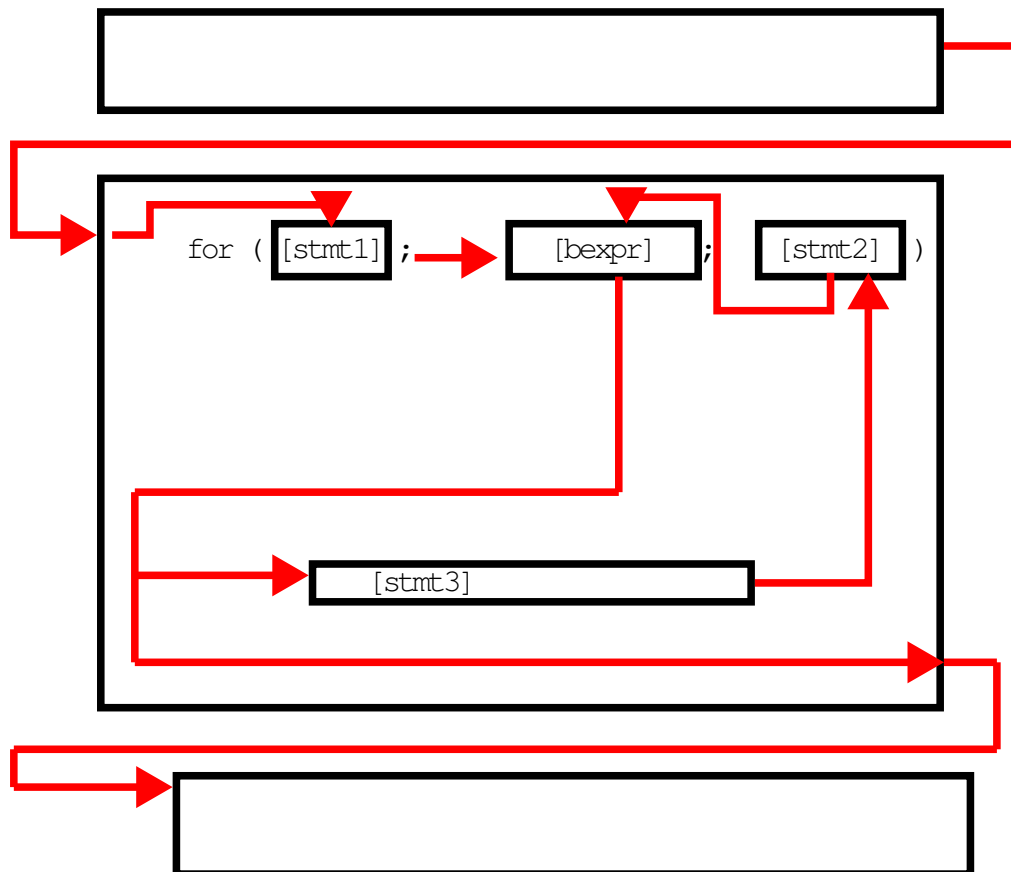
```
int sum = 0;
for (int i = 1; i <= 10; i++)
{
    sum += i;
    std::cout << "i:" << i << " sum:" << sum
                << std::endl;
}
std::cout << "i:" << i << " sum:" << sum
          << std::endl;
```

```
int sum = 0;
int i = 1;
while (i <= 10)
{
    sum += i;
    std::cout << "i:" << i << " sum:" << sum <<
                std::endl;
    i++;
}
std::cout << "i:" << i << " sum:" << sum
          << std::endl;
```

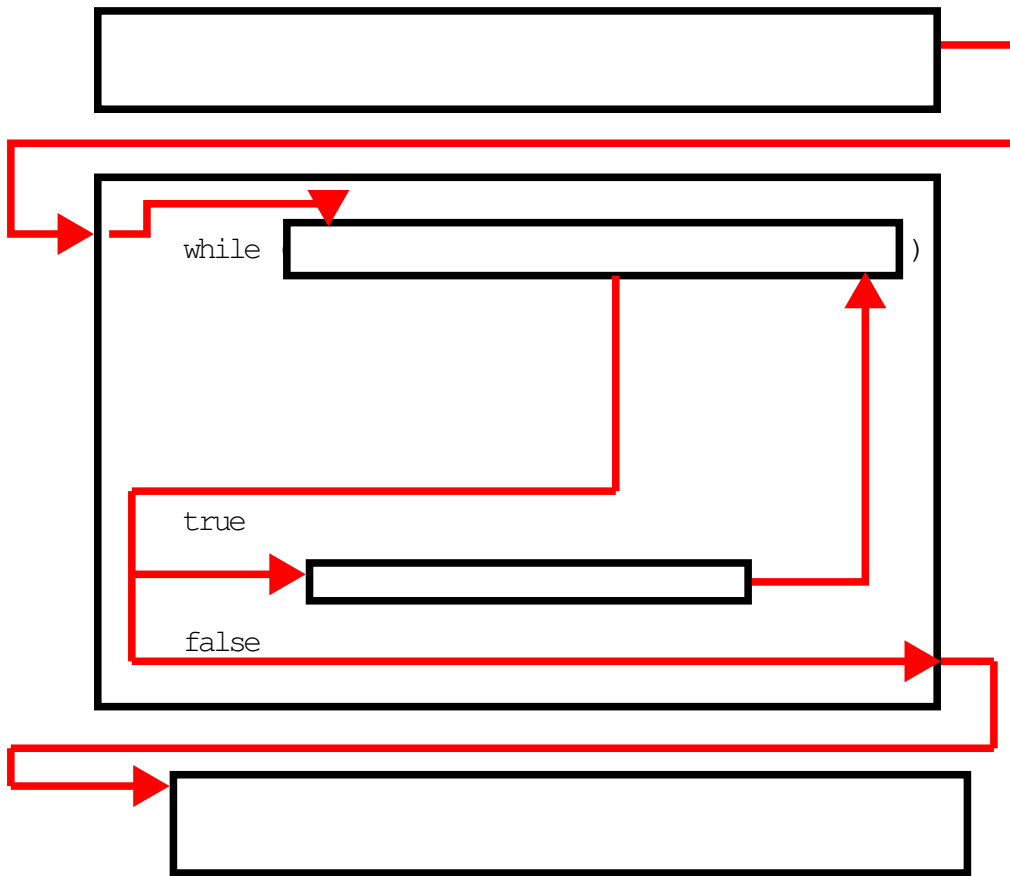
Because the variable `i` in the second example is declared outside the `while`-loop, it is still available after exiting the `while`-loop. In the case of the `for`-loop, the variable `i` is destroyed when you exit the `for`-loop.

Flow of execution in a `while` loop

Recall that the flow of execution for the for-loop looks like this:



Here's the flow of execution of the while-loop:



More Examples

Here's a typical example that uses the `while`-loop.

```
int i = 0;
std::cout << "gimme a number ...";
std::cin >> i;
while (i != 42)
{
    std::cout << "so 42 is not your fav number?";
    << std::endl;
    std::cout << "gimme a number ...";
    std::cin >> i;
}
```

Notice something about this `while`-loop?

Hmmmmthere is **no control variable** unlike our `for`-loop examples. Now try this:

```
int sum = 0;
int i = 0;
std::cout << "gimme a number ...";
std::cin >> i;
while (i != 42)
{
    sum += i;
    std::cout << "sum: " << sum
    << std::endl;
    std::cout << "gimme a number ...";
    std::cin >> i;
}
std::cout << "final sum: " << sum << std::endl;
```

In general, if you want to process a bunch of numbers, you can run over these numbers in two different ways: Either the program knows how to compute them (example: the numbers are 1, 2, 3, 4, 5, 6, 7, 8, 9, 10) or the program does not (example: it's based on user input).

For the first case, you should use the **for-loop** with a **control or index or counter variable** running over the values.

For the second case, you should use the **while-loop**. To let the user stop the data processing, you have to pick a special value for the user to use to indicate end-of-data. This special value is called the **sentinel value**.

Now I want you to step back and look at the “structure” of the above program:

```
initialization
prompt user for data
while value of data is not sentinel value:
    process data
    prompt user for data
```

Note that there is code duplication:

```
initialization
prompt user for data
while value of data is not sentinel value:
    process data
    prompt user for data
```

Exercise -1.8.142. Write a program that continually “rolls a die” and prints the face value of the die until 6 is reached. (Use `rand()` to generate a random integer between 1 and 6.)

Exercise -1.8.143. Write a program that computes the product of integers entered by the user. Use a sentinel value of 0 to indicate the end of data. Here’s the skeleton code:

```
int product = ____;
int i = 0;

std::cout << "gimme a number ...";
std::cin >> i;
while (_____)
{
    _____;
    std::cout << "product: " << product
               << std::endl;
    std::cout << "gimme a number ...";
    std::cin >> i;
}
std::cout << "final product: " << product << std::endl;
```

Exercise -1.8.144. Write a program that computes the cost of head

grafting surgery. Each head costs \$100.42. Display the total number of heads requested. The program exits the loop when the number of heads requested is zero. Here is the skeleton code:

```
int heads = 0;
double cost = 0.0;
int totalHeads = ____;
double totalCost = ____;

std::cout << "how many heads? ";
std::cin >> heads;
while (____)
{
    cost = ____;
    totalHeads = ____;
    totalCost = ____;
    std::cout << "total heads requested: "
               << totalHeads << std::endl
               << "total cost: "
               << totalHeads <<
               '\n' << std::endl;
    // prompt for heads
}

std::cout << "your final bill:" << totalCost << '\n'
          << "i hope you enjoy your extra "
          << totalHeads << " heads" << std::endl;
```

Exercise -1.8.145. Due to popular demand and lack of staff and surgeons (and donors), we have to limit the number of extra heads per visit. Modify the above program so that the program exits the while-loop when the number of heads requested is zero or when the total number of heads requested exceeds 42.

Exercise -1.8.146. Write a simple number guessing game. The program generates a random integer from 1 to 10 (inclusive), and continually prompts the user to guess the number until the correct answer is given. When you're done with that, get the program to print the number of tries.

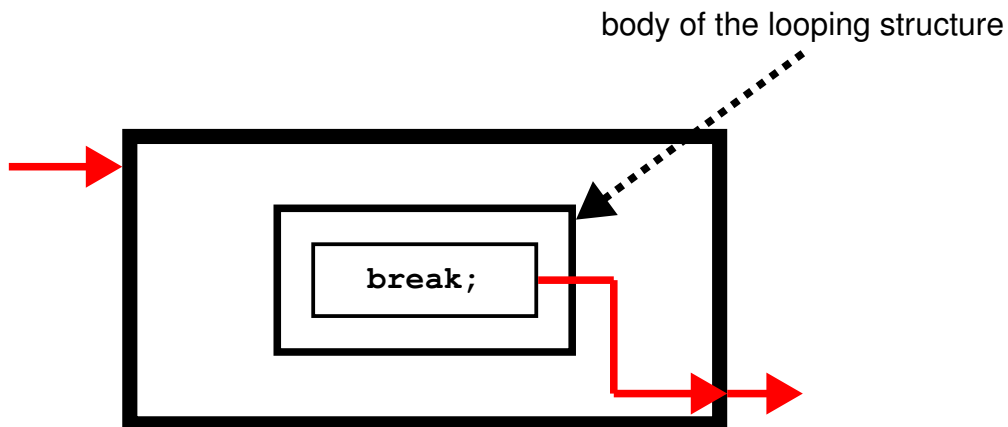
Exercise -1.8.147. Modify our multiplication game so that it keeps posing questions until the user answers 5 questions correctly. While prompting the user, print the number of correct answers. (Once you're done with that, make a slight modification to prompt the user

for the number of correct answers before terminating the program.)

Exercise -1.8.148. Modify the above program so that the program terminates when 5 **consecutive** correct answers are given.

break

Fortunately the `break` statement works the same way for the `while`-loop as it does for the `for`-loop.



```
int i = 0;
while (i < 1000)
{
    std::cout << i << std::endl;
    if (i == 100) break;
    i++;
}
std::cout << "out ... " << i << std::endl;
```

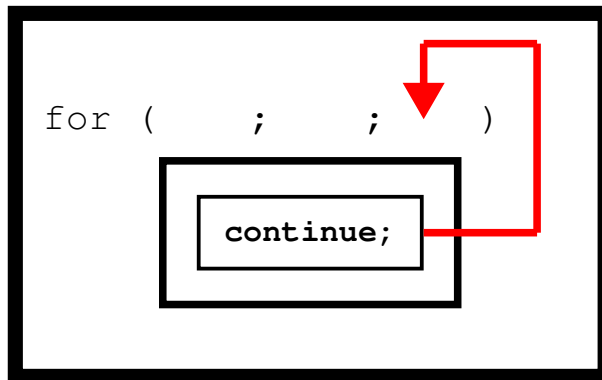
Again, the same advice for using the `break` statement in the `for`-loop applies: Don't overuse it.

Exercise -1.8.149. What is the output? Work it out by hand and then check it by running the program.

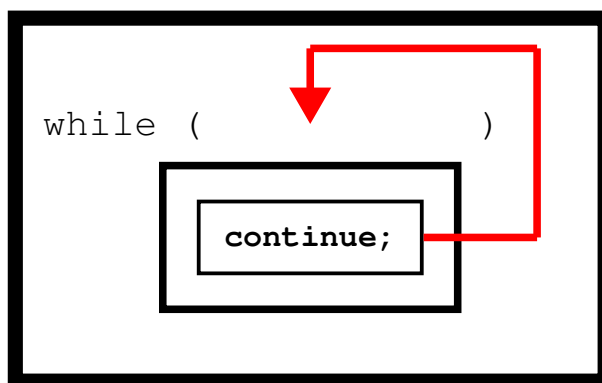
```
int i = 0;
while (i <= 10)
{
    std::cout << "A " << i << ' ';
    i += 2;
    std::cout << "B " << i << ' ';
    if (i > 5) break;
    std::cout << "C " << i << ' ';
    i += 3;
    std::cout << "D " << i << '\n';
}
std::cout << i << std::endl;
```

continue

Recall that this is the flow of execution for `continue` in the `for`-loop:



The `continue` statement works like this for the `while`-loop:



Exercise -1.8.150. What is the output? Work it out by hand and then check it by running the program.

```
int i = 0;
while (i <= 10)
{
    i += 5;
    if (i > 5) continue;
    i += 3;
}
std::cout << i << std::endl;
```

Exercise -1.8.151. Here's an example. Run it.

```
int numHeads = 0;
std::cout << "how many heads do you have? ";
std::cin >> numHeads;
while (numHeads != 0)
{
    std::cout << "cool!" << std::endl;
    std::cout << "how many heads do you have?";
    std::cin >> numHeads;
}
```

Now insert code that prints “rubbish” and continue to the next iteration of the loop if numHeads is negative:

```
int numHeads = 0;
std::cout << "how many heads do you have? ";
std::cin >> numHeads;
while (numHeads != 0)
{
    // INSERT CODE HERE
    std::cout << "cool!" << std::endl;
    std::cout << "how many heads do you have?";
    std::cin >> numHeads;
}
```

More examples

Remember our example on computing the maximum of a list of values?

```
int x, y, z;
std::cin >> x >> y >> z;

int max = x;
if (max < y) max = y;
if (max < z) max = z;
std::cout << max << std::endl;
```

Let's write a program that prompts the user for integers and computes the maximum of all integers entered. The program stops prompting when the user enters -99999. Here's the skeleton code:

```
int x = 0;
std::cin >> x;
int max = x;

while (x != -99999)
{
    // process x
    std::cout << max << std::endl;
    std::cin >> x;
}
std::cout << "final max: " << max
          << std::endl;
```

Note that we need a “special value” of -99999 to stop the loop. What do we do with x? We check if it's larger than what we have in max:

```
int x = 0;
std::cin >> x;
int max = x;

while (x != -99999)
{
    if (max < x) max = x;
    std::cout << max << std::endl;
    std::cin >> x;
}
std::cout << "final max: " << max << std::endl;
```

Exercise -1.8.152. Modify the above program so that it computes the minimum instead of the maximum.

Exercise -1.8.153. Modify the above program so that it computes the sum of all values entered by the user until 0 is entered. When 0 is entered, the program prints the final sum and stops.

Exercise -1.8.154. Write a program that continually prompts the user for numbers and prints the running average. The program stops prompting when the user enters `-99999`.

Summary

The format of the `while`-loop looks like this:

```
while ([bool expr])  
    [stmt]
```

where `[bool expr]` is a boolean expression and `[stmt]` is a single statement or a block of statements.

The `while`-loop is just the `for`-loop without the initialization and the update parts.

In general, if your loop requires a variable that runs over a sequence of numbers that can be computed by the program, you should use the `for`-loop. Otherwise you should use the `while`-loop. (There are exceptions. But this advice will do for now).

The `break` statement, if executed in the body of the `while`-loop, will exit the `while`-loop and continues with the execution of the statement immediately after the `while`-loop.

The `continue` statement, if executed in the body of the `while`-loop, will immediately bring the execution to the boolean condition of the `while`-loop.

Exercises

Q1. Write a program that computes the gas mileage of a car. The program continually prompts the user for the relevant data. In each iteration of the loop, the program prompts the user for the miles traveled and the gas used. When the user entered 0 for miles, the program stops prompting the user, and displays the gas mileage (i.e. total miles divided by the total gas usage).

Q2. Modify our multiplication game so that it keeps posing questions until the user answers 80% of the questions correctly. While prompting the user, print the percentage of his/her correct answers.

Q3. Investigate the following problem: How many die rolls do you need to get the value 6? What about two 6's? What about three 6's? ... What is the likelihood of getting a 6? What is the likelihood of getting two 6's? ... What about the case of two consecutive 6's? Three consecutive 6's?

14. Loops

OBJECTIVES

Don't worry ... I won't be introducing more loop syntax.

In this section we'll look at more examples. I'll also give some general comments, suggestions, techniques, warnings etc.

We'll start off with two reviews ... general advice on when you should use a for-loop and when you should use `while`-loop ...

When do I use the `for`-Loop?

I've already shown you that you can always rewrite a `for`-loop into a `while`-loop and vice versa.

So ... how do you choose which type of loop to use?

The general advice is simple: If your loop has a control (index or counter) variable that runs across a sequence of values without user intervention and the control variable changes in a predictable way, you should use the `for`-loop. The user might need to “configure” this sequence: for instance, by specifying a starting value and an ending value and how to step through the intermediate values. But once that's done, the control variable in the `for`-loop should run without user intervention.

Otherwise you should use the `while`-loop.

Look at this familiar example:

```
int start = 0, end = 0, step = 0;
std::cin >> start >> end >> step;

int sum = 0;
for (int i = start; i <= end; i += step)
{
    sum += i;
}
std::cout << sum << std::endl;
```

Note that the loop is controlled by `i` and `i` steps through a sequence with the starting and ending values known before the loop starts.

Note that the control variable is only used in loop through some statements to achieve a goal. Look at the above example: The control variable is defined within the `for`-loop. After the `for`-loop, the control variable is destroyed.

The next section shows you a situation where the data to process is entered by the user and hence cannot be predicted ahead of time.

When do I use the `while`-Loop?

Suppose you want to write a program that adds numbers for a user but you do not know in advance how many numbers to add. Of course you need to give the user an opportunity to stop the adding process! There are two ways of doing this. Either you can ask the user each time if he/she wants to enter a number to add or to stop, or ... you can use a **sentinel value**. This is a special value entered by the user to indicate that the processing loop should stop.

Let's try the simplest possible example. This program prompts the user until he/she enters a sentinel value. I'm choosing -1 to be the sentinel value.

i dont know how to properly format this right now prompt user for data while value of data is not sentinel value: process data prompt user for data Note that there is code duplication: prompt user for data while value of data is not sentinel value: process data prompt user for data

Removing Duplication in the sentinel `while`-loop

Look at the sentinel while-loop again:

i dont know how to properly format this right now prompt user for data while value of data is not sentinel value: process data prompt user for data

Remember that there is code duplication. If you want to get rid of code duplication, you can do this:

```
while (1): // don't forget that 1 is type cast to true prompt user for data
if value is the sentinel value: break process data
```

Note that some people do not like `breaks` because it's harder to tell when you get out of a loop because you can issue a `break` statement anywhere in the body of the `while` loop. They prefer the exit condition in the header of the `while` loop so that visually one can tell quickly when you get out of the loop.

Using the above technique, the following program:

```
int i = 0, sum = 0;

std::cout << "gimme a number: "; // INPUT
std::cin >> i;
while (i != -1) // CONDITION TO REPEAT LOOP
{
    sum += i;

    std::cout << "gimme a number: "; // INPUT
    std::cin >> i;
}
std::cout << "sum: " << sum << std::endl;
```

becomes:

```
int i = 0, sum = 0;
while (1)
{
    std::cout << "gimme a number: "; // INPUT
    std::cin >> i;
    if (i == -1) break; // CONDITION TO EXIT LOOP

    sum += i;
}
std::cout << "sum: " << sum << std::endl;
```

Make sure you study this program carefully!!!

Exercise -1.8.155. Rewrite the above program so that in addition to exiting the loop when the term entered in 0 (i.e., use a sentinel value of 0), the program also exits the loop when the sum exceeds 10.

Exercise -1.8.156. Using this structure of the while-loop for data processing, rewrite your previous program that computes the product of integers entered by the user. Use a sentinel value of 0.

Example: sentinel value

Here's an easy example. Write a simple cash register program. The program prompts the cashier for the number of items and the unit price per line item (yes, it's a lousy point-of-sale program). There is a tax of 20% (this is probably in New York).

```
const double TAX = 0.2;
double total = 0.0;
double price = 0.0;
int num = 0;

while (1)
{
    std::cout << "units: ";
    std::cin >> num;

    std::cout << "unit price: ";
    std::cin >> price;

    double cost = num * price * (1 + TAX);
    std::cout << "please pay " << cost
              << std::endl;
}
```

The problem is that the program doesn't stop!!!

Exercise -1.8.157. Add code to this program so that when the cashier enters 0 for number of items, the program gets out of the loop.

Exercise -1.8.158. Add code to print the warning "units cannot be negative" and `continue` when `num` is negative.

Exercise -1.8.159. Add code to print the warning "unit price cannot be negative" and `continue` when `price` is negative.

Exercise -1.8.160. Add a running total so that the cashier can double check the amount in the till at the end of the day.

Of course for a real business application, the POS register would be networked to a database server containing the product catalog

and you simply scan the UPC to retrieve the unit price. The data computed is stored in a database directly. And of course you can have more than one line item per sale.

Note that with multiple `continues`, it makes the program harder to read. This is similar to the situation where there are multiple breaks in the body of the while-loop. Can you rewrite the above program so that there are no continues?

Other conditions

So far we have seen two different ways for a loop to terminate:

- when a control variable has finished running over it's sequence (for-loop)
- when a data input is a sentinel value (while-loop)

Of course loops can terminate based on other types of conditions. It's impossible to list all possible cases!!! You have to analyze each case carefully to see what's the right thing to do.

Example: counting digits

Here's a problem where the condition does not depend on continually prompting the user for an input. Therefore there is **no sentinel value**. You will see that there is also no control variable – so you definitely should not expect a for-loop.

Pay attention to the way I slowly work through the problem until I see the loop.

Suppose the user enters an integer and you want to count the number of digits in that integer.

For instance if the user enters 134, your program must display 3. If the user enters 97531, your program must display 5.

So how do you do that? If you count the digits from the right to left, you can think of moving a finger one digit at a time and counting each digit your finger passed by:

```
97531^
9753^1      1 digit
975^31      2 digits
97^531      3 digits
9^7531      4 digits
^97531      5 digits
( = your finger.)
```

Now you notice that the digits you've passed does not affect the rest of your digit-counting process. So let's throw them away:

```
97531^
9753^      1 digit
975^      2 digits
97^      3 digits
9^      4 digits
^      5 digits
```

Now, the symbol for our finger doesn't help. Let's get rid of that too:


```
97531
9753  1 digit
975   2 digits
97    3 digits
9     4 digits
```

Well ... that sure looks like removing the rightmost digit (one at a time) to me. And we already know how to do that: just do integer division by 10!!! If we do that, then the last number should be zero:

```
97531
9753  1 digit
975   2 digits
97    3 digits
9     4 digits
0     5 digits
```

Of course the number that is going up (the 1, 2, 3, 4, 5) requires a variable. Let's call it *x*. And let's say the number we're working on (the number entered by the user) is placed in variable *n*.

Now when does the process **stop**? When we see a zero in the left column of the above simulation, right? So the process seems to be this:

```
prompt user for integer n
if n is not zero, n = n / 10 and x = 1
if n is not zero, n = n / 10 and x = 2
if n is not zero, n = n / 10 and x = 3
if n is not zero, n = n / 10 and x = 4
if n is not zero, n = n / 10 and x = 5
n is zero. Stop.
```

We need to write this as an algorithm. Obviously there's a loop here!!!

You want the statements

```
if n is not zero, n = n / 10 and x = 1
if n is not zero, n = n / 10 and x = 2
if n is not zero, n = n / 10 and x = 3
if n is not zero, n = n / 10 and x = 4
if n is not zero, n = n / 10 and x = 5
```

to be replaced by one statement. You can “combine” them like this:

```
if n is not zero, n = n / 10 and x = x + 1
if n is not zero, n = n / 10 and x = x + 1
if n is not zero, n = n / 10 and x = x + 1
if n is not zero, n = n / 10 and x = x + 1
if n is not zero, n = n / 10 and x = x + 1
```

AHA! They are all the same! Of course x does not have an initial value. You need to initialize it. And based on the above example, you can see that x should be initialized to 0. Now you can write the following very easily:

```
while n is not 0:
    n = n / 10
    x = x + 1
```

Here's the code:

```
int n = 97531;
int numDigits = 0;

while (n != 0)
{
    n /= 10;
    ++numDigits;
    std::cout << n << ' ' << numDigits << std::endl;
}

std::cout << numDigits << std::endl;
```

I've inserted print statements to get the program to show us the intermediate steps. And instead of `x`, I'm using `numDigits` which is a lot more descriptive.

Yes, of course you can rewrite the while-loop as a for-loop:

```
int numDigits;

for (numDigits = 0; n != 0; ++numDigits)
{
    n /= 10;
    std::cout << n << ' ' << numDigits << std::endl;
}
```

Although it's not wrong, note that the for-loop does not really use a control variable: the variable updated (i.e. `numDigits`) is the variable in the boolean expression (i.e. `n`). This is an indication that the while-loop is probably best.

Remember: If the loop doesn't look like a for-loop (i.e., there's no single variable that runs across a sequence of values), then don't use it. Use the while-loop instead.

Exercise -1.8.161. There's just one problem. Run it with `n = 0`. What do you get? How many digit(s) do you see in the number 0? Of course the number 0 has 1 digit. This is a special case. So the pseudocode looks like this:

```
if n is 0:  
    numDigits = _____  
else  
    use the above digit removing algorithm to compute numDigits
```

Correct the program. Test your program thoroughly.

Error-checking for a sentinel while-loop

The following is a structure for the while-loop that includes checks for breaking and data validation:

```
some initialization
while (1):
    get data
    if data is sentinel value:
        break
    if data is not valid:
        print warning
        continue
    process data
```

OR

```
some initialization
while (1):
    get data
    if data is sentinel value:
        break
    if data is not valid:
        print warning
    else:
        process data
```

The second version removes the presence of `continue`. Make sure you understand that they work the same!!! You can use either version in your programs.

Exercise -1.8.162. Write a program that prompts the user for positive integers and computes the sum of the square root of the integers entered by the user. Of course, you can't compute the square root of a negative number, right? The program terminates when the user enters 0. The program prints a warning when a negative value is

entered.

Although this is usually not mentioned in most textbooks, the above is a very standard structure of the loop used in the industry, if you need to use it at all. In general, alternative flow of execution must be bunched up together in a spot which is easy to find. We won't have time to talk too much about this topic, but in writing robust software there are concepts such as assertions, exceptions, pre-conditions, post-conditions, etc. which are all related to catching or verification of error conditions. They are all related to flow of control. These controls are also usually placed at the "usual" places, near the top or the bottom of certain structures. I won't elaborate on "assertions, exceptions, pre-conditions, post-conditions" – you will learn some of these in future classes.

Avoiding break and continue

The basic principle in programming is simple: don't do crazy things!

In other words, don't try to surprise the reader (including yourself). Writing crazy stuff throws people off. It makes debugging really difficult – that includes yourself!!!

One corollary is that you should limit the number of exit points for your loops. This means not having too many `break`s in your loops. Likewise limit the number of `continue`s.

Exercise -1.8.163. Rewrite this without the `break`.

```
int prod = 1;
for (int i = 1; i <= 10; i++)
{
    if (i == 5) break;
    prod *= i;
}
```

Exercise -1.8.164. Rewrite the above without the `continue`.

```
int prod = 1;
for (int i = 1; i <= 10; i++)
{
    if (i == 5) continue;
    prod *= i;
}
```

“Pushing” the Control/Index/Counter Variable in the for-Loop (DIY)

Here’s the program from the previous section:

```
int prod = 1;
for (int i = 1; i <= 10; i++)
{
    if (i == 5) continue;
    prod *= i;
}
```

One method of terminating a for-loop early is what I call “pushing the control variable”. It looks like this:

```
int prod = 1;
for (int i = 1; i <= 10; i++)
{
    if (i == 5)
    {
        i == 11; // force i to have value to stop
                // the for-loop
        continue;
    }
    prod *= i;
}
```

Unfortunately this method requires knowing the bound of the values assumed by *i* and hence can be dangerous. Therefore ... DON'T DO IT!

In fact, the general advice is this: **A much as possible, do not modify the control/index/counter variable in the body of the for-loop.**

WARNING: break and continue are shallow!

It's very important to remember that the break statement only breaks out of the loop the program is **currently** in!!!

```
for (int i = 0; i < 5; i++)
{
    std::cout << "i: " << i << std::endl;
    for (int j = 0; j < 5; j++)
    {
        if (j == 3) break; // WHERE DO YOU GO?
        std::cout << " j: " << j << std::endl;
    }
}
```

Likewise continue only applies to the current loop:

```
for (int i = 0; i < 5; i++)
{
    std::cout << i << std::endl;
    for (int j = 0; j < 5; j++)
    {
        if (j == 3) continue; // WHERE DO YOU GO?
        std::cout << j << std::endl;
    }
}
```

This is the same for nested while-loops or a for-loop containing a while-loop.

Breaking out of Deeply Nested Loops

Now what if the break in the above example is really meant to break out of BOTH for-loops?

One way to accomplish this would be to use a special boolean flag to indicate that a complete break is needed. Here's an example. Suppose you have this code:

```
for (int i = 0; i < 10; i++)
{
    ... SOME STATEMENTS ...
    for (int j = 0; j < i; j++)
    {
        if (j == 3)
        {
            // WANT TO BREAK OUT OF *BOTH* LOOPS
        }
    }
    ... SOME STATEMENTS ...
}
```

You modify it in this manner:

```
for (int i = 0; i < 10 && !stop; i++)
{
    ... SOME STATEMENTS ...
    bool stop = false;
    for (int j = 0; j < i; j++)
    {
        if ( j == 3 )
        {
            stop = true;
            break; // break of of inner loop
        }
    }
    if (stop) break; // break out of outer loop

    ... SOME STATEMENTS ...
}
```

So in general (i.e. for both the for-loop and the while-loop) you have something like this:

```
begin of loop1

    bool stop = false
    begin of loop2
        if ([bool expr]):
            stop = true
            break
    end of loop2
end of loop1
```

As you can see things can go **pretty ugly**. The best is to try to simplify the actual for-loop or even the whole program whenever possible rather than trying to insert controls to break out in the middle of the for-loop (or while-loop).

Exercise -1.8.165. Here's a simple drill. You own a certain number of banks (at most 100) and each bank has a certain number of accounts (at most 1000). This program helps you calculate how much you have altogether in the banks. When entering the amount in each account at a bank, if you enter 0, you will immediately go to the next bank and if you enter -1, the program will print the total and stop. Complete the program!

```
double total = 0;
for (int bank = 0; bank < 100; ++bank)
{
    std::cout << "bank #" << i << '\n';
    for (int account = 0; account <= 1000; ++account)
    {
        std::cout << "account #" << account << "\n";
        std::cout << "amount (0:next bank, -1:exit)";
        double amount = 0;
        std::cin >> amount;
        if (amount == 0)
        {
            // exit this for-loop and go to
            // next bank
        }
        if (amount == -1)
        {
            // exit both for-loops and go to
            // print total
        }
        total += amount;
    }
}

// print total
std::cout << total << '\n';
```

Numerical Approximation

Look at this sum:

$$t = \frac{1}{1 * 1} + \frac{1}{2 * 2} + \frac{1}{3 * 3} + \frac{1}{4 * 4} + \frac{1}{5 * 5}$$

This is a sum of 5 terms. It's easy to write a program to compute the sum – you have seen similar examples before.

Exercise -1.8.166. Write a program that computes t . Should you use a for-loop or a while-loop? The value you get should be approximately 1.46361.

Exercise -1.8.167. Now modify the above program to compute

$$t = \frac{1}{1 * 1} + \frac{1}{2 * 2} + \frac{1}{3 * 3} + \frac{1}{4 * 4} + \frac{1}{5 * 5} + \frac{1}{6 * 6}$$

The value should be 1.49139. (If you get 1 then you know you're wrong.)

Exercise -1.8.168. The first program of this section computes a sum of 5 terms. The second computes a sum of six terms. Now do the obvious: Rewrite your program so that it prompts the user how many terms to use in the sum. Let's say the variable for the number of terms is n .

Exercise -1.8.169. Modify your program to print the following values:

```

  1
  ----
1 * 1

  1      1
  ---- + ----
1 * 1  2 * 2

  1      1      1
  ---- + ---- + ----
1 * 1  2 * 2  3 * 3

  1      1      1      1
  ---- + ---- + ---- + ----
1 * 1  2 * 2  3 * 3  4 * 4

  1      1      1      1      1
  ---- + ---- + ---- + ---- + ----
1 * 1  2 * 2  3 * 3  4 * 4  5 * 5

up to

  1      1      1      1
  ---- + ---- + ---- + ... + ----
1 * 1  2 * 2  3 * 3      10 * 10

```

You should get this:

```

1
1.25
1.36111
1.42361
1.46361
1.49139
1.5118
1.52742
1.53977
1.54977

```

Now let the printout go up to a sum of 20 terms, then 30 terms, etc. Keep going until the first 5 decimal places stops changing.

Basically, what you're doing in the previous exercises is to compute the approximations to

$$s = \frac{1}{1 * 1} + \frac{1}{2 * 2} + \frac{1}{3 * 3} + \frac{1}{4 * 4} + \frac{1}{5 * 5} + \dots$$

Note that this goes on forever. The reason why it stops changing after a while is that the term you're adding to the sum gets smaller and smaller "very quickly" :

$\frac{1}{i * i}$ If you want to know more, you should take Calc 2. This is not a Calc class.

(To the math folks: If you keep going forever you get

$$\frac{\pi^2}{6}$$

correct?)

Computations of approximations is important for many kinds of applications including scientific, mathematical, and business applications.

Now suppose you work for NASA and you need to compute the above sum

$$s = \frac{1}{1 * 1} + \frac{1}{2 * 2} + \frac{1}{3 * 3} + \frac{1}{4 * 4} + \frac{1}{5 * 5} + \dots$$

(which goes on forever) up to 10 decimal places. You can do what we did early: Keep including more and more terms. You need to get C++ to reveal more decimal places (use `std::setprecision()`). That would be really time consuming.

Another way is to let the for-loop run without an upper limit for i (how would you do that?) and when you see the first 10 decimal places not changing you can write it down (if you can see it scrolling by) and stop the program manually. But this might take a while and if you go out to lunch, the for-loop keeps running even after the first 10 decimal places stop changing. This is a waste of CPU time.

We need to figure out how to stop the loop AUTOMATICALLY.

By the way if you look at the term:

$$\frac{1}{i * i}$$

obviously you want to avoid integer division. So in C++ you should do

$$1.0 / (i * i)$$

But besides fixing the division problem, you have another: recall that an int value in C++ does not go on forever. If you want to represent bigger numbers, you should do

$$1.0 / (\text{double}(i) * i)$$

i.e. even the denominator multiplication should be “doubled” -up.

Exercise -1.8.170. Repeat the above example with this:

$$s = \frac{1}{1*1*1} + \frac{1}{2*2*2} + \frac{1}{3*3*3} + \frac{1}{4*4*4} + \frac{1}{5*5*5} + \dots$$

Exercise -1.8.171. Repeat the above example with this:

$$s = \frac{1}{1 * 1} + \frac{1}{3 * 3} + \frac{1}{5 * 5} + \dots$$

Exercise -1.8.172. Repeat the above example with this:

$$s = \frac{1}{1 * 1} - \frac{1}{3 * 3} + \frac{1}{5 * 5} - \frac{1}{7 * 7} + \dots$$

(WARNING: Note the alternating + and – signs.)

Exercise -1.8.173. Repeat the above example with this:

$$s = \frac{1}{1} + \frac{1}{1*2} + \frac{1}{1*2*3} + \frac{1}{1*2*3*4} + \frac{1}{1*2*3*4*5} + \dots$$

Numerical Approximation: Stopping the Approximation Process

To really answer the last question, you need to know a little more math. So I'm going to simplify the problem.

Instead of stopping the computation when the further computations will not change the first 10 decimal places, I'll replace the stopping condition with this: Stop the approximation process when the **first 5 decimal places of two consecutive approximations are the same**. I'll give you the code structure for the general problem.

What I mean is this. Suppose the following are values from an approximation and you want to stop when two consecutive approximations have the same first 3 decimal places:

1.1 1.123 1.1345 1.1355 **1.1356** 1.1357

You should stop at the number in bold: Look at its first 3 decimal places and the first decimal places of the previous number. Get it?

OK. Let's do it.

Note that since you need to compare **two consecutive approximations**, that tells you that the computer is thinking about two approximations at the same time: You need two variables to hold consecutive approximations. Let's use the previous values as a simulation. The values computed are:

1.1 1.123 1.1345 1.1355 **1.1356** 1.1357

Let's call the two approximation variables a0, a1. You can also think

of a0 and a1 scanning over the above numbers hand-in-hand:

1.1	1.123	1.1345	1.1355	1.1356	1.1357
<i>a0</i>	<i>a1</i>				
1.1	1.123	1.1345	1.1355	1.1356	1.1357
	<i>a0</i>	<i>a1</i>			
1.1	1.123	1.1345	1.1355	1.1356	1.1357
		<i>a0</i>	<i>a1</i>		
1.1	1.123	1.1345	1.1355	1.1356	1.1357
			<i>a0</i>	<i>a1</i>	

and we stop at this point because the first three decimal places of a0 and a1 at this point are the same.

Hmmm ... it seems like we need to initialize a0 and a1 with the first two approximations. But there's something about the loop Look at the first two lines.

1.1	1.123	1.1345	1.1355	1.1356	1.1357
<i>a0</i>	<i>a1</i>				
1.1	1.123	1.1345	1.1355	1.1356	1.1357
	<i>a0</i>	<i>a1</i>			

The pseudocode for the above two data lines looks like this:

```
compute first approximation and put into a0
compute second approximation and put in to a1
compute second approximation and put into a0
compute third approximation and put in to a1
```

But a better way is this:

```
compute first approximation and put into a0
compute second approximation and put in to a1
a0 = a1
compute third approximation and put in to a1
```

See it? By the way, note that the computation of the next (third) approximation should depend on the current (second) approximation. You don't have to do it from scratch. For instance, to compute

$$\frac{1}{1 * 1} + \frac{1}{2 * 2} + \frac{1}{3 * 3}$$

you just need to add $1.0 / (3 * 3)$ to

$$\frac{1}{1 * 1} + \frac{1}{2 * 2}$$

What about the ending condition? The right thing to do is to first compute the new approximation and then terminate the loop if $a0$ and $a1$ are close enough, say they are less than 0.0001 apart. What does that mean? That means we stop the loop if either

$$a0 - a1 < 0.0001$$

or

$$a1 - a0 < 0.0001$$

In other words we stop if mathematically

$$|a1 - a0| < 0.0001$$

where $| \dots |$ means “the absolute value of”. In C++ the absolute value is given by the `abs()` function for integer values and `fabs()` for double. The above becomes

$$fabs(a0 - a1) < 0.0001$$

You might need to `#include <cmath>` to use the `abs()` or `fabs()` function.

Exercise -1.8.174. Review ... try this:

```
std::cout << abs(5) << std::endl;
std::cout << abs(-5) << std::endl;
std::cout << fabs(12.34) << std::endl;
std::cout << fabs(-12.34) << std::endl;
```

The pseudocode looks something like this:

```
error = 0.00001
approx0 = [some value]
approx1 = [another value]
while (|approx0 -- approx1| >= error):
    approx1 = approx0
    compute approx1
```

This is correct but we can simplify the code. Look at the previous simulation:

1.1	1.123	1.1345	1.1355	1.1356	1.1357
<i>a0</i>	<i>a1</i>				
1.1	1.123	1.1345	1.1355	1.1356	1.1357
	<i>a0</i>	<i>a1</i>			
1.1	1.123	1.1345	1.1355	1.1356	1.1357
		<i>a0</i>	<i>a1</i>		
1.1	1.123	1.1345	1.1355	1.1356	1.1357
			<i>a0</i>	<i>a1</i>	

It requires initializing *a0* and *a1* with the first two approximations. But I can initialize *a0* with a dummy value say 0:

0.0	1.1	1.123	1.1345	1.1355	1.1356	1.1357
<i>a0</i>	<i>a1</i>					
0.0	1.1	1.123	1.1345	1.1355	1.1356	1.1357
	<i>a0</i>	<i>a1</i>				
0.0	1.1	1.123	1.1345	1.1355	1.1356	1.1357
		<i>a0</i>	<i>a1</i>			
0.0	1.1	1.123	1.1345	1.1355	1.1356	1.1357
			<i>a0</i>	<i>a1</i>		
0.0	1.1	1.123	1.1345	1.1355	1.1356	1.1357
				<i>a0</i>	<i>a1</i>	

As long as *a1* is far enough from *a0*, I'm guaranteed that the approximation will be the same as the previous code.

For the above problem we have the following code:

```
double error = 0.00001;
double approx0 = 0.0;
double approx1 = 1.0; // i.e. 1.0 / (1 * 1)
int i = 1;
while (abs(approx0 - approx1) >= error)
{
    approx0 = approx1;
    i++;
    approx1 += 1.0 / (i * i);
    std::cout << approx1 << std::endl;
}
```

Verify by hand that your program works. For instance, use a calculator to compute $1 + 1/4 + 1/9$ and look for this number in your output.

Here's a curious problem. Look at the following sequence of numbers:

etc. Write a program to compute an approximation of this number.

Stop the loop when consecutive approximations differ by at most 0.0001.

Hint: What's the relationship between the last and the second last expressions above? If x is the second last, then the last is

1

$1 + \frac{1}{1+x}$ See it?

Here's the code:

```
double error = 0.00001;
double x = 0;
double nextx = 1;
while (fabs(x -- nextx) >= error)
{
    x = nextx;
    nextx = 1 + 1/(1 + x);
}
```

For the math geeks: What number is it? Recognize it?

Exercise -1.8.175. Here's an ancient algorithm for computing square roots known to the Babylonians. Suppose S is given and you want to compute the square root of S .

1. Start with an arbitrary positive start value x_0 (the closer to the root, the better).
2. Let x_{n+1} be the average of x_n and S/x_n i.e. $x_{n+1} = (1/2)(x_n + S/x_n)$
3. Repeat step 2, until the desired accuracy is achieved.

In other words, if you repeat the above algorithm, the sequence of values $x_0, x_1, x_2, \dots$, will get closer and closer to the actual square root of S .

To understand how it works, try to run this algorithm with $S = 1$ and $x_0 = 1$ to compute approximations to the square root of 1. Next try it with $S = 2$ and $x_0 = 1$ to compute the square root of 2.

Write a program that prompts the user for an integer value S and computes an approximation. To start off, your first approximation x_0 should be computed as follows:

x_0 is the integer such that

$$x_0^2 \leq S \leq (x_0 + 1)^2$$

(Use a for-loop.) You should stop when two consecutive approxima-

tions agree up to 5 decimal places. [Obviously you should not use the `pow()` or `sqrt()` function for this problem!!!]

Exercise -1.8.176. Here's another curious one: Design an approximation to this:

if you go on forever.

[Hint: If x is

what is

in terms of x ?]

Queue

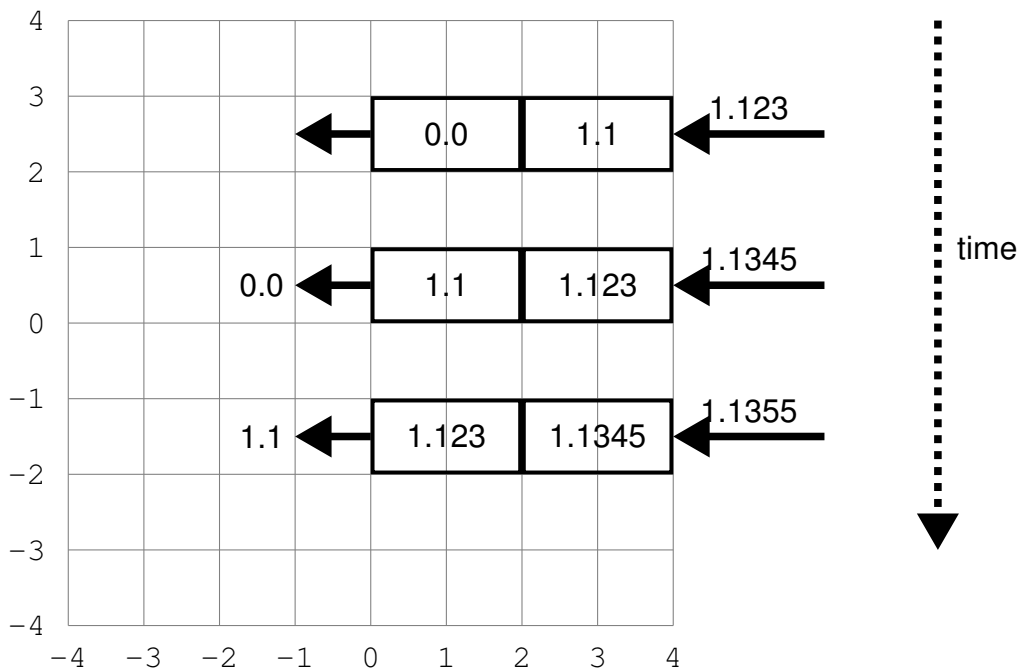
Look at our first example on approximations again. Once again we have two variables `a0` and `a1` and they scan over a bunch of values.

0.0	1.1	1.123	1.1345	1.1355	1.1356	1.1357
<i>a0</i>	<i>a1</i>					
0.0	1.1	1.123	1.1345	1.1355	1.1356	1.1357
	<i>a0</i>	<i>a1</i>				
0.0	1.1	1.123	1.1345	1.1355	1.1356	1.1357
		<i>a0</i>	<i>a1</i>			
0.0	1.1	1.123	1.1345	1.1355	1.1356	1.1357
			<i>a0</i>	<i>a1</i>		
0.0	1.1	1.123	1.1345	1.1355	1.1356	1.1357
				<i>a0</i>	<i>a1</i>	

You can also think about it this way: Think of a data structure (i.e. something that contains data and comes with operations):



and data goes in at one end and comes out from the other end.



The structure you see, as a blackbox, can keep two values. There are also operations on this structure: You can take things out of the structure and you can put things into this structure. The important thing is that the thing you take out is also the thing that has been in there the longest. Such a structure is called a **first-in-first-out (FIFO)** data structure, also known as a **queue**.

Exercises

Q1. Recall that we already have a way of determining when a positive integer is prime. Write a program that prints the following

$$n \text{ primes from 2 to } n \text{ (primes from 2 to } n) / n$$

where n runs from 1 to 100. This prints the so-called density of primes. If you make a plot of n and $(\text{primes from 2 to } n) / n$, you will see an interesting pattern.

Q2. Modify the previous program so that you only print n when n is 100, 200, 300, etc. Let n run up to 1000000.

16. Scope

OBJECTIVES

- Understand the scope of a variable
- Understand how C++ searches for a variable
- Understand the importance of using minimal scopes

I've already mentioned scopes of variables in several places. In this set of notes, I will collect what you already know about scopes. So much of it is review. I will also add a few new scope rules.

Scope

A variable has a name, a value, and you should know by now, a variable also has a **scope**. This refers to the place in your code where you can refer to that variable.

Here's an old example (very very very old ...):

```
x = 42; // x not declared yet bozo!!!

int x = 0;
std::cout << x << std::endl;
```

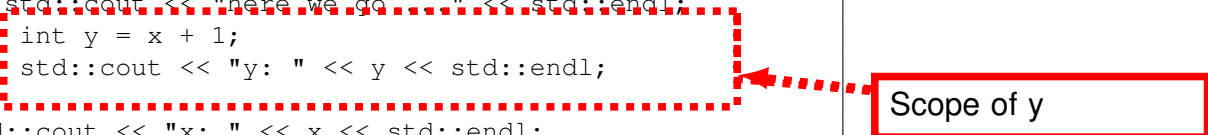
In general the scope of a variable is from the **point of declaration** to the **end of the block where it is declared**. During the execution of the program, when the point of execution exits that block, the variable is destroyed. Try this:

```
#include <iostream>

int main()
{
    int x = 0;
    std::cin >> x;

    if (x == 42)
    {
        std::cout << "here we go ..." << std::endl;
        int y = x + 1;
        std::cout << "y: " << y << std::endl;
    }
    std::cout << "x: " << x << std::endl;

    return 0;
}
```



Scope of y

Of course you should know by now that this won't work (try it):

```
#include <iostream>

int main()
{
    int x = 0;
    std::cin >> x;

    if (x == 42)
```

```
{  
    std::cout << "here we go ..." << std::endl;  
    int y = x + 1;  
    std::cout << "y: " << y << std::endl;  
}  
std::cout << "y:  " << y << std::endl;  
  
return 0;  
}
```



BAD!!!

You should visualize the variables in your code as being created in blocks of memory spaces. Suppose the user enters 42 for `x` and we are about to execute the print statement in the body of the `if` statement. The memory looks like this:

```
#include <iostream>

int main()
{
    int x = 0;
    std::cin >> x;

    if (x == 42)
    {
        std::cout << "here we go ..." << std::endl;
        int y = x + 1;
        std::cout << "y: " << y << std::endl;
    }
    std::cout << "y: " << y << std::endl;

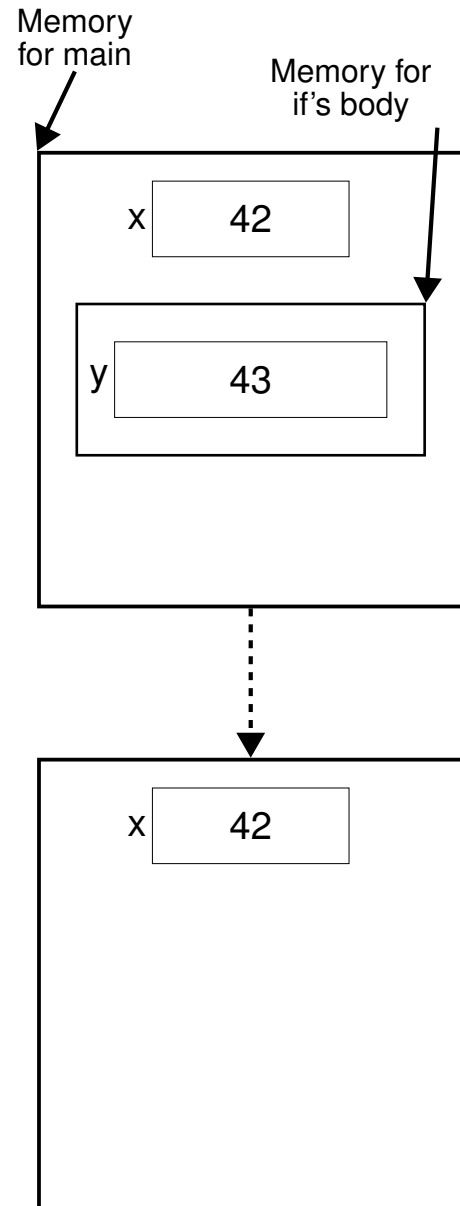
    return 0;
}
```

```
#include <iostream>

int main()
{
    int x = 0;
    std::cin >> x;

    if (x == 42)
    {
        std::cout << "here we go ..." <<
        std::endl;
        int y = x + 1;
        std::cout << "y: " << y << std::endl;
    }
    std::cout << "y: " << y << std::endl;

    return 0;
}
```



As you can see, immediately on exiting the `if` statement, the **variable in the body of the `if` statement is destroyed**.

Of course `x` was created in the `main()` block. So `x` is not destroyed.

This is one of the reasons why proper indentation is so important. Not only does it help you read the logic of your code, it tells you when variables are destroyed.

for-loops and while-loops

You should think of the `for`-loop as having a block and the body of the `for`-loop as having an inner block.

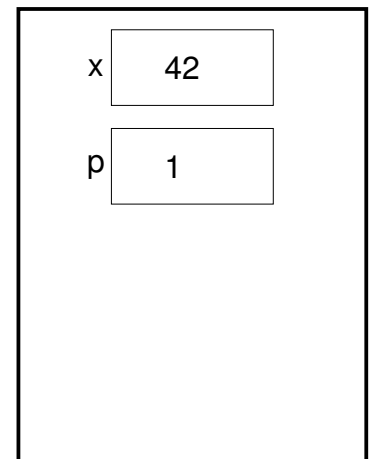
```
#include <iostream>

int main()
{
    int x = 42, p = 1;

    for (int i = 1; i <= x; ++i)
    {
        int y = i * i;
        p *= y;
    }
    std::cout << p << std::endl;

    return 0;
}
```

Memory for main ▲

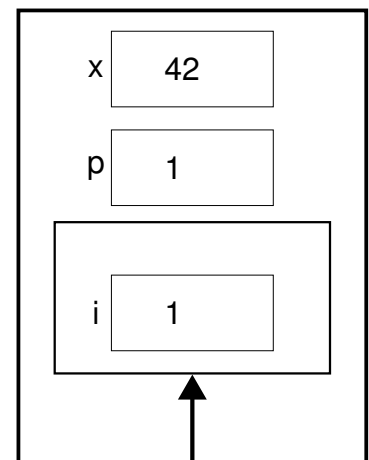


As we enter the `for`-loop, we get a block of memory for the `for`-loop. Variable `i` in the `for`-loop resides in this block.

```
#include <iostream>

int main()
{
    int x = 42, p = 1;
    for (int i = 1; i <= x; ++i)
    {
        int y = i * i;
        p *= y;
    }
    std::cout << p << std::endl;

    return 0;
}
```



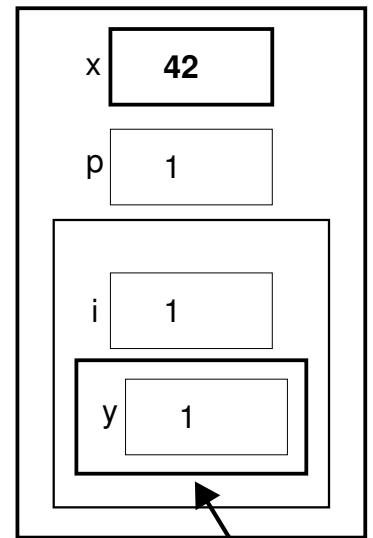
Memory of for-loop

When we step into the body of the `for`-loop, we have **another** block. Variable `y` is created in this inner block:

```
#include <iostream>

int main()
{
    int x = 42, p = 1;
    for (int i = 1; i <= x; ++i)
    {
        int y = i * i;
        p *= y;
    }
    std::cout << p << std::endl;

    return 0;
}
```



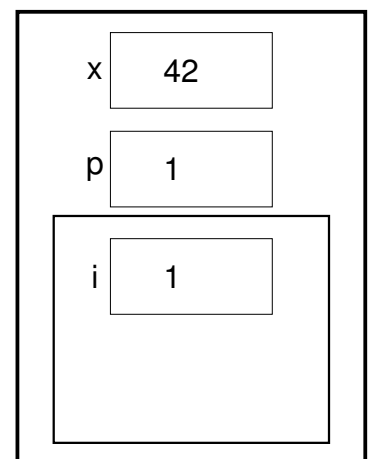
When the last statement of the body of the `for`-loop is executed, we exit the body and variable `y` is destroyed:

```
#include <iostream>

int main()
{
    int x = 42, p = 1;

    for (int i = 1; i <= x; ++i)
    {
        int y = i * i;
        p *= y;
    }
    std::cout << p << std::endl;

    return 0;
}
```



Of course when we enter the body of the `for`-loop the second time (when `i = 2`), variable `y` is created again.

Get it?

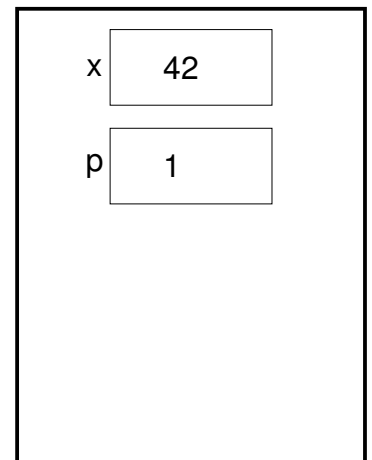
And when we exit the `for`-loop altogether, variable `i` is destroyed because it is created in the `for`-loop:

```
#include <iostream>

int main()
{
    int x = 42, p = 1;

    for (int i = 1; i <= x; ++i)
    {
        int y = i * i;
        p *= y;
    }
    std::cout << p << std::endl;

    return 0;
}
```



Note that variable `x` and `p` are still in scope; they are not destroyed.

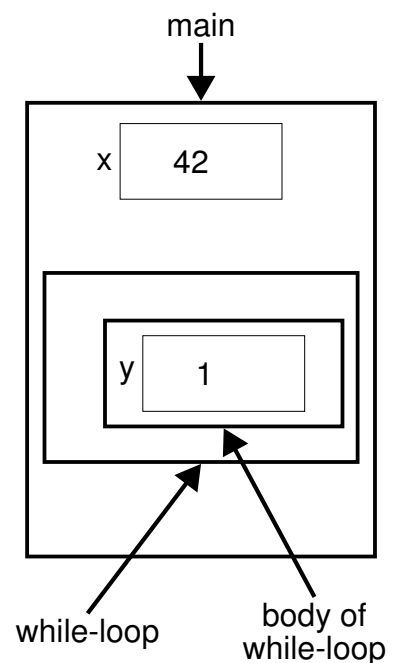
What about the `while`-loop? The idea is similar: The `while`-loop has its own memory and the body of the `while`-loop has an inner block.

```
#include <iostream>

int main()
{
    ...
    int x = 42;

    while (...)
    {
        int y = 24;
        ...
    }

    ...
}
```



So remember this: each loop has its own memory for variables and the body of the loop itself has its own memory.

Exercise -1.8.177. What is the problem? Fix it!

```
std::cout << "Compute product of squares.";
std::cout << "Enter 0 to stop.\textbackslash n";

int prod = 1;
int x;
std::cin >> x;
while (x != 0)
{
    int square = x * x;
    prod = prod * square;
    std::cout << "Product:" << prod << "\n";
    std::cin >> x;
}

std::cout << "\nProduct:" << prod << std::endl;
std::cout << "Last square: " << square
    << std::endl;
```

If you need **several** temporary variables in a `for`-loop you can do this:

```
for (int i = 0, s = 5, t = 100; i < 10; i++)
{
    ...
}
```

For instance if you want to print the running partial sums of summing 1 to 100 you can do this:

```
for (int i = 0, s = 0; i < 10; i++)
{
    s += i;
    std::cout << i << ' ' << s << '\n';
}
```

Of course note that in this case `s` is local to the `for`-loop and hence goes out of scope when the `for`-loop is done, i.e., it cannot be referenced outside the `for`-loop:

```
for (int i = 0, s = 0; i < 10; i++)
{
    s += i;
    std::cout << i << ' ' s << '\n';
}
std::cout << s << '\n'; // BAAAAAAAAAAAAAAAAAAAA!!!
```

Exercise -1.8.178. Can you also create variables in the header of an `if` statement? Can this work:

```
int i = 0;

if (int x = i * i < 10)
{
    ... do something with i and x ...
}

... do not use x!!! ...
```

Variables with the same name

Pay attention!!! This is something new!!!

You can actually create variables with the same name ... provided they are either declared in non-overlapping blocks or one is defined in an inner block.

Here's the case where names are created in **disjoint blocks**:

```
...
for (int i = 0; i < 10; ++i)
{
    int x, y, z;
    ...
}

for (int i = 0; i < 100; ++i)
{
    ...
    int y;
}

...
```

No problem! You won't get a re-declaration because the previous `i` is already destroyed!

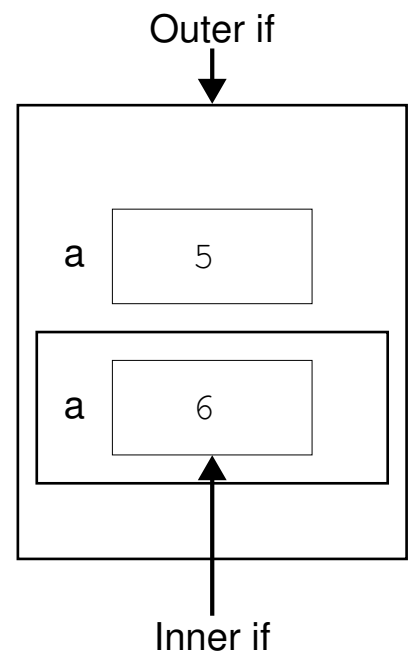
Do you see why there won't be conflicts or confusion to C++?

Here's the case where two names are created in **nested blocks**:

```
int x = 0, y = 0;
std::cin >> x >> y;

if (x == 0)
{
    int a = 5;
    std::cout << a << std::endl;

    if (y == 1)
    {
        int a = 6;
        std::cout << a << std::endl;
    }
}
```



YIKES!!!

Note that the first print statement is not ambiguous. (Why?)

But what about the second? Which `a` is used in the second print statement???... go to next section to find out ...

WARNING: Although technically you can do the above (i.e. have two variables with the same name nest within blocks, it makes the code hard to read. Not only that, some languages do not allow this at all. Therefore you should **AVOID DECLARING VARIABLES WITH THE SAME NAME IN NESTED BLOCKS**. The only **exception** is a temporary (scratch) variable for instance like the counter variable of a `for`-loop

```
for (int i = 0; i < 100; i++)
{
    ...
    for (int i = -10; i < 10; i++)
    {
        ...
    }
    ...
}
```

and only when the logic is easy to understand. But even then, you might still be shooting yourself in your foot later if you need to change the logic of your code.

Here's a simple example:

i is not found in this block. Look for i in the nearest enclosing block. FOUND!

C++ keeps searching for `i` from the current block of memory to the next enclosing block until its found.

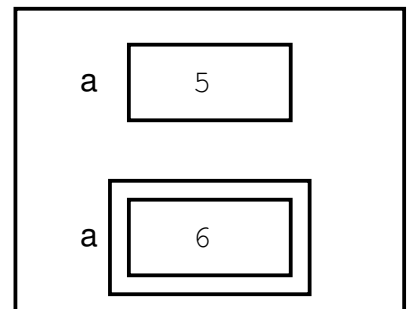
i is here

The diagram shows five concentric rectangles representing nested memory blocks. A black arrow points from the top-left corner of the outermost rectangle to the top-left corner of the second rectangle from the outside, labeled "i is here". A red dashed arrow points from the top-left corner of the innermost rectangle up to the top-left corner of the second rectangle from the outside, following the path of the black arrow.

Exercise -1.8.179. Now look at the example from the previous section. Why is C++ not confused?

```
int x = 0, y = 0;
std::cin >> x >> y;

if (x == 0)
{
    int a = 5;
    std::cout << a << std::endl;
    if (y == 1)
    {
        int a = 6;
        std::cout << a << std::endl;
    }
}
```



In this case when the variable `a` in the inner if-statement is created, we say that the innermost `a` **hides** the outer `a`.

Exercise -1.8.180. What is the output? (Or is there an error?)

```
int x = 0, y = 1;

if (x == 0)
{
    int a = 5;
    if (y == 1)
    {
        std::cout << a << std::endl;
        int a = 6;
        std::cout << a << std::endl;
    }
}
```

Exercise -1.8.181. What is the output? (Or is there an error?)


```
int x = 0, y = 1;

for (int x = 5; x < 10; x++)
{
    std::cout << x << ' ' << y << std::endl;
    int y = 5;
    while (y > 0)
    {
        int x = y + 1;
        std::cout << x << ' ' << y << std::endl;
        y++;
    }
    std::cout << x << ' ' << y << std::endl;
}
```

Exercise -1.8.182. What is the output? (Or is there an error?)

```
int i = 5, j = 6;
std::cout << i << ' ' << j << std::endl;
for (int i = 0; i < 3; i++)
{
    std::cout << i << ' ' << j << std::endl;
    if (i < j)
    {
        std::cout << i << ' ' << j << std::endl;
        int i = 24;
        std::cout << i << ' ' << j << std::endl;
    }
    else
    {
        std::cout << i << ' ' << j << std::endl;
        int j = 42;
        std::cout << i << ' ' << j << std::endl;
    }
    std::cout << i << ' ' << j << std::endl;
}
std::cout << i << ' ' << j << std::endl;
```

Exercise -1.8.183. What is the output? (Or is there an error?)

```
int i = 5, j = 6;}
std::cout << i << ' ' << j << std::endl;
for (int i = 0; i < 3; i++)
{
    int i = 8;
    std::cout << i << ' ' << j << std::endl;
    if (i < j)
    {
        std::cout << i << ' ' << j << std::endl;
        int i = 24;
        std::cout << i << ' ' << j << std::endl;
    }
    else
    {
        std::cout << i << ' ' << j << std::endl;
        int j = 42;
        std::cout << i << ' ' << j << std::endl;
    }
    std::cout << i << ' ' << j << std::endl;
    int i = 9;
}
std::cout << i << ' ' << j << std::endl;
```

Principle of minimal scope

Here's the principle of minimal scope:

**Keep the scope of variables
as small as possible.**

(The above is suppose to blink).

For instance this is NOT GOOD:

```
// compute the factorial of n
int i = 0;
int prod = 1;

int n = 0;
std::cin >> n;

for (i = 1; i <= n; i++)
{
    prod *= i;
}
std::cout << prod << std::endl;
```

This is BETTER

```
// compute the factorial of n
int prod = 1;

int n = 0;
std::cin >> n;

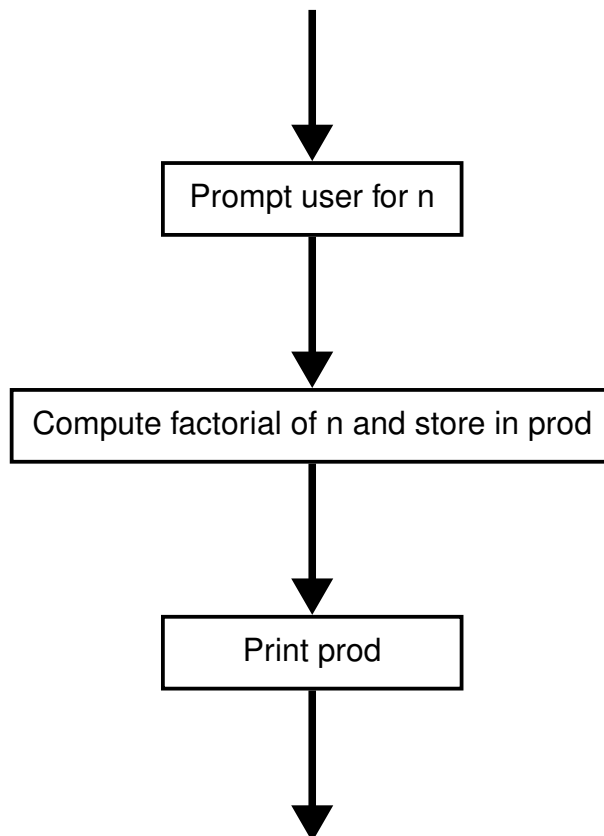
for (int i = 1; i <= n; i++)
{
    prod *= i;
}
std::cout << prod << std::endl;
```

Exercise -1.8.184. What's the difference?

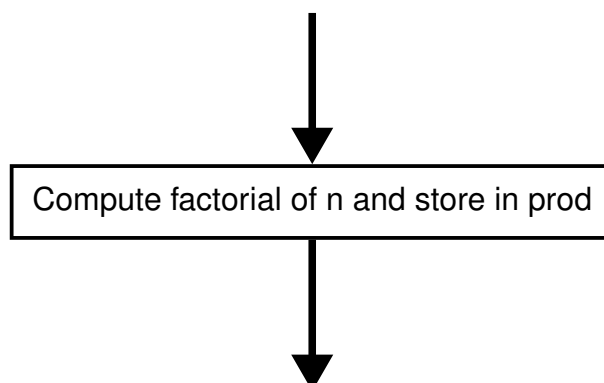
Why? Frequently when you're solving a problem (not just a programming problem), you go through a sequence of steps of computation or transformation on the data. Each step usually requires some input data to modify something (such as a variable) or to perform a task

(such as printing something to the screen in a game).

For our program above you should be able to see these steps:

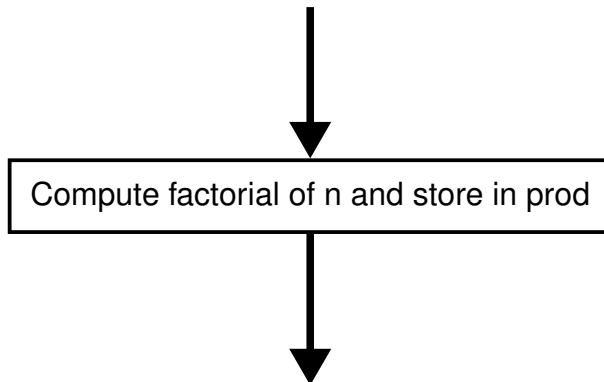


A well-written program will always try to minimize the inputs and outputs (or change) within a step such as this:



Why? Because the more stuff you have in a step the more complex it will be. That means that the code is harder to read, debug, and

maintain for future change. In the real world, software engineers write new code and maintain code base (maintain = correct, add and delete). Now look at this:



What variables are involved in this step? Don't think about the internals of this chunk of code. Think of the code as a black box. You need `n` (that's the input) and you want to compute the factorial of `n` (that's the output) and you want to put that value in to `prod` (that's the transformation).

So two variables take part in this step.

Now look at the code from the first program (the part in bold):

```
// compute the factorial of n
int i = 0;
int prod = 1;

int n = 0;
std::cin >> n;

for (i = 1; i <= n; i++)
{
    prod *= i;
}

std::cout << prod << std::endl;
```

You see clearly that besides “sending in” variables `n` and `prod` to this step, **you also “send in” variable `i`.**

Now look at the improved version:

```
// compute the factorial of n
int prod = 1;
```

```
int n = 0;
std::cin >> n;

for (int i = 1; i <= n; i++)
{
    prod *= i;
}

std::cout << prod << std::endl;
```

Variable `i` is internal to the step that computes the factorial of `n` and stores it in `prod`. Outside the code that computes and stores the factorial of `n` in `prod`, variable `i` does not exist.

Get it?

This also prevent “side effects” in code. (I’ve already mentioned this.) If a variable has a large scope, and you modify it in some place, it might have unexpected effects. Here’s something similar to a previous example – it’s important enough to repeat!!! For instance suppose work in a team and you’re given this code segment:

```
...
int i = 42;
...
for (; i < 100; i++)
{
    ...
}
....
```

where you expect the `for`-loop to run from `i = 42` to `i = 99`. One of your colleagues might see the variable `i` and add some code like so:

```
...
int i = 42;
...
for (i = 1000; i > 0; --i)
{
    ...
}
...
for (; i < 100; i++)
{
    ...
}
....
```

So guess what? When you use `i`, the initial value for you is now 0 and not 42 and your program doesn't work anymore. Tough. What's worse is when you try to debug, you might not even see it because you kept saying to yourself "But, but, but ... I didn't change anything ever since it was working!!! ... and my code looks the same as before!!!"

This is one of the most important reasons why we also try to restrict the scope of a variable. If it's meant to be a "scratch" variable, then create it in such a way that it gets destroyed ASAP.

As you can see from the example, one way to minimize scopes is to declare only when you need to and in the **innermost block** whenever possible. For our example, we declare `i` within the for-loop block.

NOTE: Some people declare all variables at the beginning of a function. That's because some programming languages specify that all declaration statements must come before other types of statements. That's not the case for C++ or other modern programming languages.

Now let's go back to that program again. There's one other spot where you can minimize the scope of a variable.

```
// compute the factorial of n
int prod = 1;

int n = 0;
std::cin >> n;

for (int i = 1; i <= n; i++)
```

```
{
    prod *= i;
}

std::cout << prod << std::endl;
```

Note that `prod` is not used until the `for`-loop. So you can write this:

```
// compute the factorial of n
int n = 0;
std::cin >> n;

int prod = 1;
for (int i = 1; i <= n; i++)
{
    prod *= i;
}
std::cout << prod << std::endl;
```

This however is **not as crucial** as the previous change because `prod` is not destroyed. Furthermore `prod` is part of the larger goal of computing the factorial. Hence it's OK to declare `prod` early so that readers of your code can see it early, especially if you mention `prod` in your comments like this:

```
// compute the factorial of n and store it in prod
int n = 0;
int prod = 1;

std::cin >> n;

for (int i = 0; i <= n; i++)
{
    prod *= i;
}

std::cout << prod << std::endl;
```

See that?

Exercise -1.8.185. Rewrite the following program so that variables have minimal scopes.


```
#include <iostream>

int main()
{
    int answer, i, first, last, sum, term;
    std::cout << "Find sum? (1 -- yes or 0 - no) ";
    std::cin >> answer;

    if (answer == 1)
    {
        std::cout << "First int:";
        std::cin >> first;
        std::cout << "Last int:";
        std::cin >> last;

        sum = 0;
        for (i = first; i <= last; ++i)
        {
            term = i * i * i * i;
            sum += term;
        }
        std::cout << "Sum:" << sum << std::endl;
    }

    return 0;
}
```

Exercise -1.8.186. The following program computes the sum

$$(1/1) + (1/1)(1/2) + (1/1)(1/2)(1/3) + (1/1)(1/2)(1/3)(1/4) + \dots$$

(that's in mathematical notation and not C++ notation) where the number of terms is specified by the user. Verify by hand that it does not work correctly. Explain why and correct the program; there are several problems.

```
int n = 0, i = 0, j = 0, term = 0, sum = 0;

std::cin >> n;

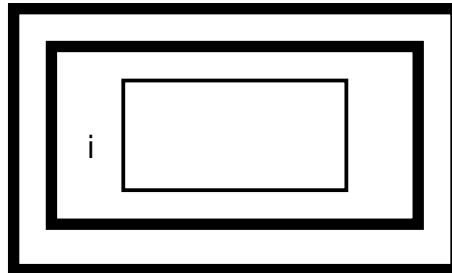
for (i = 1; i <= n; ++i)
{
    // Compute i-th term, i.e. 1/1*...*1/i
    for (j = 0; j < i; ++j)
    {
        term *= 1.0 / j;
    }

    // Accumulate term into sum by addition
    sum += term;
}

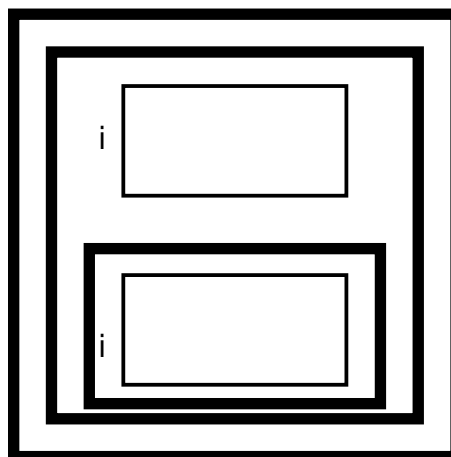
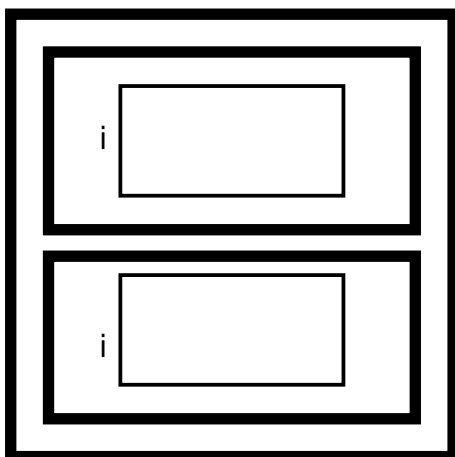
std::cout << sum << std::endl;
```

Summary The scope of a variable is where the variable can be accessed. This is from the declaration of the variable to the end of the block where it is declared.

```
...  
{  
    ...  
    double i = 0.0;  
    ...  
}  
...
```

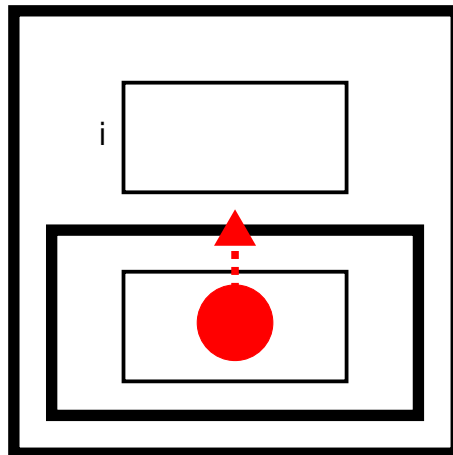


If a variable name is used in two declarations, then the two blocks where they are declared do not overlap or one is nested within the other.



If the program tries to access a variable and it's not found in the current block, then the program will look for the variable in the **nearest** outer block.

Program execution in this block and looks for `i`. Program will keep looking in the next outer block until it's found or if not, we have an error.



The `for`-loop and `while`-loop create a block and their bodies create a block within the loop's block.

17. 1D Arrays

OBJECTIVES

- Declare an array
- Declare and initialize an array with or without a size
- Access an element of an array using the `[]` operator
- Loop over the elements of an array

Subscripts

Suppose you have the following code:

```
int x = 0, y = 0, z = 0;
std::cin >> x;
std::cin >> y;
std::cin >> z;
```

Note that you're doing the *same thing* to three variables. There's a C++ concept that allows you (more or less) to say

loop through variables x , y , z and perform input for each

This sure looks like a loop.

To implement this idea, C++ uses the concept of a subscript. You have already seen this in your math classes: Suppose one of your math instructors has variables x_0 , x_1 , x_2 , and x_3 . He/she might write the following on the board: "For each x_i with $i = 0, 1, 2, 3$, we want to let x_i be 42."

It's the same idea. Your instructor wanted to do something to all the variables x_0 , x_1 , x_2 , and x_3 . But instead of saying "Set x_0 to 42", "Set x_1 to 42", "Set x_2 to 42", "Set x_3 to 42", your math instructor was smart enough to say: "For each x_i with $i = 0, 1, 2, 3$, we want to set x_i be 42". He/she is basically using subscripts.

C++ supports the same way of applying operations through a bunch of variables, and not just one.

That's the concept of an **array** of values.

Declaring arrays

We have seen many types: `int`, `double` (and `float`), `bool`, and `char`. These are called basic (or primitive) types. There are others.

Now we want to talk about creating a variable which is made up of a **collection of values** of the **same type**. In other words such a variable contains more than one value.

Try this:

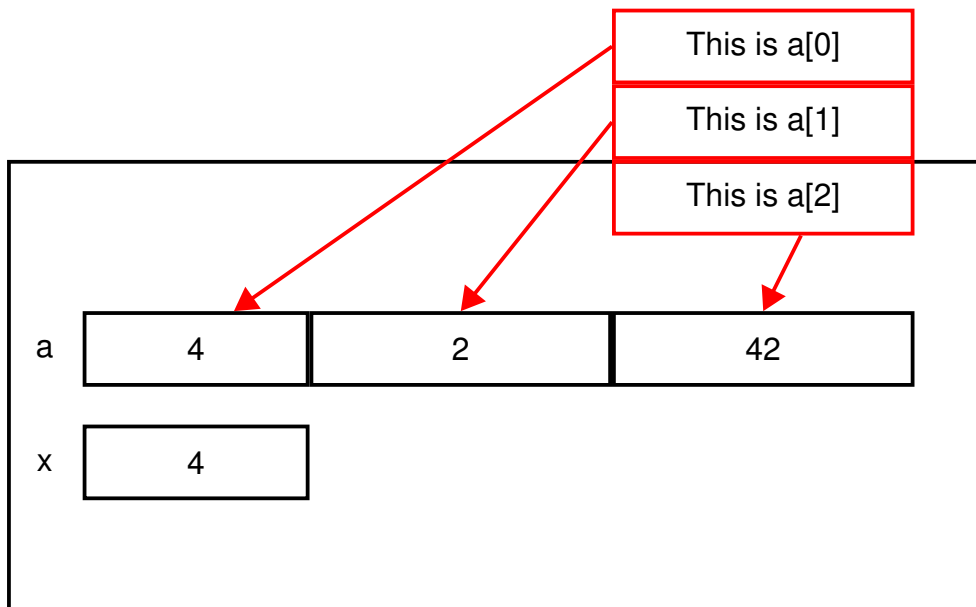
The **size** of `a` is 3

```
int a[3];  
  
a[0] = 4;  
  
a[1] = 2;  
  
a[2] = 42;  
  
std::cout << a[0] << std::endl;  
int x = a[1];  
std::cout << x << std::endl;
```

`a` is an **array** of 3
integers

the **index** of this
value in array `a` is 0.
If the size of `a` is 3,
the possible index
values are 0, 1 and
2

This is how you visualize an array. The following is a picture describing the memory of `main()`. Compare `a` to `x` in the picture.



So this is how you declare an array:

[type] [array name][[size]];

As a programmer, you can think of `a[0]`, `a[1]`, `a[2]` as integer variables inside the array variable `a`. But really `a` itself is the variable.

WATCHOUT! There are **two uses of []**. During the declaration of the array it is used to specify the **size** of the array. When accessing a value of the array, `[]` is used to specify the **index** of the value. Make sure you see the difference!

Although I'm calling `a[0]`, `a[1]`, and `a[2]` the values into array `a`, you can **think of them as integer variables**. Therefore you do know how to use them.

You might say: "Well what's the point? Can't we just use three variables such as `x`, `y`, and `z` instead of `a[0]`, `a[1]`, and `a[2]`???" Don't worry we'll come to that very soon.

Exercise -1.8.187. Add two lines of code to the above code segment.

First assign the sum of `a[0]` and `a[1]` to `a[2]`. Second print `a[2]`.

Exercise -1.8.188. Declare an array `b` of 4 doubles. Set `b[0]` to 3.1, `b[1]` to 3.14, `b[2]` to 3.141, and `b[3]` to 3.1415. Compute and display the average of all the values in `b`.

Exercise -1.8.189. Declare an array `prime` of 5 booleans. Set `prime[0]` to `true` if 0 is a prime, otherwise set `prime[0]` to `false`. Set `prime[1]` to `true` if 1 is a prime, otherwise set `prime[1]` to `false`. Etc. Print all the values in `prime` starting with the value at index 0 with consecutive values separated by a space. (You should see two 1's.)

Declaring an array with initialization

Try this:

```
int a[3] = 4, 2, 42;  
std::cout << a[0] << std::endl;  
int x = a[1];  
std::cout << x << std::endl;
```

`{4, 2, 42}`
is called an
initializer

This declares `a` as an array of 3 integers and initialize `a[0]` to 4, `a[1]` to 2, and `a[2]` to 42.

Exercise -1.8.190. What if you declare an integer array without initialization and immediately print all the values in the array. What do you think you will get? Test it out with your compiler.

You can partially initialize an integer array. Uninitialized values will be automatically initialized to zero.

```
int a[3] = {4};  
  
std::cout << a[1] << std::endl;
```

If on the other hand `a` is an array of `doubles`, then the uninitialized values are set to 0.0.

You can also declare an array **without specifying the size** if you have an initializer. In that case the size will be the number of values in the initializer:

```
double a[] = {1.1, 3.3, 5.5, 7.7};
```

In the above example `a` is an array of four doubles. In other words the above statement is the same as this:

```
double a[4] = {1.1, 3.3, 5.5, 7.7};
```

Gotchas

Here are some gotchas.

Exercise -1.8.191. Correct this statement which declares an array `x` of four doubles.

```
double[4] x;
```

Exercise -1.8.192. Does this work? Correct it!

```
double a[4];  
a = {1.1, 3.3, 5.5, 7.7};
```

Exercise -1.8.193. Does this work?

```
std::cout << "how many integers in the array? ";  
std::cin >> size;  
  
int a[size] = {0};
```

POINT: _____

Exercise -1.8.194. (Assignment operator) Does this work?

```
bool a[4] = {true, true, false};  
bool b[4];  
b = a; // trying to copy all the values in a into b
```

Correct it.

Exercise -1.8.195. (Output of an array) Does this work?

```
std::cout << a << std::endl; // trying to print all  
                             // the values in a
```

The [] operator

Here's an example from the previous section.

```
int a[3] = {4, 2, 42};
std::cout << a[0] << std::endl;

int x = a[1];

std::cout << x << std::endl;
```

Actually you can specify an index value using any **integer expression**. Try this:

```
int a[3] = {4, 2, 42};

int i = 0
std::cout << a[i] << std::endl;

int x = a[i + 1];
std::cout << x << std::endl;
```

In fact this is the reason why we have arrays ... Try this:

```
int a[3] = {4, 2, 42};

for (int i = 0; i < 3; ++i)
{
    std::cout << a[i] << std::endl;
}
```

The following code declares an array and populates it with integers entered by the user:

```
int y[5] = {0};

for (int i = 0; i < 5; ++i)
{
    std::cin >> y[i];
}
```

Compare this to this code:

```
int y0 = 0, y1 = 0, y2 = 0, y3 = 0, y4 = 0;

std::cin >> y0 >> y1 >> y2 >> y3 >> y4;
```

They more or less achieve the same thing, right? Now suppose instead of 5 variables, there are 1000 variables. The first version becomes:

```
int y[1000] = {0};

for (int i = 0; i < 1000; ++i)
{
    std::cin >> y[i];
}
```

Now let's see how much time you'd need to modify the `std::cin` statement if you have not heard of arrays:

```
int y0 = 0, y1 = 0, y2 = 0, y3 = 0, y4 = 0, ...;

std::cin >> y0 >> y1 >> y2 >> y3 >> y4 >> y5 >> ...;
```

YIKES!!!

Get the point of arrays now?

In other words previously our for-loop allows us to iteratively execute a block of statements. With the array, the for-loop allows us to execute a block of statements on one variable from a huge list of variables one at a time (if you think of an element of an array as a variable). Previously our for-loop executes a bunch of statements on a single variable (or maybe two or three). But here we apply the same computation across a bunch of variables. POWERFUL IDEA!

In the next few sections, I'll show you how to apply this principle in different scenarios.

One final thing before I show you different uses of the array. It's always a good idea not to hardcode constants, remember? The size of an array is not an exception. Therefore

```
int y[1000] = {0};
for (int i = 0; i < 1000; ++i)
{
    std::cin >> y[i];
}
```

should be

```
int y[SIZE] = {0};

for (int i = 0; i < SIZE; ++i)
{
    std::cin >> y[i];
}
```

For our example in this set of notes I will not use constants for array sizes since the code is very short. But when the program is longer (such as your assignments) you should use constants.

There's another way to avoid hardcoding the size of an array. The function `sizeof()` will tell you the mount of memory used by the computer to store the value of a variable. Try this:

```
double y = 0.0;

char z = 'a';

int a[5] = {0};

std::cout << sizeof(x) << '\n'
          << sizeof(y) << '\n'
          << sizeof(z) << '\n'
          << sizeof(a) << '\n';

std::cout << sizeof(int) << '\n'
          << sizeof(double) << '\n'
          << sizeof(char) << '\n';
```

So here's how you can determine the number of elements in an array. Try this:

```
int a[5] = {0};
double b[10] = {0};
char c[100];

std::cout << sizeof(a) / sizeof(int) << '\n';
std::cout << sizeof(b) / sizeof(double) << '\n';
std::cout << sizeof(c) / sizeof(char) << '\n';
```

Get it?

One final thing. You should NEVER use an index value outside the bound of your array. In other words if the size of your array `a` is 5, you **MUST** not access `a[5]` or `a[1000]`; you should only access `a[0]`, `a[1]`, `a[2]`, `a[3]`, and `a[4]`.

As mentioned earlier in the section on gotchas, you cannot assign arrays:

```
int a[3] = {1, 2, 3};
int b[3];
b = a;                // BAD BAD BAD!!!
```

This is what you have to do:

```
int a[3] = {1, 2, 3};
int b[3];
for (int i = 0; i < 3; ++i)
{
    b[i] = a[i];
}
```

Exercise -1.8.196. Declare an array of 10 integers; call this array `dice`. Using a for-loop, put random integers 1 to 6 into the array. Using another for-loop, print all the values in the array.

Max and min of an array of values

Exercise -1.8.197. The following program computes the maximum value in variables a, b, c, d, e:

```
double b = 1.2;
double c = 5.9;
double d = 3.14;
double e = 2.718;

double max = a;
if (max < b) max = b;
if (max < c) max = c;
if (max < d) max = d;
if (max < e) max = e;
```

We can use arrays instead:

```
double a[] = {5.4, 1.2, 5.9, 3.14, 2.718};

double max = a[0];

for (int i = 1; i < 5; ++i)
{
    if (max < a[i]) max = a[i];
    {
        std::cout << max << std::endl;
    }
}
```

If you don't see it, just unroll the for-loop.

Neat isn't it?

Of course you can also compute the minimum:

```
double a[] = {5.4, 1.2, 5.9, 3.14, 2.718};

double min = a[0];

for (int i = 1; i < 5; ++i)
{
    if (min > a[i]) min = a[i];
    {
        std::cout << min << std::endl;
    }
}
```


Sum of an array of values

You can easily compute the sum of the values in an array:

```
double a[] = {5.4, 1.2, 5.9, 3.14, 2.718};

double sum = 0.0;

for (int i = 0; i < 5; ++i)
{
    sum += a[i];
    std::cout << sum << std::endl;
}
```

Exercise -1.8.198. Modify the above to so that instead of computing the sum of the values in the array `a`, your program computes the product of the values in the array.

Exercise -1.8.199. Declare an array of 1,000,000 random doubles with values between 0.0 and 1.0 using the `rand()` function. If the `rand()` is a good random number generator, you would expect the average of the 1,000,000 to be close to 0.5. Compute the average of the values in the array and print it.

Exercise -1.8.200. Declare an array of 1,000,000 random doubles with values between 0.0 and 1.0 using the `rand()` function. Compute the standard deviation of the numbers. (The formula for the standard deviation is in an earlier assignment.)

Linear search for a value in an array

Suppose now we want to look for a value in an array and set a boolean variable to true exactly where the value is found. This is how you do it:

```
char x[] = {'h', 'e', 'l', 'l', 'o'};

char target = '';
std::cin >> target;
bool found = false;

for (int i = 0; i < 5; ++i)
{
    if (x[i] == target)
    {
        found = true;
        break;
    }
}

std::cout << found << std::endl;
```

Exercise -1.8.201. Rewrite the above so that it does not have the break statement.

You might be interested in the index of the array when the value first occurred. (And if it cannot be found, the index value is -1.) This is an example code:

```
char x[] = {'h', 'e', 'l', 'l', 'o'};

char target = '';
std::cin >> target;
int index = -1;
for (int i = 0; i < 5; ++i)
{
    if (x[i] == target)
    {
        index = i;
        break;
    }
}

std::cout << index << std::endl;
```

Exercise -1.8.202. The above code scans the array from the small-

est index value to the largest. Rewrite it so the scanning starts from the largest index value.

Exercise -1.8.203. Now write code to count the number of occurrences of a value in an array.

Note that if a value is not in an array, in the worse case scenario, you have to scan the whole array to realize that the value is not present. On the average you need to scan $n/2$ elements where n is the size of the array.

Random access

It's important to understand that there's no reason that you must scan the array from the first index to the last or from the last to the first.

You can and should think of the array as a collection of values and you have the freedom of accessing any value in the array.

This leads us to a useful idea. Recall some of the ASCII art problems. One of the difficulties is that print follows a strict direction. On the output window, your print cursor must move top-to-bottom, left-to-right.

Here's an ASCII problem. Prompt the user for two integers for variables i and j (from 0 to 9). The program prints two X's at column i and j . For instance if the user enters 0 and 3, the program prints

```
X X
```

And if the user enters 2 7 the program prints

```
X      X
```

Now your program's logic should be like this:

```
print i spaces
print 'X'
print j - i - 1 spaces
print 'X'
```

Correct? For instance when the user enters 2 and 7, we have $i = 2$ and $j = 7$. So the above pseudocode will actually execute

```
print 2 spaces
print 'X'
print 4 spaces
print 'X'
```

which does give this

```
X      X
```

But WAIT!!! What if the user enters 7 and then 2? Then the first two lines of your pseudocode executes:

```
print 7 spaces
print 'X'
...
```

But now you need to go backwards ... and you can't! The print cursor can only go forward! So you really need to sort i and j. The correct algorithm is

```
sort i, j in ascending order
print i spaces
print 'X'
print j - i - 1 spaces
print 'X'
```

But there's another way to solve this problem. You can create an array of characters, put the 'X' in the array of characters and then print the array. Why would you do that? Because you have complete control over where you want to put characters into the array!

```
declare array a of characters
set a[i] to 'X'
set a[j] to 'X'
print all the values in a
```

In C++ you have

```
std::cin >> char a[10];

for (int i = 0; i < 10; i++)
{
    a[i] = ' ';
}

int i = 0, j = 0;
std::cin >> i >> j;
a[i] = 'X';
a[j] = 'X';

for (int i = 0; i < 10; i++)
{
    std::cout << a[i];
}
```

The point here is to illustrate the fact that an array gives you random access to its values. At this point in the code:

```
a[i] = 'X';  
a[j] = 'X';
```

we don't really care if *i* is greater than *j*.

Exercise -1.8.204. Do the same problem but allow the user to input four integers. (Try to do it without arrays!)

Why arrays?

The above explains only the syntax and how to work with arrays. It also explains that if you need to work with a lot of variables, performing the same operation on these variables (such as printing their values), then you might want to use an array.

But ... probably the most important “view” of an array is that you should view it as a **storage container** of values. Why is that important?

You see containers of values everywhere. For instance when you buy something online (say at amazon.com), amazon has to retrieve information on the product. The product information is in a container – the products catalog database. At this point we only have a container (or array) of integer values or double values or boolean values or characters. Much later you will learn about containers of not just single values but aggregate values such as a container of customer data made up of firstname, lastname, email address, cellphone number, etc.

An array allows us to store computation so that a program can save time when the program runs and avoid recomputation all the time.

Read the above several times!!!

So how does an array “store computation” and “avoid recomputation”?

Remember that we already know how to check if a number is prime: A positive integer n greater than 1 is a prime if we test $n \% d$ for $d = 2, 3, 4, \dots, n - 1$. If $n \% d$ is 0, then d is a divisor and therefore n is not a prime. If $n \% d$ is not zero for all $d = 2, 3, 4, \dots, n - 1$, then n must be a prime.

The next thing I told you is that you can improve on the speed of the above algorithm by testing only $d = 2, 3, 4, \dots$ up to including the square root of n . For the case of $n = 101$, you only need to test $d = 2, 3, 4, \dots, 10$ instead of $d = 2, 3, 4, \dots, 100$. That’s a saving of 90% ...!!!

But ... there’s yet another way to speed up the program: you only need to test $n \% d$ for d running through `primes` up to the square

root of n . Therefore for $n = 101$, you only need to test $n \% d$ for $d = 2, 3, 5, 7$. So from $d = 2, 3, 4, \dots, 10$ you go down to $d = 2, 3, 5, 7$, from 9 values of d to 4. That's about 50% ...!

How would an array help?

One way is to compute an array of 1000 primes before you run your program. Let's say the array is called:

```
int prime[1000];
```

After that for every n you want to test for primality, you simply test $n \% \text{prime}[i]$ for $i = 0, 1, 2, \dots$ and stop when $\text{prime}[i]$ is greater than the square root of n . The only caveat is that this array is finite. So if you need to check for $n \% d$ when d is larger than the 1000th prime then you need to perform some tests without using the `prime` array. For instance, since the 1000th prime is 7919, and the square root of 1357931 is 1165.30, to check if 1357931 is a prime we test

```
1357931 % 2
1357931 % 3
1357931 % 5
:
1357931 % 1637
```

(the prime after 1637 is 1657 which is greater than 1165.30) and of course the primes 2, 3, 5, ..., 1637 are taken from our `prime` array. By the way 1637 is the 259th prime. Therefore there are 259 checks:

```
1357931 % 2
1357931 % 3
1357931 % 5
:
1357931 % 1637
```

Using our crude method of testing

```
1357931 % 2
1357931 % 3
1357931 % 4
1357931 % 5
:
```


1357931 % 1357930

required 1357929 checks. If we test up to the square root of 1357929:

1357931 % 2
1357931 % 3
1357931 % 4
1357931 % 5
...
1357931 % 1165

we have to perform 1164 checks. In summary:

- Test 1357931 % d for d = 2, 3, 4, ..., 1357930: 1357929 tests
- Test 1357931 % d for d = 2, 3, 4, ..., 1165: 1164 tests
- Test 1357931 % d for d = 2, 3, 5, ..., 11637: 259 tests

See the difference?

The square of 7919 is 62710561. Therefore with an array of 1000 primes, we can check if n is a prime for n up to 62710561. If n is greater than 62710561, then we need a larger `prime` array.

This is also the reason why Google works so fast. When you search for web pages containing “dog”, Google would send you the relevant URLs in a split second because Google continually scans for billions of web pages, precomputes, and stores keywords of the web pages before you even ask for “dog” related web pages. The only difference is that Google stores the results in databases and not arrays.

Bubblesort

Recall that we have the follow earlier. Suppose you have four integer variables a , b , c , d . You want to sort the values in a , b , c , d into ascending order and put them into a , b , c , d . For instance if we initially have this:

$$a = 5 \ b = 3 \ c = 6 \ d = 0$$

we want to end up with this:

$$a = 0 \ b = 3 \ c = 5 \ d = 6$$

Recall that I gave you the **bubblesort algorithm** for doing that:

Pass 1:

swap the values in a , b so that $a \leq b$
swap the values in b , c so that $b \leq c$
swap the values in c , d so that $c \leq d$
// at this point the largest value of a , b , c , d must be in d

Pass 2:

swap the values in a , b so that $a \leq b$
swap the values in b , c so that $b \leq c$
// at this point the largest value of a , b , c must be in c

Pass 3:

swap the values in a , b so that $a \leq b$
// at this point the largest value of a , b must be in b

[Don't recall any of the above? Quickly review your previous notes on bubblesort!!! Now!!!]

Of course you know what's going to happen next.

We're going to sort values in an array!!! (cheers ... clapping ... and wild whistling).

Let's do this slowly. Using the above pseudocode but with a, b, c, d replaced by a[0], a[1], a[2], a[3], we have the following:

Pass 1:

```
swap the values in a[0], a[1] so that a[0]  $\neq$  a[1]
swap the values in a[1], a[2] so that a[1]  $\neq$  a[2]
swap the values in a[2], a[3] so that a[2]  $\neq$  a[3]
// at this point the largest value of a[0], ..., a[3] must be in a[3]
```

Pass 2:

```
swap the values in a[0], a[1] so that a[0]  $\neq$  a[1]
swap the values in a[1], a[2] so that a[1]  $\neq$  a[2]
// at this point the largest value of a[0], ..., a[2] must be in a[2]
```

Pass 3:

```
swap the values in a[0], a[1] so that a[0]  $\neq$  a[1]
// at this point the largest value of a[0], a[1] must be in a[1]
```

We can rewrite Pass 1 as a for-loop:

Pass 1:

```
for i = 0, 1, 2:
    swap the values in a[i], a[i+1] so that a[i]  $\neq$  a[i+1]
```

See it? Pass 2 looks like this:

Pass 2:

```
for i = 0, 1:
    swap the values in a[i], a[i+1] so that a[i]  $\neq$  a[i+1]
```

Hmmm ... Pass 2 looks like Pass 1 ... You can combine them like this:

Pass 1 and 2:

```
for j = 2, 1:
    for i = 0, ..., j:
        swap the values in a[i], a[i+1] so that a[i]  $\neq$  a[i+1]
```

See that? What about Pass 3? Pass 3 looks like this (I'm writing this as a for-loop just to see the pattern):

Pass 3: for $i = 0$: swap the values in $a[i]$, $a[i+1]$ so that $a[i] \leq a[i+1]$

AHA! This can also be combined with Pass 1 and Pass 2:

Pass 1 and 2 and 3: for $j = 2, 1, 0$: for $i = 0, \dots, j$: swap the values in $a[i]$, $a[i+1]$ so that $a[i] \leq a[i+1]$

Note that this is for an array of size 4. What about an array of size 5? Do you see that the first value of j in the above (i.e. 2) comes from $4 - 2$. In general if you have an array with size `SIZE`, the bubblesort to sort in ascending order is this:

```
for j = SIZE - 2, ..., 1, 0:
    for i = 0, ..., j:
        swap the values in a[i], a[i+1] so that a[i] ≤ a[i+1]
```

Now we translate this to C++:

```
for (int j = SIZE - 2; j >= 0; --j)
{
    for (int i = 0; i <= j; ++i)
    {
        if (a[i] > a[i+1])
        {
            int t = a[i];
            a[i] = a[i+1];
            a[i+1] = t;
        }
    }
}
```

Make sure you understand and memorize this!!!

Exercise -1.8.205. Using the above code, create an array of 100 random integers from 0 to 1000. Sort the array in ascending order and print all the integers. Check visually that the output does give you integers in ascending order.

Exercise -1.8.206. Redo the previous exercise, sorting the array in **descending** order.

Exercise -1.8.207. Note that the bubblesort algorithm (ascending or

descending version) or course works for an array of doubles as well. Create an array of 100 random doubles from 0.0 to 1.0, sort it in ascending order and print the values of the array so that you can visually check that the values in the array is in ascending order.

Binary search

One reason for sorting an array is to improve search. Once an array is sorted in ascending order we can perform a **binary search** on the array. Let's do a simple example before we talk about code.

Suppose you have the following list of numbers:

i	0	1	2	3	4	5	6	7
a[i]	1	5	6	8	9	10	11	15

and you want to search for the number 10; 10 is our target. You can start from the leftmost and scan to the right until the number is found. Here's another way of doing it. Put your left finger at index 0 and your right finger at index 7:

i	0	1	2	3	4	5	6	7
a[i]	1	5	6	8	9	10	11	15
	left							right

Look at the middle index between left and right. This is

$$(left + right) / 2 = (0 + 7) / 2 = 7 / 2 = 3$$

(don't forget it's integer division!). So put your nose at index 3 (or one of your toes): nose = 3

i	0	1	2	3	4	5	6	7
a[i]	1	5	6	8	9	10	11	15
	left			nose				right

Now ask yourself this question:

$a[nose] = a[3] = 8$ which is less than the target (i.e. 10)

where should you look? $a[left]$, ..., $a[nose-1]$ or $a[nose+1]$, ..., $a[right]$?

Is it obvious (because the array is ascending) that the target (if present in the array) must be in $a[nose+1]$, ..., $a[right]$. Right?

So let's put our left finger at $nose + 1$, i.e. $left = nose + 1$. We now have

i	0	1	2	3	4	5	6	7
a[i]	1	5	6	8	9	10	11	15
					left			right

Let's repeat the process:

$\text{nose} = (\text{left} + \text{right}) / 2 = (4 + 7) / 2 = 11 / 2 = 5$

i	0	1	2	3	4	5	6	7
a[i]	1	5	6	8	9	10	11	15
					left	nose		right

And

$a[\text{nose}]$ is target

TARGET IS FOUND!

What about a scenario where the target is **not** found? Then at some point left and right will cross over each other. Let me explain with an example.

Say we look for target = 7. Again we start with this:

i	0	1	2	3	4	5	6	7
a[i]	1	5	6	8	9	10	11	15
		left						right

And

$\text{nose} = (\text{left} + \text{right}) / 2 = 3$

and the picture is:

i	0	1	2	3	4	5	6	7
a[i]	1	5	6	8	9	10	11	15
		left		nose				right

Since

$a[\text{nose}] > \text{target}$

we look search for target in $a[\text{left}]$, ..., $a[\text{nose}-1]$, i.e. we set

$\text{right} = \text{nose} - 1 = 3 - 1 = 2$

and the picture is now

i	0	1	2	3	4	5	6	7
a[i]	1	5	6	8	9	10	11	15
		left		right				

Again we compute the midpoint between left and right:

$$\text{nose} = (\text{left} + \text{right}) / 2 = (0 + 2) / 2 = 1$$

i.e.,

i	0	1	2	3	4	5	6	7
a[i]	1	5	6	8	9	10	11	15
		left	nose	right				

We have

$$a[\text{nose}] = a[1] = 5 \text{ is less than target}$$

So we look in $a[\text{nose}+1]$, ..., $a[\text{right}]$, i.e.

$$\text{left} = \text{nose} + 1 = 1 + 1 = 2$$

i	0	1	2	3	4	5	6	7
a[i]	1	5	6	8	9	10	11	15
			left					
				right				

We look for the mid point again:

$$\text{nose} = (\text{left} + \text{right}) / 2 = (2 + 2) / 2 = 2$$

and the picture becomes:

i	0	1	2	3	4	5	6	7
a[i]	1	5	6	8	9	10	11	15
			left					
			nose					
			right					

Since

$$a[\text{nose}] = 6 \text{ is less than the target}$$

We set

$$\text{left} = \text{nose} + 1 = 3$$

Now note that left is actually greater than right!!!

i	0	1	2	3	4	5	6	7
a[i]	1	5	6	8	9	10	11	15
				right	left			

This means that the target cannot be found.

Now I'm ready to give you the code. This will set `index` to the index where the target is found. If it is not found `index` is set to -1. The name of the array is `x` and the size of the array is `SIZE`.

```
int lower = 0;
int upper = SIZE - 1;
int mid = 0;
int index = -1;

while (lower <= upper)
{
    mid = (lower + upper) / 2;
    if (x[mid] < target)
    {
        lower = mid + 1;
    }
    else if (x[mid] > target)
    {
        upper = mid - 1;
    }
    else
    {
        index = mid;
        break;
    }
}
```

Now if you have an array of length n that is not sorted, linear searching will require on the average $n/2$ searches. The worse case requires n searches. What about binary search? In the worse case scenario, it takes $\log_2 n$. Here's a table of comparison:

n	linear search (n)	binary search ($\log_2 n$)
10	10	3.3
100	100	6.6
1000	1000	10.0
1000000	1000000	19.9
1000000000	1000000000	29.9

See the difference? Better organization and a good search algorithm implies less time in data retrieval.

[For math geeks, if you have taken Calculus you know that the gap between n and $\log_2 n$ widens. Specifically, by l'Hôpital's rule, the limit of $n/(\log_2 n)$ as n approaches infinity is in fact infinity.]

Why is the worse case $\log_2 n$? Well look at the algorithm. In each

iteration, you cut down the search space by $\frac{1}{2}$ right? Supposing the worse case, your search space reaches one value. So the sizes of the search spaces looks like this:

$$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots \rightarrow 1$$

and you go through say k iterations. In other words you have

$$n/2^k = 1$$

i.e.

$$2^k = n$$

If you take log-to-base-2 you get

$$\log_2 2^k = \log_2 n,$$

i.e., the number of iteration $k = \log_2 n$

Details on the study of algorithms and measure of what we call “time complexity” is found in CISS350 (Data Structure and Advanced algorithms) and CISS358 (Algorithmic Analysis). Deep studies in the extent of computability is found in CISS362 (Automata theory) where one is even interested in questions like “What are computers? Are there computational problems that cannot be solved by an algorithm or by a computer?”

Example: Party Planning Problem

You are planning a party for a group of friends who live on the same street:

0 1 2 3 4 5 6 7 8 9 10 11 12

Here (including yourself), you have people living at 1, 4, 4, 9, 10, 12. (Note: there are two people living at 4.) Suppose you host the party at 9. Then the total distance traveled by everyone to the house at 9 is

$$|1 - 9| + |4 - 9| + |4 - 9| + |10 - 9| + |12 - 9| = 8 + 5 + 5 + 1 + 3 = 22$$

You want to be a good host. So your goal is to find an address (i.e., an integer) so the that total distance traveled by everyone to the address is the smallest possible. If there are two address that gives the same minimal total distance, you pick the smaller address.

You may assume there are at most 100000 addresses.

Your program first accepts an integer n for the total number of addresses. It then accepts the n addresses (i.e., integers) and then prints the best address to hold the party.

For instance for the above test case, your console would look like this:

```
6
1 4 4 9 10 12
9
```

if your program thinks address 9 is the best.

WARNING: Incoming hint on next page.

Hint

There are 2 ways to solve this problem. Based on what you know, the following is the slower algorithm ...

Suppose the number of addresses is stored in n and the addresses are stored in array x .

You test every house $x[i]$. In other words for each $x[i]$, compute the total distance traveled by everyone going to address $x[i]$. You compute a running minimum of these distances.

Of course there's a loop over all the potential party address $x[i]$ for i running from 0 to $n - 1$. And for each address $x[i]$, you'll need to compute the total distance traveled to $x[i]$, i.e., you need to add up all the $x[j] - x[i]$ for j running over $0, 1, \dots, n-1$. The $x[j] - x[i]$ is the absolute value of $x[j] - x[i]$. You use the **fabs** function. I have already used this in the computation of the roots of a function.

Since you are performing a running minimum computation, you will need a **min** variable. Note that your problem does not print the **min** – you actually want the address $x[i]$. This is very similar to the algorithm for computing the maximum value of a function in the chapter on for-loops. So besides variable **min**, need also need a **min_i** where $x[\text{min_i}]$ is the desired address.

Example: Cellular automata

A **cellular automata (CA)** is simply a grid of values where the value at a point in the grid can change its value. The way such a value v changes depends on the values near v .

CAs are studied in math, CS, physics, biology, social science, etc. You name it. They can be as practical as image processing where they are used to remove noise from images to create cleaner images. Yet they can be as abstract and complex as you like – they appear in AI, dynamical systems, and chaos theory. You can find lots of information about cellular automata on the web – go ahead and check out the CA entry at wikipedia.

Let me be more specific by looking at a simple example. Suppose we look at a 1-dimensional CA with these values:

0,0,1,1,0,1,1,1

This CA is made up of 8 cells. The values are either 0 or 1. So a CA can be as simple as an array.

Now suppose the value at a cell changes according to these rules:

- If the value is 1 and together with the values on its left and right, there are three or one 1s, then the value becomes 0. Otherwise it stays as 1. In other words, if the value is 1 and either the left or right neighbor is 1 (but not both), then 1 stays as 1. Otherwise it becomes 0. You can think of it this way:
 - Companionship: If 1 has exactly one companion, he/she/it lives on. If 1 has no companion, it dies.
 - Overcrowding: If 1 has too many companions, overcrowding kills he/she/it.
- If this value is 0, and together with the values on its left and right, there are two 1s, then the value becomes 1. Otherwise it stays as 0. You can think of it this way:
 - Reproduction: If a spot is available, then the 1 on the left and right produces a 1. A reproduction (i.e., $0 \rightarrow 1$) occurs only when there are two adjacent 1s next to the 0.

For instance, the overcrowding rule gives is this:

1,1,1 $\downarrow$ 0

So applying this rule to the value at index 6 we get

```
0, 0, 1, 1, 0, 1, 1, 1
      ↓
0, 0, 1, 1, 0, 1, 0, 1
```

I'll write the rule as $111 \rightarrow 0$ instead of

```
1, 1, 1
  ↓
  0
```

Here are more examples:

- Using the rule $001 \rightarrow 0$, we get

```
0, 0, 1, 1, 0, 1, 1, 1
      ↓
_, 0, _, _, _, _, _, _
```

- Using the rule $011 \rightarrow 1$, we get

```
0, 0, 1, 1, 0, 1, 1, 1
      ↓
_, _, 1, _, _, _, _, _
```

- Using the rule $101 \rightarrow 1$, we get

```
0, 0, 1, 1, 0, 1, 1, 1
      ↓
_, _, _, _, 1, _, _, _
```

In general the new i -th value depends on the current $(i-1)$ -th, i -th, $(i+1)$ th values:

```
_, _, _, ?, _, _, _
  ↓ ↓ ↓
  ↘ ↓ ↙
_, _, _, ?, _, _, _
      ↓
```

The value at the ends (the first and last) do not have two neighbors. So we will not change the values at the left and right end points:

```
0, 0, 1, 1, 0, 1, 1, 1
↓           ↓
0, _, _, _, _, _, 1
```

(There are other ways to compute the value of the cells at end points. For instance you can view the CA as being wrapped around at the end points so that the new value for the first cell depends on the values of first, second, and last cell).

So using the above rules we get the following behavior

```
0,0,1,1,0,1,1,1
0,0,1,1,1,1,0,1
```

You can think of the above as an evolving CA that changes with time. With three times we get

0,0,1,1,0,1,1,1 (time 0)

0,0,1,1,1,1,0,1 (time 1)

0,0,1,0,0,1,1,1 (time 2)

etc.

We will use CA to generate some ASCII art by printing the values of the cells in 1D CA: If the value is 0, we print a space and if the value is 1 we print X. We will let our CA run through a certain number of time steps, printing the CA for each time step. For instance the above CA goes through three time steps:

0,0,1,1,0,1,1,1

0,0,1,1,1,1,0,1

0,0,1,0,0,1,1,1

and we print

```
+-----+
  XX  XXX
XXXX  X
  X  XXX
+-----+
```

We will use a 1D CA of size $2 * n + 1$ where n is an input from the user. We will run this for n time steps. We will start off with a CA with a 1 in the middle of the 1D array and 0 elsewhere. The set of rules to use is:

000	→	0
001	→	1
010	→	0
011	→	1
100	→	1
101	→	0
110	→	1
111	→	0

Here's the pseudocode:

```
declare an array ca of size  $2 \times 500 + 1$ .
note that the maximum size of the ca is  $2 \times 500 + 1$ .
declare an array t of size  $2 \times 500 + 1$ .
get n from user (at most 500), size of the ca is  $2 * n + 1$ .
note that we will only use ca[0], ..., ca[2*n].
set the values in ca to all 0s except for a 1 in the middle.
// time = 0
print ca (for value 1 print 'X' and for value 0 print ' '; use a loop).
for time = 1, 2, ..., n - 1:
{
    for each value in ca,
    apply above rules and fill corresponding value in t
    copy values in t back to ca (use a loop).
    print ca (for value 1 print 'X' and for value 0 print ' ').
}
```

Note that you can specify $n = 500$ for a maximum CA of size $2 * 500 + 1 = 1001$. However, you can't see the ASCII art clearly on your console window because of wraparound. You can probably right-click and choose a smaller font size and larger window size and then you can see the ASCII art up to about $n = 100$. For larger sizes, you can save the output to a document (say MS Word), choose a really tiny font and a larger page size and print it out.

NOTE: Recall that a number such as 1425 is just $1 \times 1000 + 4 \times 100 + 2 \times 10 + 5$. You can extract the 1, 4, 2, and 5 from 1425 by using integer division / and integer mod %. This is viewing an integer "written in base 10". If I give you 1,0,1 or any sequence of zeroes and ones, then it's sometimes convenient to convert that into a single integer. For instance you can convert 1,0,1 into 5 using "binary representation" – I'll explain the conversion in a bit. But first, why would you want to do that? Well,

compare this

```
int a = 1, b = 0, c = 1; // 1,0,1

if (a == 1 && b == 0 && c == 1)
{
    ...
}
```

with this:

```
int d = 5; // 1,0,1 converted to 5

if (d == 5)
{
    ...
}
```

Clearly the second code fragment is simpler because after the conversion there's only one variable and not three. Also, because you are comparing a single integer value, you can use a switch:

```
switch (d)
{
    case 5:
        ...
        break;
}
```

But what is a nice way to convert 1,0,1 to an integer? You use the same idea as base 10 representation of numbers but instead of powers of 10, you use powers of 2:

$$1,0,1 \rightarrow 1 \times 4 + 0 \times 2 + 1 \times 1$$

Here's another example:

$$10111 \rightarrow 1 \times 16 + 0 \times 8 + 1 \times 4 + 1 \times 2 + 1 \times 1$$

In the case of our CA, the three “bits” determining how to change a bit can be converted to an integer. The rules:

000	→	0
001	→	1
010	→	0
011	→	1
100	→	1
101	→	0
110	→	1
111	→	0

if you convert the 0s and 1s on the left to base 10 numbers, becomes

0	→	0
1	→	1
2	→	0
3	→	1
4	→	1
5	→	0
6	→	1
7	→	0

Exercises

Exercise -1.8.208. Declare an array of 1000 integers. Set `a[0]` to 0 and `a[1]` to 1. For the remain values in the array, set each value to the sum of the previous two values. For instance set `a[2]` to the sum of `a[0]` and `a[1]`. Set `a[3]` to the sum of `a[1]` and `a[2]`. Etc. In general set `a[i]` to the sum of `a[i-1]` and `a[i-2]` for appropriate values of `a[i]`. In other words, the value at an index position is the sum of the values at two previous positions. Print all the values in your array. Visually verify the correctness of the values. The first values are of course

0, 1, 1, 2, 3, 5, 8, 13, 21, ...

This is the famous Fibonacci sequence and appears in many areas of science including math, computer science, physics, etc.

Exercise -1.8.209. Declare an array of 100 random doubles between 0.0 and 1.0. Now set all the elements of the array with **even** index values to 0.0. Print all the values in your array. Visually verify your work.

Exercise -1.8.210. Declare an array of 20 characters. Initialize all the value of the array to the space character. In a while-loop continually print the characters separated by spaces, prompt the user for an index position and a character and set the value in the array to the character specified by the user. If the user enters -1 for the index, the program ends the while-loop and print the characters. Here is an execution:

```
index: 0
character: h

h
index: 1
character: e

h e
index: 2
character: l

h e l
index: 3
character: l
```

```
h e l l
index: 4
character: o

h e l l o
index: 6
character: w

h e l l o   w
index: 7
character: o

h e l l o   w o
index: 8
character: r

h e l l o   w o r
index: 9
character: l

h e l l o   w o r l
index: 10
character: d

h e l l o   w o r l d
index: 11
character: !

h e l l o   w o r l d !
index: -1
final: hello world!
```

Exercise -1.8.211. Write a program that generates 10 random integers between 0 and 9 (inclusive), puts these values into an array, prints the integers in the array, counts the number of even integers in the array using a for-loop, and prints the number of even integers in the array.

Exercise -1.8.212. Write a program that generates 10 random integers between 0 and 9 (inclusive), puts these values into an array, prints the integers in the array, counts the number of even integers in the array from the first index to either the last index or to the first 9 in the array, and prints the number of even integers (up to when you stop) in the array.

Exercise -1.8.213. For a tic-tac-toe game, one could use an array of 9 characters to denote the characters on the tic-tac-toe board. In other words, if we create this variable:

```
char board[9];
```

then `board[0]` is the character at the top left corner of the tic-tac-toe board, ..., and `board[8]` is the bottom right corner of the board. Rewrite our tic-tac-toe game using this variable. In fact with arrays you can write your game for any n-by-n board:

```
const int SIZE = 5;  
char board[SIZE * SIZE];
```

Here's a hint. The code for drawing the tic-tac-toe board would look something like this:

```
for (int i = 0; i < SIZE * SIZE; ++i)  
{  
    if (i % SIZE < SIZE - 1)  
    {  
        // print board[i] and '|'   
    }  
    else  
    {  
        // print board[i], newline, the row divider line  
    }  
}
```

Summary

An array is a variable containing values of the same type. The format of the statement to declare an array variable is as follows:

[type] [array name][[size]];

where [type] is the type of values you want to put into this array and [size] is the number of elements in this array. For instance to declare an array z of 10 booleans do this:

```
bool z[10];
```

You can declare with initialization. The following declares an array y of 5 ints with

```
int y[5] = {10, 11, 12, 13, 14};
```

You can omit the size:

```
int y[] = {10, 11, 12, 13, 14};
```

in that case the size is the number of initial values. You can declare and initialize partially:

```
int y[10] = {10, 11, 12, 13, 14};
```

In the case the remaining 5 are initialized to 0.

If x is an array of size 10, then you have access to `a[i]` for `i = 0, 1, 2, ..., 9`. You can (and should) think of `a[i]` as a variable. The value of `i` used to access an element of the array is called the **index** of that elements; the variable `i` is called an **index variable**. In general you can access an element with the integer value of an evaluated integer expression. In particular you can write loops to scan the elements of an array.

In particular you can write a for-loop to search for an element in the array, sum the values in an array, compute the maximum and minimum of an array, etc.

The bubblesort algorithm sorts an array. The algorithm “bubbles” the largest element to the largest index value of the array. You can rewrite the algorithm to bubble the smallest element to the smallest index value.

The binary search algorithm searches a sorted array more efficiently than a linear search.

18. Characters and C-strings

OBJECTIVES

- Declare character variables with or without initialization
- Output and input character values
- Use operators for character type
- Understand the relationship between characters and ASCII decimal values
- Declare C-string variables with or without initialization
- Use C-string functions

Containers

We're already talked about arrays (arrays of ints, arrays of doubles, etc.) Usually we use all the slots in the arrays. However an array can be used to model **containers**. What do I mean by that? For instance suppose you want to write a program to contain student IDs of current students who are at least 7 feet tall. (Not a big list I suppose ...) In fall 2009 the list might contain:

3145673, 4135778, 2356317

in spring 2009 it might be

3145673, 4135778, 2356317, 5678246

and in fall 2010 it might be

2356317, 5678246

As you can see the number of things in the list changes.

An array can be used to model a container. It can be done in two ways (at least!) You can keep an extra variable to indicate the number of things modeled by the array. For instance try this program that prompts the user for ids to be added to the id array and terminates when the user enters -1.

```
int length = 0;
int id[10];

while (1)
{
    if (length == 10)
    {
        std::cout << "oops ... I'm full!\n";
        break;
    }
    int x;
    std::cin >> x;
    if (x == -1) break;
    id[length] = x;
    length++;
    for (int i = 0; i < length; i++)
    {
        std::cout << id[i] << ' ';
    }
    std::cout << '\n';
}
```

In this case **length** is used to indicate that the actual student ids are

$$id[0], id[1], \dots, id[length-1]$$

and the rest

$$id[length], id[length+1], \dots, id[999]$$

are unused.

A second way to model a container is to use a special **sentinel** value to denote “**end-of-data**”. Try this program that does the same thing as above except that I’m using -9999 to mark the end-of-data.

```
int id[10] = {-9999};

while (1)
{
    // find length
    int length = 0;
    for (int i = 0; i < 10; i++)
    {
        if (id[i] == -9999)
        {
            length = i;
            break;
        }
    }
    if (length == 9)
    {
        std::cout << "oops ... I'm full!\n";
        break;
    }
    int x;
    std::cin >> x;
    if (x == -1) break;
    id[length] = x;
    id[length + 1] = -9999;
    // print values in container
    int i = 0;
    while (id[i] != -9999)
    {
        std::cout << id[i] << ' ';
        i++;
    }
}
```

```
std::cout << '\n';  
}
```

Note that the array begins with

-9999, ...

(the ... after the -9999) If I entered 42, the array becomes

42, -9999, ...

And if I entered 135 as the next number, the array becomes

42, 135, -9999, ...

Note that when using a sentinel to implement a container

- When adding the end of the container you always need to search for the sentinel value (more or less to compute the length of the container)
- The number of values you can put into the container is always **one less than the actual size** of the container. Why? Because the sentinel value takes up one of the spots in the array!

We'll be talking about C-strings in this set of notes. You'll see that the C-string uses the second method to describe characters in a string.

Characters

Recall that this is a character:

`'A'`

That's nothing new. To declare a character variable you do this:

`char c;`

You can of course initialize it too:

`char c = 'A';`

And of course you already know that you can output and input characters:

```
std::cout << 'A' << std::endl;
char c = 'A';
std::cout << c << std::endl;
std::cin >> c;
std::cout << c << std::endl;
```

Another thing you already know is that you can compare characters:

```
char c = 'A';
std::cout << "gimme an A: ";
std::cin >> c;
if (c == 'A')
{
    std::cout << "thanks!";
}
else
{
    std::cout << "r u the troublemaker-type?";
}
```

ASCII: Characters and integers

One important thing you should know is that the character type is actually an integer type. You can typecast between the two. Try this:

```
std::cout << (int) 'A' << std::endl;
std::cout << int('A') << std::endl;
char c = 'A';
std::cout << (int) c << std::endl;
std::cout << int(c) << std::endl;
```

As you can see the integer value corresponding to character 'A' is 65. Now try this:

```
std::cout << (char) 65 << std::endl;
std::cout << char(65) << std::endl;
```

The translation between characters and integer value is provided by the so-called ASCII table. You can google the web to read more about the ASCII standard and its history. You can also go straight to wikipedia.com and search for “ascii”. Here’s one such table. Look for 'A' and you see that the integer value corresponding to 'A' is 65 (look under the “Dec” column – “Dec” means the integer value is in base 10 **dec**imal notation.)

Exercise -1.8.214. What is the ASCII value of 'Z'? (You can of course use the web ... but it’s easier just to write a program).

Now with the ASCII value of 'Z', fill in the blank and run this program ... WAIT!!! ... can you figure out what it does BEFORE running the program?

```
for (int i = 65; i <= ____; ++i)
{
    std::cout << (char) i << std::endl;
}
```

In fact since characters can be automatically typecasted to integers you can do this too:

```
for (int i = 'A'; i <= 'Z'; ++i)
{
    std::cout << (char) i << std::endl;
}
```

Exercise -1.8.215. What is the ASCII value of 'Z' and '5'? (Use the

above ASCII table.) Is the following true or false?

`'Z' < '5'`

Now verify your guess by running this:

```
std::cout << ('Z' < '5') << std::endl;
```

YET another way to do this is to realize that there are arithmetic operators on the character type. First try this:

```
std::cout << 'A' + 1 << std::endl;
```

This tells you that 'A' is automatically typecasted to its ASCII value. Now you can do this:

```
std::cout << (char)('A' + 1) << std::endl;
```

Finally ... run this:

```
for (char c = 'A'; c <= 'Z'; ++c)
{
    std::cout << c << std::endl;
}
```

This tells us that besides == and !=, the character type has the boolean operators <, <=, >, >=. Basically these comparison operators compare the ASCII value of the character. And of course you have arithmetic operators since the character type is an integer type.

So you see ... characters are actually numbers ... at least in the computer.

Digit Character to Integer Value

Here's a neat trick.

Suppose you want to “translate” the character '0' to integer 0, character '1' to integer 1, ..., character '9' to 9. One way is to do this: Suppose `c` is a character with character '0' or '1' or ... or '9', and you want `i` which is an `int` to be 0 if `c` is '0', 1 if `c` is '1', etc. You can do this:

```
char c = ' ';
std::cin >> c;
int i = 0;
switch (c)
{
    case '0': i = 0; break;
    case '1': i = 1; break;
    case '2': i = 2; break;
    case '3': i = 3; break;
    case '4': i = 4; break;
    case '5': i = 5; break;
    case '6': i = 6; break;
    case '7': i = 7; break;
    case '8': i = 8; break;
    case '9': i = 9; break;
}
std::cout << i << std::endl;
```

Or you can also use a stack of if-else. Make sure you run this program.

That's great. It works. But it shows you're a newbie!!! Here's the smart way to do it. Notice that the ASCII table for '0', '1', ..., '9' is this:

character	ASCII integer value	value that you want
'0'	48	0
'1'	49	1
'2'	50	2
'3'	51	3
'4'	52	4
'5'	53	5
'6'	54	6
'7'	55	7
'8'	56	8
'9'	57	9

Do you see a pattern? (SPOILERS AHEAD)

The point is that the character '0', ..., '9' appear in order in the ASCII table. Look at the table again:

character	ASCII integer value	translated value
'0'	48	0 = 48 - 48
'1'	49	1 = 49 - 48
'2'	50	2 = 50 - 48
'3'	51	3 = 51 - 48
'4'	52	4 = 52 - 48
'5'	53	5 = 53 - 48
'6'	54	6 = 54 - 48
'7'	55	7 = 55 - 48
'8'	56	8 = 56 - 48
'9'	57	9 = 57 - 48

which is the same as

character	ASCII integer value	translated value
'0'	48	0 = int('0' - '0')
'1'	49	1 = int('1' - '0')
'2'	50	2 = int('2' - '0')
'3'	51	3 = int('3' - '0')
'4'	52	4 = int('4' - '0')
'5'	53	5 = int('5' - '0')
'6'	54	6 = int('6' - '0')
'7'	55	7 = int('7' - '0')
'8'	56	8 = int('8' - '0')
'9'	57	9 = int('9' - '0')

In all cases, if c is a character '0',..., '9' the integer value you want is ...

$$c - '0'$$

That's all!!! In other words, this program:

```
char c = ' ';
std::cin >> c;
int i = 0;
switch (c)
{
    case '0': i = 0; break;
    case '1': i = 1; break;
    case '2': i = 2; break;
```



```
    case '3': i = 3; break;
    case '4': i = 4; break;
    case '5': i = 5; break;
    case '6': i = 6; break;
    case '7': i = 7; break;
    case '8': i = 8; break;
    case '9': i = 9; break;
}
std::cout << i << std::endl;
```

can be **rewritten** as:

```
char c = ' ';
std::cin >> c;
int i = c - '0';
std::cout << i << std::endl;
```

I don't know about you. But I prefer the second version!!! Make sure you run this program.

Exercise -1.8.216. Write a program that prompts the user for a, b, c, ..., z (i.e. lowercase letters) and prints 0, 1, 2, ..., 25 respectively.

Exercise -1.8.217. Write a program that prompts the user for a, b, c, ..., z, A, B, C, ..., Z (i.e. lower and uppercase letters) and prints 0 for a or A, 1 for b or B, 2 for c or C, ..., 25 for z or Z respectively.

C-strings

A **C-string** is nothing more than an array of characters. The only thing different between a C-string and an array of integers (say) is that the **special escape character '0'** (the 0 is the **number** 0, not the letter O) is used to mark the **end-of-string**. This is a special character just like '\n' or '\t' because '\0', '\n', '\t' are not “visible” characters.

Try to print '\0' and you won't see a thing:

```
std::cout << '\0' << std::endl;
```

(There are in fact lots of invisible characters.)

The point of the end-of-string marker '\0', is to end some string processing. This is what I mean. Try this:

```
char spam[] = {'a', 'b', 'c', '\0', 'd', 'e'};
std::cout << spam << std::endl;
```

In this case the “processing” is the print statement:

```
std::cout << spam << std::endl;
```

The '\0' basically tells the print statement that printing should stop after printing the third character 'c'. The output looks like this:

abc

Note another thing ... **You can actually print a character array!!!** Remember that you cannot do that for an array of integers, doubles, or booleans. You have to write a for-loop in those case. As a reminder try this:

```
int x[] = {1, 2, 3, 4, 5};
std::cout << x << std::endl;
```

Let's go back to the above example:

```
char spam[] = {'a', 'b', 'c', '\0', 'd', 'e'};
std::cout << spam << std::endl;
```

Note that it does NOT mean that the size of the array is 3; it is true that three characters are printed. There are actually 6 characters in the array:

`'a', 'b', 'c', '\0', 'd', 'e'`

So you need to distinguish the difference between the **size** of the string `spam` and its **length** which is defined to be the number of characters up to, but not including, the first `'\0'`.

Informally when we think of

```
char spam[] = {'a', 'b', 'c', '\0', 'd', 'e'};
```

as a string, we say that it is made up of `'a', 'b', 'c'`. But **as a character array** is made up of six characters. Make sure you remember that.

You can think of the character `'\0'` in the array

```
char spam[] = {'a', 'b', 'c', '\0', 'd', 'e'};
```

as a sentinel value to mark the end of data for the string represented as an array. The character `'0'` is also called the **null character**. This is the reason why C-strings are also called **null-terminated strings**.

Exercise -1.8.218. What are the lengths and sizes of the following variables?

```
char ham[] = {'3', '.', '\0', '1', '4'};
char eggs[] = {'\0', 'b', '\0', 'a'};
```

You can verify your answers by counting the number of characters printed:

```
std::cout << ham << std::endl;
std::cout << eggs << std::endl;
```

There is a shorthand for `{'a', 'b', 'c', '\0'}`. Try this:

```
char spam[] = "abc"
std::cout << spam << std::endl;
```

Note in this case that `spam` has size 4 (there are 4 characters in the array) and length 3 (there are 3 characters before the first `'\0'`); `"abc"` is

`'a', 'b', 'c', '\0'`.

Exercise -1.8.219. What is the length and size of the following variable?

```
char spam[100] = "ab\0cd";
```

Exercise -1.8.220. What is the length and size of the following variable?

```
char spam[] = "ab\0cd";
```

Exercise -1.8.221. What are the length and size of the following variable?

```
char spam[100] = "";
```

Of course you can do this:

```
std::cout << "abc\0de" << std::endl;
```

You have already seen this ...

```
std::cout << "hello world" << std::endl;
```

during the first week of class. Now you know that "hello world" is an array of characters. It's just the character array made up of the following characters:

```
'h','e','l','l','o',' ','w','o','r','l','d','\0'
```

(Don't forget the '\0' !!!!)

Exercise -1.8.222. Can you do the following?

```
std::cout << {'a', 'b', 'c'} << std::endl;
```

If you have a long string you can break it up like this:

```
char foo[] = "abc"
            "def";
std::cout << foo << std::endl;
```

Basically C++ will join it up for you so that the above code is the same as this:

```
char foo[] = "abcdef";
std::cout << foo << std::endl;
```

Exercise -1.8.223. Can you do this:

```
std::cout << "abc" "def" << std::endl;
```

Can you do the same for integers?

```
std::cout << 123 456 << std::endl;
```

Exercise -1.8.224. You now know that although you cannot print an array of integers with `std::cout` (you need to write a for-loop), you can print an array of characters (it will only print up to `'\0'`). What about input? Try this:

```
char s[100];  
std::cin >> s; // type in abcde  
std::cout << s << std::endl;
```

Not too shocking right?

Exercise -1.8.225. Run the above program again and enter this as input:

```
hello world
```

What do you see?

The problem with `std::cin` is that is that `std::cin` cuts up inputs at spaces, tabs, or newline (i.e. whitespaces). So for the above exercise, C++ will only give the string "hello" to variable `s`. If you do want spaces to go into your string variable, then you need to do something else for input. Check out a later section on the `getline()` function.

The `strlen()` function

C-strings are so useful that several functions are provided with your compiler so that you can work effectively with them. Depending on your compiler, you might need to do this when you want to use these C-string functions:

```
#include <iostream>
#include <cstring>
... YOUR PROGRAM ...
```

If your compiler is really old then you might need to do this instead:

```
#include <iostream>
#include "string.h"
... YOUR PROGRAM ...
```

Here's the first function ...

The function `strlen()` returns the **length** of the string parameter. This is called the **string length function**. Try this:

```
std::cout << strlen("abc") << std::endl;
std::cout << strlen("a\0c") << std::endl;
std::cout << strlen("") << std::endl;
char s[] = "abc";
std::cout << strlen(s) << std::endl;
char t[] = "ab\0c";
std::cout << strlen(t) << std::endl;
char u[100] = "";
std::cout << strlen(u) << std::endl;
```

By the way, what if your compiler does not have the `strlen()` function? Well the code to perform the same thing as the `strlen()` function is very simple. You just loop through the character array until you see `'\0'`. You have a counter that counts the number of characters scanned until `'\0'`, not including the `'\0'` character. Here you go:

```
char x[100] = "abc";
int len = 0;
while (x[len] != '\0')
{
    len++;
}
std::cout << len << '\n';
```

Run the program. Change it to this and run it again:

```
char x[100] = "hello world";
int len = 0;
while (x[len] != '\0')
{
    len++;
}
std::cout << len << '\n';
```

There's actually a "problem" with the `strlen()` function. You see ... if a person accidentally calls the `strlen()` function with a character array that does ***not*** contain a `'\0'`, the function can actually go beyond the array bound. This might crash the program.

Here's an example:

```
char x[] = {'a', 'b', 'c'};
std::cout << strlen(x) << std::endl;
```

This is what we call a **buffer overflow**: Your string length function will access a value in the array `x` that is actually outside `x`. We'll come back to this issue later.

Other C-string functions

In fact there are MANY standard string functions that come with all C compilers. Here are some of the most basic ones. Experiment with this code and figure out what the string functions do:

```
char s[100] = "aaa";
char t[100] = "bbb";
char u[100];

// string copy function
// overwrite the string in u with the string in
// s
strcpy(u, s);
std::cout << u << std::endl;

// string concatenation function
// put the string in t to the end of the string
// in u
strcat(u, t);
std::cout << u << std::endl;

// WARNING: You cannot do strcat(u, u), i.e., the
// two arguments of the function must be different.

// string comparison function
// 0 is returned when the strings are
// the same. Otherwise a nonzero value is
// returned.
int i = strcmp(u, s);
std::cout << i << std::endl;
```

You might wonder what nonzero value is returned by `strcmp()` when the strings are different.

```
std::cout << strcmp("a", "b") << std::endl;
std::cout << strcmp("a", "c") << std::endl;
std::cout << strcmp("a", "d") << std::endl;
std::cout << strcmp("a", "z") << std::endl;
std::cout << strcmp("a", "!") << std::endl;
std::cout << strcmp("a", "ba") << std::endl;
std::cout << strcmp("b", "a") << std::endl;
std::cout << strcmp("c", "a") << std::endl;
std::cout << strcmp("d", "a") << std::endl;
std::cout << strcmp("ba", "a") << std::endl;
```

See it yet? (Hint: Check an ASCII table.)

There are other C-string functions but you will need to know a little more in order to understand how to use them.

The `strlen()`, `strncmp()`, `strncat()` functions

All the string functions we talked about run into problems if the character arrays passed into these function do not contain `'\0'` resulting in buffer overflow.

In fact this is one scenario of the so-called **buffer overflow attack**, a technique used in many virus software (malware).

Accompanying these functions are similar functions that allow you to specify a stopping point. For instance in the case of `strlen()` (the string length function) there is a function called `strnlen()`

Besides passing in a string, you also pass in a maximum possible length for the string.

```
char s[100] = ""; // s[0] = '\0';
std::cin >> s;
std::cout << strnlen(s, 99) << std::endl;
```

Note that I pass in 99 and not 100. If `s` has a **size of 100** the **maximum possible length is 99** since in this case the **last character is used by `'\0'`**. Therefore `s` (of size 100) has a maximum length of 99.

Exercise -1.8.226. What is the output of this code?

The accompanying safer functions for `strcmp()` and `strcat()` are `strncmp()` and `strncat()`. So we have the following pairs of functions:

```
strlen()   strnlen()
strcmp()   strncmp()
strcat()   strncat()
```

Try this program:

```
char s[100] = "aaa";
char t[100] = "bbb";
char u[100];

// string copy function
// In this case at most 100 chars are copied
// from s to u
strncpy(u, s, 100);
```

```
std::cout << u << std::endl;

// string concatenation function
// In this case at most 100 chars are
// concatenated from t to u
strncat(u, t, 100);
std::cout << u << std::endl;

// string comparison function
// In this case at most 100 characters are
// compared.
int i = strncmp(u, s, 100);
std::cout << i << std::endl;
```

The only problem in the above example is the call to `strncat()` function. You see if `u` already has a string of length 10, then only 90 characters are left for concatenation. If `t` has length greater than that, the program might crash. Therefore it's safe to write this:

```
// string concatenation function
// In this case at most 100 chars are
// concatenated from t to u
strncat(u, t, 100 - strlen(u, 100));
std::cout << u << std::endl;
```

But in this case some character might not be copied from `t` to `u`. Depending on what your program is supposed to do, you might want to print an error message and halt the program.

The `std::cin.getline()` function

Recall that there is a problem with input of strings ...

`std::cin` cuts up inputs at spaces, tabs, or newline (i.e. whitespaces). So if you run this code:

```
char s[100];
std::cin >> s;
std::cout << s << std::endl;
```

with this input

```
to be or not to be
```

you will find that the output is

```
to
```

In other words C++ chops up the input at spaces, tabs and newlines. So `s` is only assigned the string `"to"`.

Now ... what ever happened to the rest of the input:

```
be or not to be
```

?!? Try this:

```
char s[100];
char t[100];
char u[100];
std::cin >> s >> t >> u;
std::cout << s << '\n' << t << '\n' << u << '\n';
```

with input

```
i'm zaphod beebblebrox
```

Get it?

Exercise -1.8.227. What if you entered 5 spaces at the **beginning** of the input like this

```
i'm zaphod beebblebrox
```

What is the value of `s` in this case?

So how do you handle the problem of spaces and tabs? What if you really want spaces and tabs to go into a string? Then you have to use another function. Try running this:

```
char s[100];
std::cin.getline(s, 100);
std::cout << s << std::endl;
```

with this input:

```
to be or not to be
```

Note that the `std::cin.getline()` function also allows you to specify the size of `s`. In the above example, C++ will only read in at most 99 characters, reserving the last for `'\0'`. This prevents buffer overflow.

The `std::cin.getline()` function will assign all the characters up to but not including `'\n'` (when you press the enter key you are including a `'\n'` in your input. In other words `getline()` cuts up input at the `'\n'` character.

The `std::cin.getline()` function can actually do a lot more. You can get it to cut up input at any character you specify. For instance

```
std::cin.getline(s, 100, '\t');
```

will cut up input at `'\t'` (tab) characters. (The `'\t'` in this case is sometimes called the stop character.)

Try this

```
char s[10];
char t[10];
char u[10];
std::cin.getline(s, 10, ',');
std::cin.getline(t, 10, ',');
std::cin.getline(u, 10, ',');
std::cout << '[' << s << "]\n"
          << '[' << t << "]\n"
          << '[' << u << "]\n" ;
```

with input

```
aaa, bbb ,c c c,
```

Exercise -1.8.228. What happens when you run the above program with this input

```
0123456789,b,c,
```

`std::cin` and `std::cin.getline()`

The **extremely important thing** to remember is that `std::cin` does not work well with `std::cin.getline()`.

Exercise -1.8.229. Try to mix `std::cin` and `std::cin.getline()` like this:

```
char s[100];
int i;
std::cin >> i;
std::cout << i << std::endl;
std::cin.getline(s, 100);
std::cout << s << std::endl;
```

Run the program and see what happens.

Type conversion: C-strings to numerics

We've been using the `std::cin` for inputs and it works for input of integer values, double values, C-string values, etc. We saw above that the problem is that `std::cin` cuts up input at whitespaces. The `std::cin.getline()` does work by really taking in everything as input. But the problem is that it works for strings.

```
#include <iostream>
#include <cstdlib>

int main()
{
    char s[100] = "123";
    char t[100] = "12.3";
    int i = strtol(s, NULL, 10);
    double d = strtod(t, NULL);
    std::cout << i << '\n' << d << '\n';
}
```

The functions above are

`strtol()` string-to-long-integer
`strtod()` string-to-double

The `strtol()` converts the string to a **long int** whose largest possible value might be larger than your usual **int**.

I'll explain the `NULL` business later. Right now I just want to get the usage of the functions to you now so that you can use it.

Type conversion: numerics to C-strings

Of course you also need to know how to convert an int or a double to a C-string. There are several ways to do this.

Here's the first method in C-style (i.e. using a C function)

```
#include <cstdio> // or #include <stdio.h>

...

char c[100];
int i = 42;
double d = 3.14;

sprintf(c, "%d", i);
std::cout << c << '\n';

sprintf(c, "%f", d);
std::cout << c << '\n';
```

The second method uses C++ features:

```
#include <sstream>

...

char c[100];
int i = 123;
double d = 3.14;

std::stringstream out;

out << i;
strcpy(c, out.str().c_str());
std::cout << c << '\n';

out << d;
strcpy(c, out.str().c_str());
std::cout << c << '\n';
```


Exercises

Exercise -1.8.230. Write a program that prompts the user for a string and prints the character that occurs most frequently in the string.

Exercise -1.8.231. Write a program that prompts the user for a string and checks if the string is a palindrome. The check should ignore non-letters and the case of letters. For instance “madam, I’m Adam” is a palindrome.

Exercise -1.8.232. Write a program that prompts the user for a string, that allows user to perform a substitution of characters. For instance, if the user enters “hello world” and then request to have l replaced by r.

```
input: hello world
source: l
target: r

hello world
l->r
--rr- ---r-

source: d
target: e
hello world
l->r, d->e
--rr- ---re

source: l
target: o
l->o, d->e
hello world
--oo- ---oe

source: w
target: o
*** invalid target: o is already a target
target: i
l->o, d->e, w->i
hello world
--oo- i---oe

source: +
```

The program quits when the user enters + for source. This simple

program can be used to help you solve a type of encrypted text called substitution cipher.

Exercise -1.8.233. Write a program that prompts the user for two strings and checks if the second string occurs as a substring of the first. If so the index where the second occurs in the first is printed. Otherwise -1 is printed.

Exercise -1.8.234. Write a program that accepts a string from the user and a window column size and prints the string in the console window with the given size. For instance if the user entered “it was a dark and stormy night ...” and a size of 10, then the output is

```
it was a
dark and
stormy
night ...
```

Note that at most 10 characters from the string is printed per row. If the user entered 15 for the size then the output is

```
it was a dark
and stormy
night ...
```

Exercise -1.8.235. Write a program accepts a string that is an integer expression, interprets the expression, perform the appropriate operations and prints the results. For instance is the user enters the string “1 + 2”, the program prints 3. If the user enters “1 - 2 + 3 * 4” the program prints 11. The program should handle +, -, *, /, % and integer values.